

Analytics for Accounting Data

Sharif Islam, DBA, CPA, CMA

July 3, 2026

Table of contents

Welcome

Welcome to the book - ***Analytics for Accounting Data*** - which is slated to be published future. It will be available for purchase in both paperback and hardback, with pre-ordering available on both Amazon and online.

This is the online version of the book, which is free to use. The book is being developed. So it is recommended to use the book with caution. More chapters and materials will be added to the book gradually.

If you want me to add some materials that are necessary for the students, please do not hesitate to ask. All kinds of comments, recommendations, suggestions, criticisms are welcome.

Warning

This book is a work in progress.

Preface

About the Book

The book is written for the students in undergraduate and graduate programs.

About the Author

Sharif Islam, DBA, CPA, CMA is an Associate professor in School of Accountancy in Southern Illinois University Carbondale (SIUC). He is a licensed CPA in Illinois and a Certified Management Accountant (CMA). He teaches Advanced Cost Accounting, Auditing, Accounting Information Systems, Machine Learning, and Analytics for Accounting Data. He earned his doctorate from Louisiana Tech University. His manuscripts are selected for “Best Research Paper Award” in several conferences of American Accounting Association (AAA). His research also got 2024 “Notable Contribution to the Literature Award” by AIS section of AAA. He published research in *Accounting Horizons*, *Journal of Accounting and Public Policy*, *Journal of Information Systems*, *Advances in Accounting*, *Journal of Emerging Technologies in Accounting*, *Issues in Accounting Education*, and *Managerial Auditing Journal*. His research interests lie at the intersection of Accounting and Data Science.

How to Read the Book

Acknowledgment

To prepare the book, I took help from many sources on the internet and published materials. Many of them are cited in the book. I acknowledge the contribution of all of those resources that help me to prepare the book for the students.

Part I

Introduction to Accounting Analytics

1 Overview of Accounting Analytics

The world's most valuable resource is no longer oil, but data (The Economist 2017).

AI is ... also reshaping the skillset required in the accounting profession. Today's accountants need to be tech-savvy, with a keen understanding of how AI tools work and how they can be applied in various accounting scenarios (Forbes 2024).

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Gain an understanding about the uses and importance of data analytics in organizations
- Gain an understanding about the uses and importance of data analytics in Accounting
- Identify different characteristics of big data
- Know how data analytics are impacting different disciplines of accounting
- Know the types of Data Analytics
- Know the Data Analytics Process

1.1 Data Analytics & Big Data

Given the availability of vast amount of data, companies in numerous industries exploit such data for competitive advantage, aiming to either increase revenues or decrease costs. Data Driven Decisions (DDD) are making significant differences in productivity, on Return on Assets (ROA), Return on Equity (ROE), asset utilization, and on market value (Provost and Fawcett 2013). Firms using data analytics in their operations can outperform their competitors by 5% in productivity and 6% in profitability (Barton and Court 2012). In 2017, 53% companies have adopted big data, as compared to only 17% in 2015 (Columbus 2017). Additionally, regulators are increasingly calling for organizations to use analytics

(Protiviti 2017). This evolving landscape in industry emphasizes the significance of data analytics in organizations.

Analytics is a means of extracting value from data. Analytics is the assessment of data with technology tools. Today, there are more powerful analytics tools to more efficiently and effectively analyze a broader range of data and types of data than in the past. Thus, there is an increased opportunity for enhanced insights about what stories data can tell to address business issues and transform the way decisions are made.

The meaning of (big) data analytics varies across different disciplines and there is substantive confusion between the slightly differing characterizations of “big data,” “business intelligence,” and “data analytics” (Vasarhelyi et al. 2015). Though many people consider big data in terms of quantities, it is also related to large-scale analysis of large amounts of data to generate insights and knowledge (Verver 2015). Big data is characterized by four Vs: Volume; Velocity; Variety; and Veracity. Volume refers to the size of the dataset, velocity to the speed of data generation, variety to the multiplicity of data sources, and veracity to the elimination of noise and obtaining truthful information from big data. Sometimes big data are characterized by six Vs: Volume, Velocity, Variety, Veracity, Variability, and Value; or, even seven Vs: Volume, Velocity, Variety, Veracity, Variability, Value, and Visualization (Sivarajah et al. 2017). Some people also identify sometimes eight Vs for big data. Figure ?? and Figure ?? depict the six and eight Vs of big data respectively.

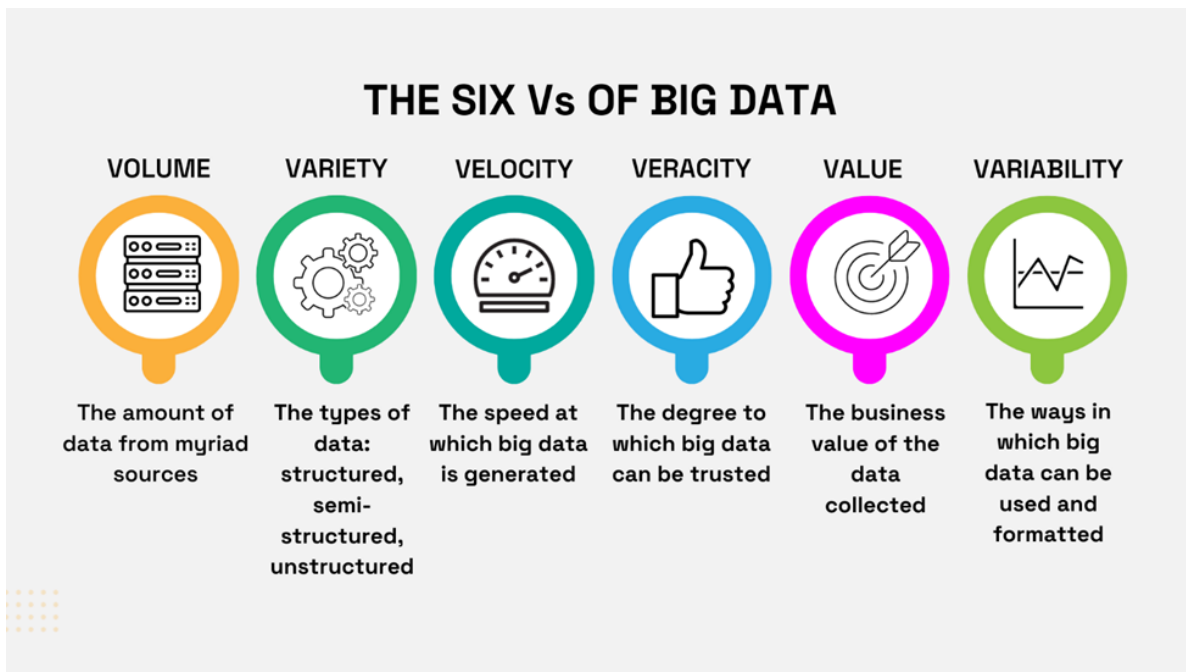


Figure 1.1: Six Vs of Big Data

Data analytics is defined as “the art and science of discovering and analyzing patterns, identifying anomalies, and extracting other useful information in data underlying or related to the subject matter

of an audit through analysis, modeling, and visualization for the purpose of planning or performing the audit” (American Institute of Certified Public Accountants (AICPA) 2015, 105). Cao et al. (2015) define big data analytics as the process of inspecting, cleaning, transforming, and modeling big data to discover and communicate useful information and patterns, suggest conclusions, and to provide support for decision-making.

Data analytics promises significant potential in auditing. Therefore, in accounting, sometimes data analytics becomes synonymous with audit analytics. Audit analytics involves the application of data analytics in the audit. Specifically, American Institute of Certified Public Accountants (AICPA) (2017) defines audit data analytics as “the science and art of discovering and analyzing patterns, identifying anomalies and extracting other useful information in data underlying or related to the subject matter of an audit through analysis, modeling and visualization for the purpose of planning or performing the audit.” In other words, audit data analytics are techniques that can be used to perform a number of audit procedures such as risk assessment, tests of details, and substantive analytical procedure to gather audit evidence. The benefits of using audit data analytics include improved understanding of an entity’s operations and associated risk including the risk of fraud, increased potential for detecting material misstatements, and improved communications with those charged with governance of audited entities.

1.1.1 Big Data Spectrum

The data dynamics and the way businesses are using data are relatively new to the world of business and something that accounting and business students must become more familiar with. Figure ?? depicts the big data spectrum. You’ll notice that the data in yellow at the left bottom portion of the spectrum has less of the four V’s. This typically is data that is sourced from enterprise resource planning (ERP) systems. This is also the type of data that a business analyst, especially an accountant, most often works with.

Continuing along the trajectory, you’ll see data generated from operating systems, such as call center records, email, voicemail, etc. From there, you’ll see data that comes from the web, including shopping cart information, web logs, browser history, promotion information, etc. This then moves beyond to the “biggest” category of data, which includes videos, radio-frequency identification (RFID), global positioning system (GPS) coordinates, social media feeds and more.

1.2 Importance of Data Analytics in Accounting

Data analytics is important for accounting profession because data gathering and analytics technologies have the potential to fundamentally change accounting and auditing task processes (Schneider et al. 2015). Scholars note that the emergence of data analytics will significantly change the infer/predict/assure (e.g., insight/foresight/oversight) tasks performed by accountants and auditors. Big data and analytics have increasingly important implications for accounting and will provide the means



Figure 1.2: Eight Vs of Big Data

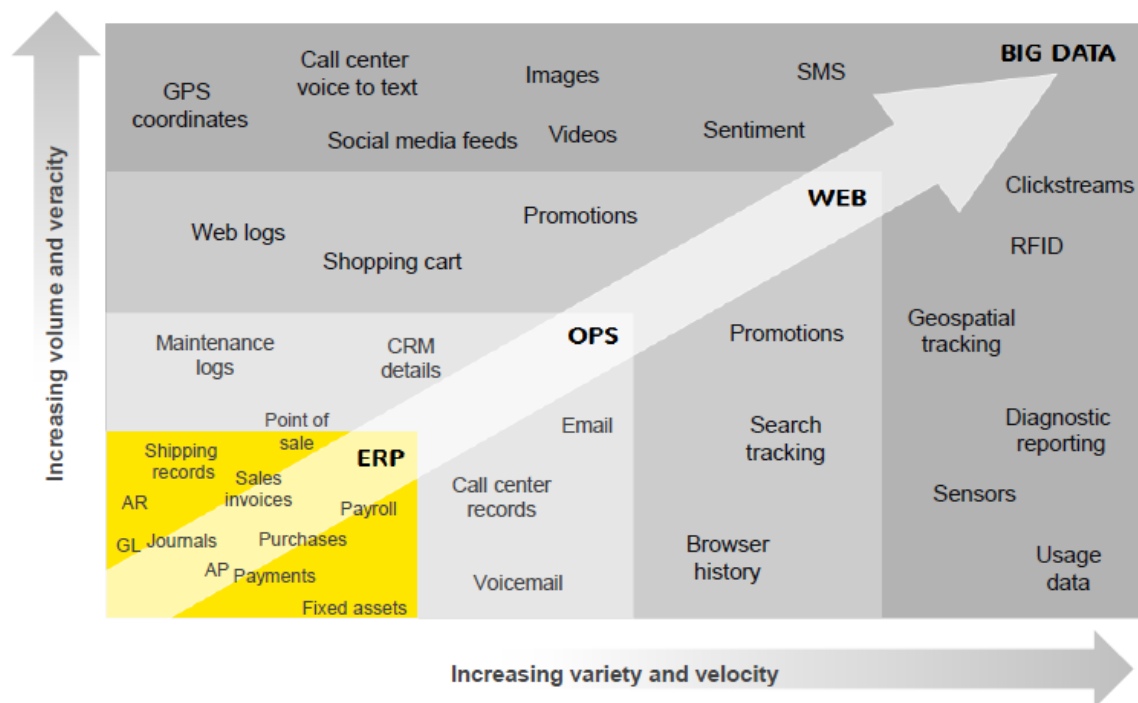


Figure 1.3: Big Data Spectrum

to improve managerial accounting, financial accounting, and financial reporting practices (Warren et al. 2015). It is further suggested that big data offers an unprecedented potential for diverse, voluminous datasets and sophisticated analyses. Research indicates that big data has great potential to produce better forecast estimates, going concern calculations, fraud, and other variables that are of concern to both internal and external auditors (Alles 2015). Moreover, auditors might reduce audit costs and enhance profitability and effectiveness by means of big data or data analytics. Sixty-six percent of internal audit departments currently utilize some form of data analytics as part of the audit process (Protiviti 2017).

1.2.1 Data Analytics in Financial Accounting

Warren et al. (2015) note that “in financial accounting, big data will improve the quality and relevance of accounting information, thereby enhancing transparency and stakeholder decision-making. In reporting, big data can assist with the creation and refinement of accounting standards, helping to ensure that the accounting profession will continue to provide useful information as the dynamic, real-time, global economy evolves.” In particular, they suggest that big data could significantly impact the future of financial accounting and Generally Accepted Accounting Principles (GAAP). Big data can also help to supplement financial statement disclosures by accumulating, processing, and analyzing information about a given intangible of interest. Furthermore, big data or data analytics can help in narrowing the differences between accounting standards such as US GAAP and International Financial Reporting Standards (IFRS) and facilitate different measurement processes such as Fair Value Accounting (FVA) by analyzing different kinds of unstructured data (Warren et al. 2015).

Crawley and Wahlen (2014) noted that data analytics allows researchers to explore a large amount of qualitative information disclosed by organizations, and examines the consequences of such disclosures. Moreover, data analytics now provides the opportunity to judge the informational content of qualitative financial information. For example, Davis et al. (2012) found that the extent of optimism expressed in firms’ earnings announcements is positively associated with Return on Assets (ROA) and stock reactions. By the same token, Li (2010) suggested that the tone of forward-looking statements is positively associated with future earnings performance. In addition, Feldman et al. (2010) found that changes in disclosure tone is indicative of future changes in earnings. Interestingly, research shows that even information on social media such as Twitter can predict stock market responses (Bollen et al. 2011).

Data analytics helps to relate textual data to earnings quality. For example, firms having more complicated and less transparent financial statement disclosures are more likely to have poor quality earnings, less persistent positive earnings and more persistent negative earnings (Li 2008). Li et al. (2013) confirmed that firms discussing their competition frequently have ROAs that mean returns more severely than the firms discussing the competition infrequently.

With the help of textual data analytics, researchers recently documented the role that qualitative disclosures have in forming the information environment of organizations; such information environments include factors such as the number of analyst following a firm, characteristics of its investors, its trading activities, and the litigation it is involved with. Less readable 10-Ks are associated with greater number of analysts following the firm and a greater amount of effort needed to generate report about

it (Lehavy et al. 2011). They also find that less readable 10-Ks are associated with greater dispersion, lower accuracy, and greater uncertainty in analyst's earnings forecasts about a given firm.

Summary Point

Data analytics in Financial Accounting - A. has potential to enhance quality and relevance of accounting information, B. can supplemental financial statement disclosures, C. can facilitate different measurement processes, and D. allows to explore a large amount of qualitative information

1.2.2 Data Analytics in Management Accounting

Warren et al. (2015, 397) noted that “in managerial accounting, big data will contribute to the development and evolution of effective management control systems and budgeting processes. In particular, they elaborate on how big data or data analytics can play a role in management control systems by discovering behaviors that have correlation with specific goal outcomes. Essentially, big data analytics can locate new kinds of behaviors that might impact goal outcomes by simplifying the identification of important motivational measurement tools linked to organizational goals. Moreover, by analyzing non-structured data, big data analytics can help discern employee morale, productivity, and customer satisfaction. Data analytics can also be used to improve “beyond budgeting practices” since traditional budgeting sometimes creates barriers to creativity and flexibility (Warren et al. 2015).

Richins et al. (2017) suggest that big data analytics could improve customer service quality. They suggest that most of the time organizations use structured data that are in their records to evaluate customer service quality; however, this approach does not take into account the customer perspective. Big data analytics allow organizations to evaluate this customer perspective by using unstructured data from social media or e-commerce sites, thus permitting organizations to have a holistic view of customer service quality.

Managers recognize that financial measures, alone, are insufficient to forecast future financial success or to use for performance management. Big data analytics provides opportunities to incorporate non-financial measures by incorporating unstructured data (Richins et al. 2017). Using big data analytics (particularly the analysis of unstructured data) accountants can identify the causes of underlying problems, understand ramifications, and develop plans to mitigate adverse impacts (Richins et al. 2017). Data analytics can also provide accountants with additional tools to monitor operations and product quality, discover opportunities to reduce costs, and contribute to decision-making (Dai and Vasarhelyi 2016).

Summary Point

Data analytics in Management Accounting - A. will contribute to the development of effective management control systems and budgeting processes, B. can enhance employee morale, productivity, and customer satisfaction, C. can enhance customer service quality by evaluating customer

perspectives using unstructured data from social media or e-commerce, D. creates opportunities to incorporate non-financial measures by incorporating unstructured data, and E. provide accountants with additional tools to monitor operations and product quality, and discover opportunities to reduce costs.

1.2.3 Data Analytics in Auditing

Data analytics has the potential to improve the effectiveness of auditing by providing new forms of audit evidence. Data analytics can be used in both auditing planning and in audit procedures, helping auditors to identify and assess risk by analyzing large volumes of data. Even organizations that have very immature capabilities indicate that a strong level of value is derived from including analytics in the audit process (Protiviti 2017).

Big data is being seen by practitioners as an essential part of assurance services (Alles and Gray 2016), but its application in auditing is not as straightforward as it is in marketing and medical research. Appelbaum (2016) and Cao et al. (2015) identified several areas that are likely to benefit from the use of big data analytics. Some of the areas are:

- At the engagement phase – supplementing auditors’ industry and client knowledge
- At the planning phase – supplementing auditors’ risk assessment process
- At the substantive test phase – verifying the management assertions
- At the review phase – advanced data analytical tools as analytical procedures
- At the continuous auditing phase – enhancing knowledge about the clients

Yoon et al. (2015) suggest that big data create great opportunities through providing audit evidence. They focused on the “sufficiency” and “appropriate” criteria and noted that though there are some issues about the propriety of big data due to different kinds of “noise,” big data can be used as complementary audit evidence. Additionally, they discussed how big data can be integrated with traditional audit evidence in order to add value in the process. Big data or data analytics can also help auditors to test the existence of assertions (e.g. fixed assets) using non-conventional data such as video recording (Warren et al. 2015). In the world of big data, potential types and sources of audit evidence have changed (Appelbaum 2016). For this reason, Krahel and Titera (2015) suggest that big data might change the focus of auditors, shifting emphasis from management to the verification of data.

Data quality and reliability or verifiability have become important issues in auditors’ evaluations of audit evidence. In this way, big data can be used as part of analytical procedures, which are required at the planning and review phase, but which are optional at the substantive procedure phase. However, many issues remain unresolved about how to use big data since analytical procedures and auditing standards are not very specific about the selection of analytical audit procedures; the choice depends on the professional judgment of auditors (Appelbaum et al. 2017). For this reason, auditors need to exercise increased professional skepticism in the big data era because in many cases sources of big data

lack provenance and, subsequently, veracity, and sometimes auditors (particularly internal auditors) have little or no involvement in data quality evaluation of such sources (Appelbaum 2016). Considering the prediction that analytics will spell the demise of auditing, Richins et al. (2017) suggest that auditors in the big data era are still essential because they know “the language of business.” Particularly, they suggest that big data analytics cannot replace the professional judgment used by auditors, suggesting that analytics will instead complement auditors’ professional judgment.

Alles and Gray (2016) identify four potential advantages of incorporating big data into audit practices: strong predictive power to set expectations for financial statement audits, great opportunities to identify potential fraudulent activities, increased probabilities of discovering red flags, and the possibility of developing more predictive models for going concern assumptions. To that end, internal audit groups with dedicated analytics functions and organizations that have attained a managed or optimized to the state of analytics maturity are far more likely to conduct continuous auditing (Protiviti 2017). Though big data creates many opportunities for improving auditing, it also suffers from different shortcomings that hinder its application in Continuous Auditing (CA). For example, Zhang et al. (2015) suggest big data characteristics such as volume, velocity, variety, and veracity creates problems in its application in CA through different gaps such as data consistency, data integrity, data identification, data aggregation, and data confidentiality.

Rose et al. (2017) found that the timing of the introduction of data analytics tools into the audit process affects the evaluation of evidence and professional judgment. Barr-Pulliam et al. (2023) found that jurors consider auditors more negligent when they use traditional auditing technique rather than audit data analytics techniques. Additionally, they confirmed that audit data analytics tools increase the perceptions of audit quality. Schneider et al. (2015) suggest that data analytics can be used by auditors to evaluate the internal control effectiveness and policy compliance. They further suggest that by analyzing unusual data flows, unexpected large volumes of data, high frequency transactions, or duplicate vendor payments, auditors can better detect fraud.

1.2.4 Data Analytics in Tax Accounting

Traditionally, tax analytics has focused on *hindsight*, particularly dealing with data from transactions that have already occurred. However, recently tax organizations are looking to use data more for gaining *insight* and sometimes for *foresight*. Analytics can help to move tax toward insight and foresight, thus changing the mindset from “what do I need to do?” to “what do I need to know?” (Deloitte 2016).

Data analytics can help an organization and its tax function drive toward becoming an insight-driven organization, or IDO (Deloitte 2016). Various types of analytics can be applied to tax issues. Organizations have used tax analytics mostly in creating descriptive scorecards and visualizations (*hindsight*). These kinds of tax analytics help to determine where to allocate resources, focus on anomalies in results, and identify potential areas of risk.

1.3 Types of Data Analytics

Data analytics can be classified in many ways, but usually there are four types of data analytics. Figure ?? depicts the types -

- Descriptive Analytics (Business Intelligence & Data Mining)
- Diagnostic Analytics
- Predictive Analytics (Forecasting)
- Prescriptive Analytics (Optimization & Simulation)

1.3.1 Descriptive Analytics

Descriptive analytics looks at data and analyze past event for insight as to how to approach future events. It looks at past performance and understands the performance by mining historical data to understand the cause of success or failure in the past. Almost all management reporting such as sales, marketing, operations, and finance uses this type of analysis. Common types of descriptive analytics are -

- Data Queries
- Reports
- Descriptive Statistics
- Data Dashboard

1.3.2 Diagnostic Analytics

In this analysis, we generally use historical data over other data to answer any question or for the solution of any problem. We try to find any dependency and pattern in the historical data of the particular problem. For example, companies go for this analysis because it gives a great insight into a problem, and they also keep detailed information about their disposal otherwise data collection may turn out individual for every problem and it will be very time-consuming. Common techniques used for Diagnostic Analytics are:

- Data Discovery
- Data Mining
- Correlations

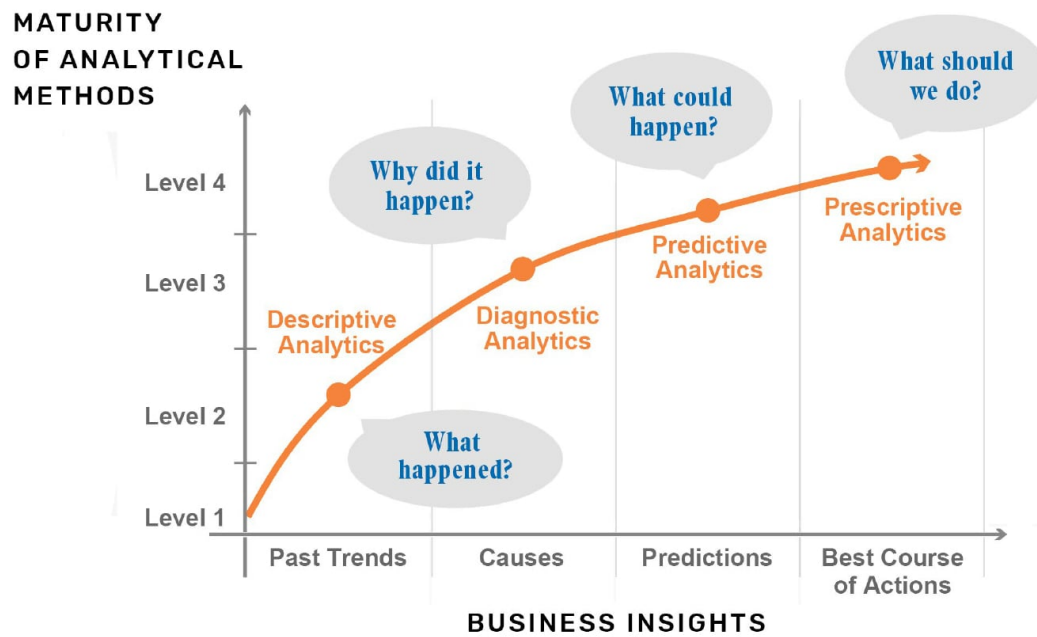


Figure 1.4: Types of Data Analytics

1.3.3 Predictive Analytics

Predictive analytics turn the data into valuable, actionable information. predictive analytics uses data to determine the probable outcome of an event or a likelihood of a situation occurring. Predictive analytics holds a variety of statistical techniques from modeling, machine learning, data mining, and game theory that analyze current and historical facts to make predictions about a future event. Techniques that are used for predictive analytics are:

- Linear Regression
- Time Series Analysis and Forecasting
- Data Mining

1.3.4 Prescriptive Analytics

Prescriptive Analytics automatically synthesize big data, mathematical science, business rule, and machine learning to make a prediction and then suggests a decision option to take advantage of the prediction. Prescriptive analytics goes beyond predicting future outcomes by also suggesting action benefits from the predictions and showing the decision maker the implication of each decision option. Prescriptive Analytics not only anticipates what will happen and when to happen but also why it will happen. Further, Prescriptive Analytics can suggest decision options on how to take advantage of a future opportunity or mitigate a future risk and illustrate the implication of each decision option.

1.4 Data Analytics Processes

As in in any scientific discipline, data analytics involves a rigorous step-by-step process. Each step demands different skills and know-how. However, to realize the full potential of data analytics, understanding the whole process is important. Figure ?? delineates the whole process.

1.4.1 Defining the Questions

The first step in any data analysis process is to define your objective. In data analytics jargon, this is sometimes called the ‘problem statement’. Defining your objective means coming up with a hypothesis and figuring how to test it. For instance, your organization’s senior management might pose an issue, such as: “Why are we losing customers?” It’s possible, though, that this doesn’t get to the core of the problem. A data analyst’s job is to understand the business and its goals in enough depth that they can frame the problem the right way.

1.4.2 Collecting the Data

1.4.3 Cleaning the Data

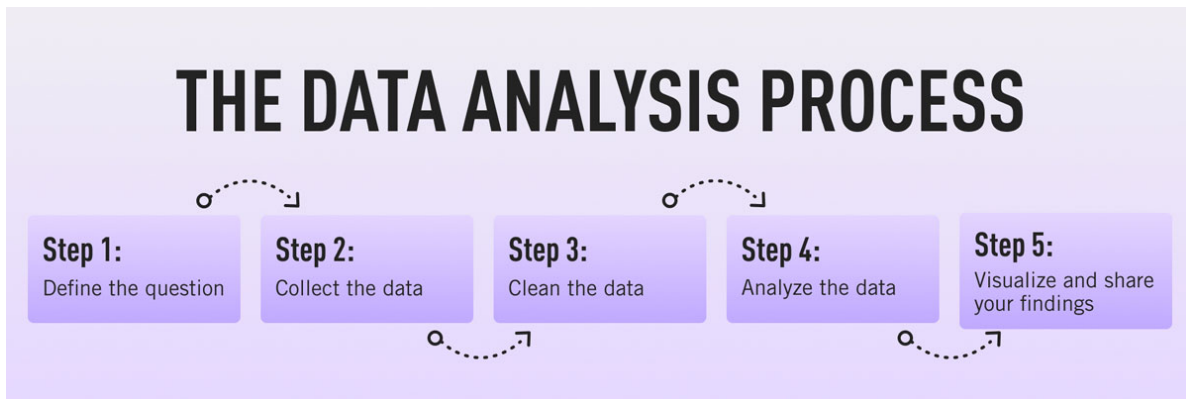


Figure 1.5: Data Analytics Process

1.4.4 Analyzing the Data

1.4.5 Sharing Your Results

1.5 Analytics Mindset

Having an analytics mindset is important while performing data analytics processes. An analytics mindset is the ability to:

- Ask the right questions
- Extract, transform and load relevant data (i.e., the ETL process)
- Apply appropriate data analytics techniques

- Interpret and share the results with stakeholders

1.5.1 Ask the Right Questions

To drive better decisions, one must **ask the right questions** first and then seek answers in the data. Then one looks to find relevant data and the appropriate data sources to perform the analytics. These analytics will provide the insights and answer the business questions being asked, which then drive the decisions. Throughout the process, there is a continuous feedback loop that makes the process iterative. An analytics mindset keeps asking questions until the right answers emerge. The right questions are those that lead to the right answers. Figure ?? depicts how analytics mindset works.

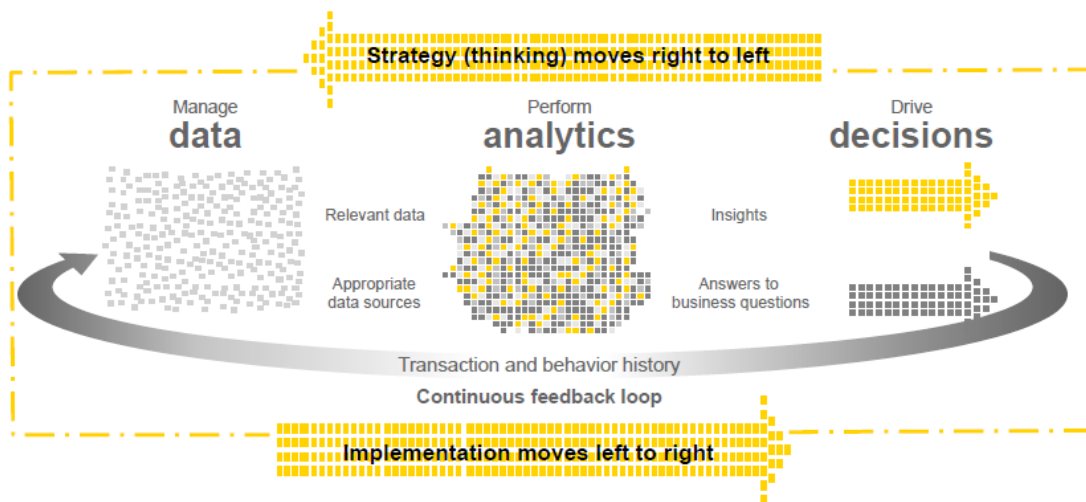


Figure 1.6: Analytics Mindset Decision Making

Being able to ask the right question is not easy; rather it heavily relies on several factors. First, the analyst needs to understand who the relevant stakeholders are and their objectives. Knowing your audience and what they want to accomplish is critical to understanding value and how to identify a right question. Second, the analyst needs to have an understanding of the business and the underlying business processes — the overall business context. As an example, if you were asked to perform a competitive analysis across the high-tech industry and if you didn't have a strong understanding of the industry and key performance indicators, you might not ask the right questions (e.g., select the right indicators to analyze).

1.5.2 Extract, Transform, and Load Relevant Data (The ETL Process)

The first focus with this competency is understanding **data characteristics** and their **relevance**. In terms of data characteristics, we already discussed the four/six V's and the big data spectrum. In determining relevance, this is a focus on what data aligns with the analysis you need to perform to

answer your question. It is also important to understand the flow of data in an accounting information system to understand where your data is coming from and how it is generated. This understanding includes - type of accounting information systems, what modules are in the context, capabilities and the limitations of the data and so on. Once this understanding is established, the ETL process can begin. This starts with the extraction of data. For extraction, key things need to be known, including: what data to ask for, how to ask for data, how to manage data security, what format the data needs to be in.

The next step is transformation, which also is referred to as data cleansing. This involves converting data from one format to another to load it into an analytics tool. This includes making certain that only the data needed is extracted and that this data is complete and accurate. Data cleansing needs to be performed both before and after the data loading process. Loading data includes knowing which tool the data should be loaded into for the most efficient and effective analysis. For example, this might be driven by the amount of data and the capacity of a given analytics tool. Throughout the ETL process, it is important to maintain the integrity of your data. This is often done through data validation, for example, a control total of your data matching an account balance total in the general ledger.

1.5.3 Apply Appropriate Data Analytics Techniques

In determining how to apply appropriate data analytics techniques, it is important to understand: the purpose of different types of data analytics techniques, how to determine which techniques are most appropriate for the objectives of your analysis. Objectives might include a need to prove or disprove your expectation if one was developed. For example, during the planning phase of an audit, the auditor is required to assess risk. One way of doing this is by exploring the data and looking for anomalies. As the audit progresses into the execution phase, the auditor considers the risk of error or intentional misstatement, the effectiveness of controls and the amounts actually recorded. The objective is to confirm or disconfirm an expectation regarding recorded amounts.

There are many ways to analyze data. Some of the more fundamental analyses that you should be able to understand and apply include:

- Ratios (e.g., gross margin or a day's sales in accounts receivable)
- Sorting (e.g., by industry or month)
- Aggregation (e.g., total of an account balance)
- Trends (e.g., the movement in inventory associated with both purchases and sales)
- Comparisons (e.g., sales month to month)
- Forecasting (e.g., budgeted expenses)

It is also important to gain familiarity with analytics tools. There are many tools capable of performing analytics and it isn't necessary for you to know how to use each one, but you should have some hands-on experience with a few of the more fundamental tools that are most readily used by an analyst. Some of the fundamental tools include: Excel, Basic databases (Access), Visualization (Tableau, Power BI).

It is also good to have a working knowledge or awareness of other tools, including those that might be specific to the career path you are choosing. Note that beyond these fundamental tools, there are other tools students should be familiar with, to a lesser extent (a working knowledge or awareness level).

1.5.4 Interpret and Share the Results

As discussed previously, the end goal is to provide insights to your stakeholders based on the objectives that were identified. Your insights are derived from your interpretation of the analytics results. Therefore, it is important that you interpret the outcomes of your analysis appropriately, based on your question and expectation, if you had one. When you look at the results of your analysis, use your critical thinking and ask yourself:

- What do you see?
- Do you see what you expected to see?
- Do your results make sense to you?
- Is any further analysis required to meet your objective?

Once you have interpreted your results, you need to summarize them in a manner conducive to and compelling for your stakeholder. Visualization can be used as a technique and a way to present findings as well. You can make choices about displaying your analysis in a variety of ways, which might include tables, area charts, map charts, heat maps, Gantt charts, horizontal or vertical bar charts, pie charts, line charts, scatter plots, bubble charts and more. When making choices about which visualization is appropriate, there are many design principles to consider. These might include color, sizing, labeling, visual simplicity (e.g., elimination of visual clutter), etc

1.6 Data Analytics Skillset

Proficiency in data analytics involves a combination of skills. For example, a solid understanding in mathematics and statistics helps one to gain the foundational knowledge in data science. Figure ?? depicts the skills necessary to be proficient in data science.

1.7 Roles in Data, Analytics, and AI

Different kinds of data professionals work in modern organizations. These roles include data analysts, business analysts, data scientists, machine learning engineers, and generative AI engineers. Figure ?? shows the skills these roles require and the tools professionals in those roles use. These roles share a common foundation in working with data, yet they differ significantly in focus, required skills, and

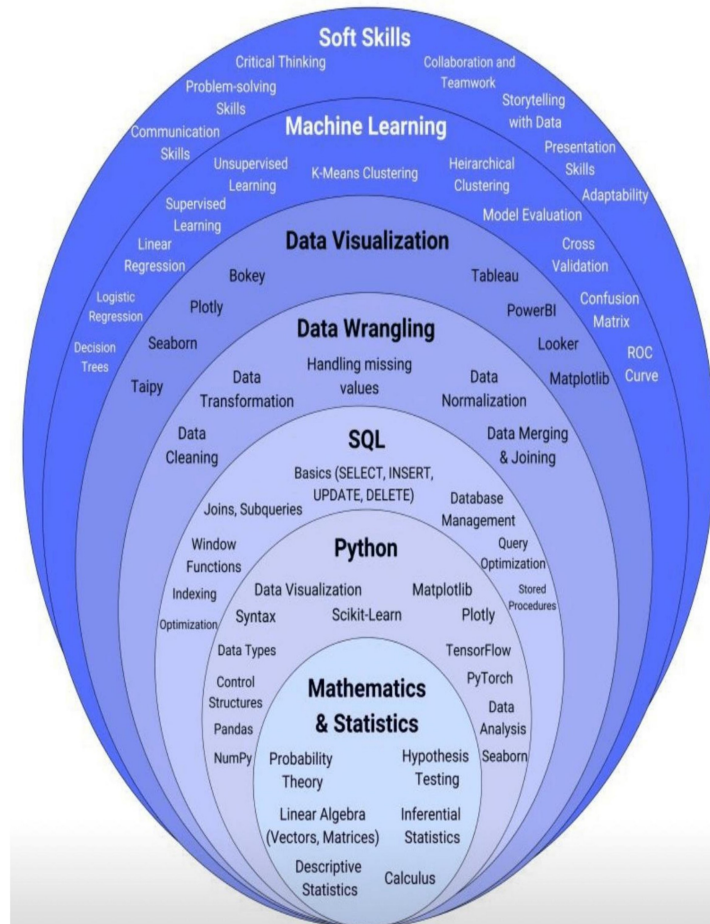


Figure 1.7: Data Science Skillset

Data Analyst	Data Scientist	Business Analyst	ML Engineer	GenAI Engineer
				
Analyze existing data to generate insights and support data-driven decision-making	Develop and implement statistical models and machine learning algorithms to derive insights and make data driven predictions	Analyze and document business processes to identify opportunities, requirements, and recommendsl for improvement	Design, develop, and deploy machine learning systems to ensure scalability, performance, and reliability in production	Develop and deploy generative AI models and applications for content generation, automation, and personalized experiences
Skills: SQL Excel Statistics Data Visallization Python	Skills: Python / R Machine Learning Statistics and Probability Data Wrangling and Feature Eng Data Visaulization	Skills: Business Process Modeling Communication & Requirements Gathering Data Analysis Problem-Solving Basic SQL & Excel	Skills: Machine Learning Data Engineering (ETL, Pipelines) Python / Java SQL Big Data Tools	Tools: Python (Transformers PyTorch, TensorFlow) HuggingFace LangChain
Tools: SQL Excel Tableau Jira	Tools Excel Tableau	Tools Microsoff-Office Suite	Software Engineering Tools Spark, Hadoop	Tools LangGAPIs LLAMs

Figure 1.8: Roles in Data Science and AI

typical deliverables, ranging from descriptive reporting and business decision support to building production-grade machine learning and generative AI systems.

A **data analyst** is a professional who collects, cleans, and organizes data to produce insights that support business decisions. This role typically involves working with data from multiple sources, applying statistical and analytical techniques to identify trends and patterns, and translating those findings into clear reports, dashboards, and visualizations for nontechnical stakeholders. Data analysts often collaborate closely with business teams to clarify questions, define key performance indicators, and recommend data-driven improvements to processes, products, or strategies, making them a central link between raw data and actionable decision-making.

A **data scientist** is a specialist who uses advanced statistical modeling, machine learning, and programming to extract deeper patterns and predictive insights from large, complex datasets. Data scientists design experiments and build models, validate them using rigorous evaluation techniques, and then deploy or operationalize these models to support strategic decisions and automated systems. They typically work end-to-end across the data lifecycle—exploring data, engineering features, selecting algorithms, and interpreting results—while collaborating with business stakeholders and engineering teams to ensure that their solutions are both technically sound and aligned with organizational goals.

2 Foundations of Accounting Data

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Gain an understanding about the types of accounting data
- Gain an understanding about the sources of accounting data
- Gain an understanding how accounting data can be collected
- Gain an understanding how accounting data are evolving
- Collect some accounting data

2.1 Types of Accounting Data

2.2 Data Sources and Collection Methods

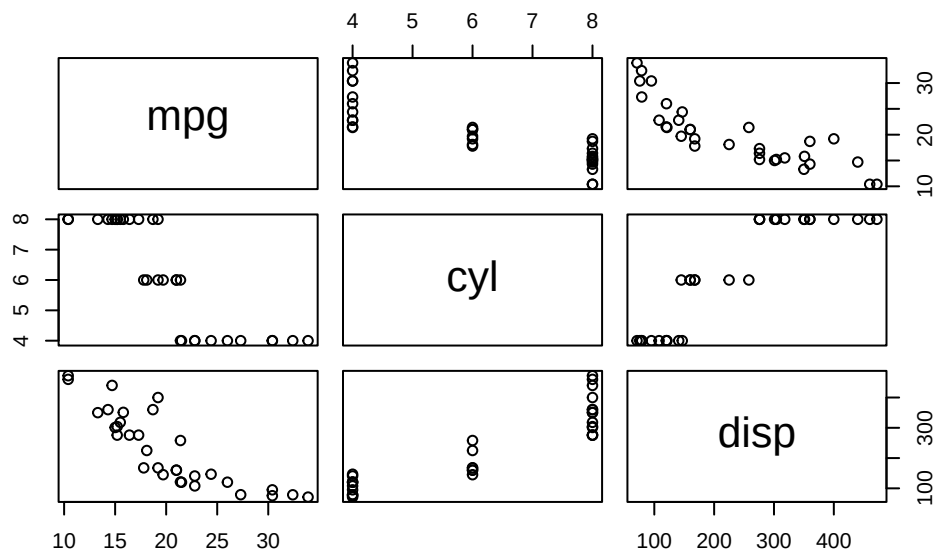
2.3 Data Quality & Integrity

summary, this book has no content whatsoever.

```
1 + 1
```

```
[1] 2
```

```
plot(mtcars[1:3])
```



See Figure ?? for first ggplot graph

```
library(tidyverse)
mtcars %>%
  as_tibble() %>%
  ggplot(mapping = aes(x = mpg, y = disp))+
  geom_point()+
  geom_smooth()+
  labs(x = 'Miles Per Gallon (mpg)',
       y = 'Displacement in Cubic Inches (disp)')
```

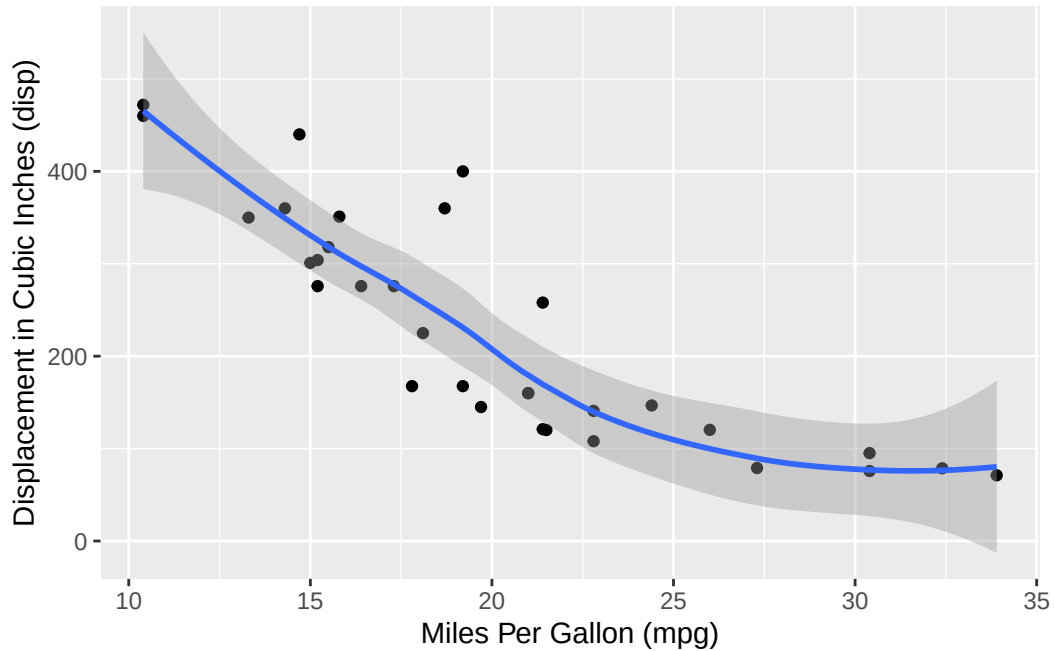


Figure 2.1: Scatter plot and line plot of the relation between mpg and disp

```
library(lubridate)
```

```
import os
import sys
sys.version
```

```
'3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)]'
```

```
for x in os.listdir():
    print (x)
```

```
.env
.git
.gitattributes
.gitignore
.quarto
.README.md.swp
.README_BASE_1260.md.swp
.README_LOCAL_1260.md.swp
```

.README_REMOTE_1260.md.swp
.Rhistory
.vdoc.b7f6298e-a6af-42d8-806b-03c5619ab147.py
accounting_analytics_book
advanced_analytics.html
advanced_analytics.qmd
advanced_analytics.quarto_ipynb
advanced_analytics.quarto_ipynb_1
advanced_analytics.quarto_ipynb_2
advanced_analytics.quarto_ipynb_3
advanced_analytics.quarto_ipynb_4
advanced_analytics.quarto_ipynb_5
advanced_analytics.quarto_ipynb_6
advanced_analytics.quarto_ipynb_7
advanced_analytics.quarto_ipynb_8
chapter1_solution.html
chapter1_solution.qmd
chapter2_solution.html
chapter2_solution.qmd
cover.png
dashboard.html
dashboard.qmd
dashboard.quarto_ipynb
dashboard.quarto_ipynb_1
dashboard.quarto_ipynb_2
dashboard.quarto_ipynb_3
dashboard.quarto_ipynb_4
dashboard.quarto_ipynb_5
dashboard.quarto_ipynb_6
dashboard.quarto_ipynb_7
dashboard.quarto_ipynb_8
DATA
data_management.html
data_management.qmd
eda.html
eda.qmd
ethics.html
ethics.qmd
file3e9c78b886c
file6ed81b1d1d61
file729017a834a5
foundations.html
foundations.qmd

foundations.rmarkdown
foundations_files
fraud.html
fraud.qmd
future.html
future.qmd
images
index.aux
index.html
index.log
index.pdf
index.qmd
index.tex
index.toc
intro.qmd
machine_learning.html
machine_learning.qmd
machine_learning.quarto_ipynb
machine_learning.quarto_ipynb_1
machine_learning.quarto_ipynb_2
machine_learning.quarto_ipynb_3
machine_learning.quarto_ipynb_4
machine_learning.quarto_ipynb_5
machine_learning.quarto_ipynb_6
machine_learning.quarto_ipynb_7
machine_learning.quarto_ipynb_8
machine_learning_files
nlp.html
nlp.qmd
nlp.quarto_ipynb
nlp.quarto_ipynb_1
nlp.quarto_ipynb_2
nlp.quarto_ipynb_3
nlp.quarto_ipynb_4
nlp.quarto_ipynb_5
nlp.quarto_ipynb_6
nlp.quarto_ipynb_7
nlp.quarto_ipynb_8
nlp_files
overview.html
overview.qmd
performance_measurement.html
performance_measurement.qmd

predictive.html
predictive.ipynb
predictive.qmd
predictive_files
prescriptive.html
prescriptive.qmd
python_basics.html
python_basics.qmd
python_basics.quarto_ipynb
python_basics.quarto_ipynb_1
python_basics.quarto_ipynb_2
python_basics.quarto_ipynb_3
python_basics.quarto_ipynb_4
python_basics.quarto_ipynb_5
python_basics.quarto_ipynb_6
python_basics.quarto_ipynb_7
python_basics.quarto_ipynb_8
python_version_used.txt
README.html
README.md
README_files
references.bib
references.html
references.qmd
requirements.txt
r_basics.html
r_basics.qmd
r_basics_files
site_libs
visualization.html
visualization.qmd
visualization_files
_book
_quarto.yml

Part II

Data Exploration and Management

3 Exploratory Data Analysis (EDA)

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Learn about the purpose of Exploratory Data Analysis (EDA)
- Understand different techniques of transforming and cleaning data
- Learn about Different R and Python Packages for EDA
- Understand how to use six verbs for EDA
- Perform EDA on some real world data sets
- Learn about how to interpret results from EDA

3.1 Introduction

Exploratory Data Analysis (EDA) is the initial process of examining a dataset to understand its main characteristics before applying formal statistical models or machine learning algorithms. It typically combines summary statistics with visual tools such as histograms, boxplots, and scatterplots to uncover patterns, spot anomalies, test assumptions, and check data quality.

By revealing relationships among variables, distributions, and potential outliers, EDA helps refine research questions, choose appropriate modeling techniques, and prevent misleading conclusions. In this way, EDA acts as a bridge between raw data and rigorous analysis, ensuring that subsequent steps are grounded in an accurate and transparent understanding of the underlying information.

3.2 Data Collection & Importing

3.3 Data Cleaning

3.4 Packages for Exploratory Data Analysis (EDA)

In order to use `pyjanitor`, the data frame must be pandas because `pyjanitor` extends pandas data frame functionality.

3.5 dplyr

```
# loading packages
library(tidyverse)
library(lubridate)
library(janitor)
```

3.6 pandas

```
# loading the package
import numpy as np
import pandas as pd
# from pyjanitor package
# pip install pyjanitor
import janitor
from janitor import clean_names, remove_empty
```

3.7 polars

```
# loading the package
import polars as pl
```

3.8 Importing the Dataset

3.9 dplyr

```
# importing data frame
df = read_csv("https://raw.githubusercontent.com/msharifbd/DATA/main/Al-Bundy")
```

3.10 pandas

```
# importing data frame
df_pd = pd.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/main/Al-Bundy")
```

3.11 polars

```
# importing data frame
df_pl = pl.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/main/Al-Bundy")
```

3.12 Meta Data

Meta data is data about the data. Before we put the data into analysis, we need to learn about our dataset. This learning involves knowing about the number of rows, number of columns, the types of the fields, the appropriateness of those types, the missing values in the dataset and so on.

3.13 dplyr

```
glimpse(df)
```

```
Rows: 14,967
Columns: 14
$ InvoiceNo      <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396~
$ Date          <chr> "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2~
$ Country       <chr> "United Kingdom", "United States", "Canada", "United S~
$ ProductID     <dbl> 2152, 2230, 2160, 2234, 2222, 2173, 2200, 2238, 2191, ~
```

```

$ Shop      <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "~
$ Gender    <chr> "Male", "Male", "Male", "Female", "Female", "Male", "F~
$ `Size (US)` <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0,~
$ `Size (Europe)` <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40~
$ `Size (UK)` <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, ~
$ UnitPrice <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129,~
$ Discount  <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,~
$ Year      <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, ~
$ Month     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ SalePrice <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0~

```

```

map_df(df, ~sum(is.na(.))) |>
  glimpse()

```

```

Rows: 1
Columns: 14
$ InvoiceNo      <int> 0
$ Date          <int> 0
$ Country       <int> 0
$ ProductID     <int> 0
$ Shop          <int> 0
$ Gender        <int> 0
$ `Size (US)`   <int> 0
$ `Size (Europe)` <int> 0
$ `Size (UK)`   <int> 0
$ UnitPrice     <int> 0
$ Discount      <int> 0
$ Year          <int> 0
$ Month         <int> 0
$ SalePrice     <int> 0

```

```
ncol(df)
```

```
[1] 14
```

```
nrow(df)
```

```
[1] 14967
```

```
head(df)
```

```
# A tibble: 6 x 14
  InvoiceNo Date      Country ProductID Shop Gender `Size (US)` `Size (Europe)`
    <dbl> <chr>    <chr>      <dbl> <chr> <chr>      <dbl> <chr>
1     52389 1/1/2014 United ~    2152 UK2   Male         11  44
2     52390 1/1/2014 United ~    2230 US15  Male        11.5 44-45
3     52391 1/1/2014 Canada    2160 CAN7   Male         9.5 42-43
4     52392 1/1/2014 United ~    2234 US6    Female        9.5 40
5     52393 1/1/2014 United ~    2222 UK4    Female         9  39-40
6     52394 1/1/2014 United ~    2173 US15  Male        10.5 43-44
# i 6 more variables: `Size (UK)` <dbl>, UnitPrice <dbl>, Discount <dbl>,
#   Year <dbl>, Month <dbl>, SalePrice <dbl>
```

```
tail(df)
```

```
# A tibble: 6 x 14
  InvoiceNo Date      Country ProductID Shop Gender `Size (US)` `Size (Europe)`
    <dbl> <chr>    <chr>      <dbl> <chr> <chr>      <dbl> <chr>
1     65772 12/31/20~ United~    2168 US13  Male         8   41
2     65773 12/31/20~ United~    2154 UK2   Male        9.5 42-43
3     65774 12/31/20~ United~    2181 US12  Female       12  42-43
4     65775 12/31/20~ Canada    2203 CAN6   Male        10.5 43-44
5     65776 12/31/20~ Germany    2231 GER1  Female        9.5 40
6     65777 12/31/20~ Germany    2156 GER1  Female        6.5 37
# i 6 more variables: `Size (UK)` <dbl>, UnitPrice <dbl>, Discount <dbl>,
#   Year <dbl>, Month <dbl>, SalePrice <dbl>
```

```
dplyr::sample_n(df, 10)
```

```
# A tibble: 10 x 14
  InvoiceNo Date      Country ProductID Shop Gender `Size (US)` `Size (Europe)`
    <dbl> <chr>    <chr>      <dbl> <chr> <chr>      <dbl> <chr>
1     63768 9/7/2016 Germany    2242 GER1  Female        7.5 38
2     58683 11/24/2~ Canada    2149 CAN5  Female         7  37-38
3     63977 9/17/20~ Germany    2163 GER1  Male        10.5 43-44
4     60327 3/11/20~ United~    2234 US14  Female       11.5 42
5     63743 9/6/2016 Canada    2182 CAN1  Female        10  40-41
6     59805 2/8/2016 United~    2166 US13  Female        7.5 38
7     61504 5/18/20~ Germany    2164 GER1  Female        9.5 40
```



```

8      55037 2/2/2015 United~      2171 US9   Male      10.5 43-44
9      59913 2/15/20~ Germany      2241 GER1  Male      11    44
10     61861 6/7/2016 United~      2177 US13  Male      9.5 42-43
# i 6 more variables: `Size (UK)` <dbl>, UnitPrice <dbl>, Discount <dbl>,
#   Year <dbl>, Month <dbl>, SalePrice <dbl>

```

3.14 Pandas

```
df_pd.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   InvoiceNo              14967 non-null  int64
1   Date                  14967 non-null  object
2   Country                14967 non-null  object
3   ProductID             14967 non-null  int64
4   Shop                  14967 non-null  object
5   Gender                 14967 non-null  object
6   Size (US)              14967 non-null  float64
7   Size (Europe)          14967 non-null  object
8   Size (UK)              14967 non-null  float64
9   UnitPrice              14967 non-null  int64
10  Discount               14967 non-null  float64
11  Year                   14967 non-null  int64
12  Month                  14967 non-null  int64
13  SalePrice              14967 non-null  float64
dtypes: float64(4), int64(5), object(5)
memory usage: 1.6+ MB

```

```
df_pd.shape
```

```
(14967, 14)
```

```

print('The total number of rows and columns of the product data is \
    {} and {} respectively.'.format(df_pd.shape[0], df_pd.shape[1]))

```

The total number of rows and columns of the product data is 14967 and 14 respectively

```
print(f'The total number of rows and columns of the product data is \
      {df_pd.shape[0]} and {df_pd.shape[1]} respectively.')
```

The total number of rows and columns of the product data is 14967 and 14 respectively

```
df_pd.columns
```

```
Index(['InvoiceNo', 'Date', 'Country', 'ProductID', 'Shop', 'Gender',
      'Size (US)', 'Size (Europe)', 'Size (UK)', 'UnitPrice', 'Discount',
      'Year', 'Month', 'SalePrice'],
      dtype='object')
```

```
df_pd.head()
```

	InvoiceNo	Date	Country	...	Year	Month	SalePrice
0	52389	1/1/2014	United Kingdom	...	2014	1	159.0
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
4	52393	1/1/2014	United Kingdom	...	2014	1	159.0

[5 rows x 14 columns]

```
df_pd.tail()
```

	InvoiceNo	Date	Country	...	Year	Month	SalePrice
14962	65773	12/31/2016	United Kingdom	...	2016	12	139.0
14963	65774	12/31/2016	United States	...	2016	12	149.0
14964	65775	12/31/2016	Canada	...	2016	12	125.3
14965	65776	12/31/2016	Germany	...	2016	12	199.0
14966	65777	12/31/2016	Germany	...	2016	12	125.1

[5 rows x 14 columns]

```
df_pd.isna().sum()
```

```

InvoiceNo      0
Date           0
Country        0
ProductID      0
Shop           0
Gender         0
Size (US)      0
Size (Europe)  0
Size (UK)      0
UnitPrice      0
Discount       0
Year           0
Month          0
SalePrice      0
dtype: int64

```

```
df_pd.dtypes
```

```

InvoiceNo      int64
Date           object
Country        object
ProductID      int64
Shop           object
Gender         object
Size (US)      float64
Size (Europe)  object
Size (UK)      float64
UnitPrice      int64
Discount       float64
Year           int64
Month          int64
SalePrice      float64
dtype: object

```

```
df_pd.sample(n=10)
```

	InvoiceNo	Date	Country	...	Year	Month	SalePrice
10887	62123	6/20/2016	United States	...	2016	6	139.0
10805	62052	6/17/2016	United States	...	2016	6	179.0
12822	63838	9/11/2016	Germany	...	2016	9	135.2
4018	55904	4/30/2015	United States	...	2015	4	152.1

3733	55640	4/4/2015	Canada	...	2015	4	135.2
7458	59070	12/21/2015	Germany	...	2015	12	189.0
10884	62120	6/20/2016	United Kingdom	...	2016	6	159.0
5307	57082	7/30/2015	United States	...	2015	7	139.3
14662	65492	12/11/2016	Germany	...	2016	12	111.2
10348	61646	5/26/2016	United States	...	2016	5	199.0

[10 rows x 14 columns]

3.15 polars

```
# metadata
df_pl.shape # rows and columns
```

(14967, 14)

```
df_pl.height # rows
```

14967

```
df_pl.width # columns
```

14

```
df_pl.columns
```

['InvoiceNo', 'Date', 'Country', 'ProductID', 'Shop', 'Gender', 'Size (US)', 'S

```
len(df_pl.columns)
```

14

```
df_pl.schema # column names and their types
```

Schema([('InvoiceNo', Int64), ('Date', String), ('Country', String), ('Product

```
df_pl.dtypes
```

```
[Int64, String, String, Int64, String, String, Float64, String, Float64, Int64,
```

```
df_pl.head()
```

```
shape: (5, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	Sa
---	---	---	---		---	---	---	
i64	str	str	i64		i64	i64	f64	
52389	1/1/2014	United Kingdom	2152	...	0.0	2014	1	159.0
52390	1/1/2014	United States	2230	...	0.2	2014	1	159.2
52391	1/1/2014	Canada	2160	...	0.2	2014	1	119.2
52392	1/1/2014	United States	2234	...	0.0	2014	1	159.0
52393	1/1/2014	United Kingdom	2222	...	0.0	2014	1	159.0

```
df_pl.tail()
```

```
shape: (5, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---		---	---	---	
i64	str	str	i64		i64	i64	f64	
65773	12/31/2016	United Kingdom	2154	...	0.0	2016	12	13
65774	12/31/2016	United States	2181	...	0.0	2016	12	14
65775	12/31/2016	Canada	2203	...	0.3	2016	12	125.3
65776	12/31/2016	Germany	2231	...	0.0	2016	12	199.
65777	12/31/2016	Germany	2156	...	0.1	2016	12	125.

```
df_pl.sample(n=10)
```

```
shape: (10, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
-----------	------	---------	-----------	-----	----------	------	-------	---

---	---	---	---	---	---	---	---	---
i64	str	str	i64	f64	i64	i64	f64	
60373	3/13/2016	United States	2177	... 0.0	2016	3	169.	
52719	2/25/2014	Canada	2212	... 0.0	2014	2	149.0	
62775	7/21/2016	Germany	2164	... 0.3	2016	7	104.3	
52437	1/8/2014	United States	2207	... 0.0	2014	1	169.	
52815	3/12/2014	United States	2159	... 0.0	2014	3	179.	
59612	1/27/2016	Canada	2229	... 0.0	2016	1	159.0	
55536	3/24/2015	United States	2207	... 0.1	2015	3	179.	
56012	5/11/2015	United States	2200	... 0.0	2015	5	199.	
65640	12/21/2016	Canada	2235	... 0.0	2016	12	199.0	
60332	3/11/2016	Germany	2230	... 0.5	2016	3	94.5	

```
df_pl.describe()
```

```
shape: (9, 15)
```

statistic	InvoiceNo	Date	Country	...	Discount	Year	Month	
---	---	---	---	---	---	---	---	
str	f64	str	str	f64	f64	f64	f64	
count	14967.0	14967	14967	... 14967.0	14967.0	14967.0	14967.0	
null_count	0.0	0	0	... 0.0	0.0	0.0	0.0	
mean	59050.2615	null	null	... 0.124013	2015.3082	6.689517		
	09			11		3		
std	3889.59871	null	null	... 0.170112	0.76232	3.319909		
	4							
min	52389.0	1/1/2014	Canada	... 0.0	2014.0	1.0	64	
25%	55649.0	null	null	... 0.0	2015.0	4.0	125.	
50%	59092.0	null	null	... 0.0	2015.0	7.0	149.	
75%	62433.0	null	null	... 0.2	2016.0	10.0	169	
max	65777.0	9/9/2016	United States	... 0.5	2016.0	12.0	19	

3.16 Cleaning the Dataset

3.17 dplyr

```
df |>
  rename_all(toupper) |>
  janitor::clean_names() |>
  rename_all(toupper) |>
  glimpse()
```

Rows: 14,967

Columns: 14

```
$ INVOICENO <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE      <chr> "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2014"~
$ COUNTRY   <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID <dbl> 2152, 2230, 2160, 2234, 2222, 2173, 2200, 2238, 2191, 2237~
$ SHOP      <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
$ GENDER    <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US   <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK   <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR      <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~
```

```
df = df |>
  rename_all(toupper) |>
  janitor::clean_names() |>
  rename_all(toupper)
glimpse(df)
```

Rows: 14,967

Columns: 14

```
$ INVOICENO <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE      <chr> "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2014", "1/1/2014"~
$ COUNTRY   <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID <dbl> 2152, 2230, 2160, 2234, 2222, 2173, 2200, 2238, 2191, 2237~
$ SHOP      <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
```

```
$ GENDER      <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US     <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK     <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE   <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT   <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR        <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE   <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~
```

3.18 panads

```
df_pd.columns.str.upper().to_list()
```

```
['INVOICENO', 'DATE', 'COUNTRY', 'PRODUCTID', 'SHOP', 'GENDER', 'SIZE (US)', 'S
```

```
(df_pd
  .pipe(remove_empty)
  .pipe(lambda x: x.clean_names(case_type = "upper"))
  .pipe(lambda x: x.rename(columns = {'SIZE_US_': 'SIZE_US', 'SIZE_EUROPE_
  .pipe(lambda x: x.info())
  )
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 14967 entries, 0 to 14966
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	INVOICENO	14967 non-null	int64
1	DATE	14967 non-null	object
2	COUNTRY	14967 non-null	object
3	PRODUCTID	14967 non-null	int64
4	SHOP	14967 non-null	object
5	GENDER	14967 non-null	object
6	SIZE_US	14967 non-null	float64
7	SIZE_EUROPE	14967 non-null	object
8	SIZE_UK	14967 non-null	float64
9	UNITPRICE	14967 non-null	int64
10	DISCOUNT	14967 non-null	float64


```

11  YEAR          14967 non-null  int64
12  MONTH         14967 non-null  int64
13  SALEPRICE     14967 non-null  float64
dtypes: float64(4), int64(5), object(5)
memory usage: 1.6+ MB

```

```

# Changing the names of the columns to uppercase
df_pd.rename(columns = str.upper, inplace = True)
df_pd.columns

```

```

Index(['INVOICENO', 'DATE', 'COUNTRY', 'PRODUCTID', 'SHOP', 'GENDER',
      'SIZE (US)', 'SIZE (EUROPE)', 'SIZE (UK)', 'UNITPRICE', 'DISCOUNT',
      'YEAR', 'MONTH', 'SALEPRICE'],
      dtype='object')

```

```

new_column = df_pd.columns \
    .str.replace("(", "'').str.replace(")", "'") \
    .str.replace(' ', '_') # Cleaning the names of the variables
new_column

```

```

Index(['INVOICENO', 'DATE', 'COUNTRY', 'PRODUCTID', 'SHOP', 'GENDER',
      'SIZE_US', 'SIZE_EUROPE', 'SIZE_UK', 'UNITPRICE', 'DISCOUNT', 'YEAR',
      'MONTH', 'SALEPRICE'],
      dtype='object')

```

```

df_pd.columns = new_column
df_pd.columns

```

```

Index(['INVOICENO', 'DATE', 'COUNTRY', 'PRODUCTID', 'SHOP', 'GENDER',
      'SIZE_US', 'SIZE_EUROPE', 'SIZE_UK', 'UNITPRICE', 'DISCOUNT', 'YEAR',
      'MONTH', 'SALEPRICE'],
      dtype='object')

```

```

df_pd.rename(columns=str.upper, inplace = True)
df_pd.columns

```

```

Index(['INVOICENO', 'DATE', 'COUNTRY', 'PRODUCTID', 'SHOP', 'GENDER',
      'SIZE_US', 'SIZE_EUROPE', 'SIZE_UK', 'UNITPRICE', 'DISCOUNT', 'YEAR',
      'MONTH', 'SALEPRICE'],
      dtype='object')

```

3.19 polars

```
# upper case column name
df_pl.rename({col: col.upper() for col in df_pl.columns})
```

shape: (14_967, 14)

INVOICENO	DATE	COUNTRY	PRODUCTID	...	DISCOUNT	YEAR	MONTH	S
---	---	---	---	...	---	---	---	---
i64	str	str	i64	f64	i64	i64	f64	
52389	1/1/2014	United Kingdom	2152	...	0.0	2014	1	159
52390	1/1/2014	United States	2230	...	0.2	2014	1	159
52391	1/1/2014	Canada	2160	...	0.2	2014	1	119.2
52392	1/1/2014	United States	2234	...	0.0	2014	1	159
52393	1/1/2014	United Kingdom	2222	...	0.0	2014	1	159
...	...	...	...	...	...	...	...	...
65773	12/31/2016	United Kingdom	2154	...	0.0	2016	12	13
65774	12/31/2016	United States	2181	...	0.0	2016	12	14
65775	12/31/2016	Canada	2203	...	0.3	2016	12	125.3
65776	12/31/2016	Germany	2231	...	0.0	2016	12	199.
65777	12/31/2016	Germany	2156	...	0.1	2016	12	125.

```
df_pl.rename({col: col.strip().upper().replace(' ', '_') for col in df_pl.col
```

shape: (14_967, 14)

INVOICENO	DATE	COUNTRY	PRODUCTID	...	DISCOUNT	YEAR	MONTH	S
---	---	---	---	...	---	---	---	---
i64	str	str	i64	f64	i64	i64	f64	
52389	1/1/2014	United Kingdom	2152	...	0.0	2014	1	159
52390	1/1/2014	United States	2230	...	0.2	2014	1	159
52391	1/1/2014	Canada	2160	...	0.2	2014	1	119.2
52392	1/1/2014	United States	2234	...	0.0	2014	1	159
52393	1/1/2014	United Kingdom	2222	...	0.0	2014	1	159
...	...	...	...	...	...	...	...	...
65773	12/31/2016	United Kingdom	2154	...	0.0	2016	12	13
65774	12/31/2016	United States	2181	...	0.0	2016	12	14

65775	12/31/2016	Canada	2203	...	0.3	2016	12	125.
65776	12/31/2016	Germany	2231	...	0.0	2016	12	199.
65777	12/31/2016	Germany	2156	...	0.1	2016	12	125.

3.19.1 Changing the Types of Variables

3.20 dplyr

```
df |>
  mutate (DATE = lubridate::mdy(DATE)) |>
  glimpse()
```

Rows: 14,967

Columns: 14

```
$ INVOICENO  <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE      <date> 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-0~
$ COUNTRY   <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID <dbl> 2152, 2230, 2160, 2234, 2222, 2173, 2200, 2238, 2191, 2237~
$ SHOP      <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
$ GENDER    <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US   <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK   <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR      <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~
```

From the above, it is now evident the the type of the DATE variable now is date.

```
df |>
  mutate (DATE = lubridate::mdy(DATE)) |>
  mutate (PRODUCTID = as.character(PRODUCTID)) |>
  glimpse()
```

Rows: 14,967

Columns: 14

```

$ INVOICENO    <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE         <date> 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-0~
$ COUNTRY      <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID    <chr> "2152", "2230", "2160", "2234", "2222", "2173", "2200", "2~
$ SHOP         <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
$ GENDER       <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US      <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE  <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK      <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE    <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT    <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR         <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE    <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~

```

From the above, it is now evident the the type of the DATE and PRODUCTID variable now is date (date) and character (chr) respectively. We can now incorporate the changes into the data frame.

```

df = df |>
  mutate (DATE = lubridate::mdy(DATE)) |>
  mutate (PRODUCTID = as.character(PRODUCTID))
glimpse(df)

```

Rows: 14,967

Columns: 14

```

$ INVOICENO    <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE         <date> 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-0~
$ COUNTRY      <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID    <chr> "2152", "2230", "2160", "2234", "2222", "2173", "2200", "2~
$ SHOP         <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
$ GENDER       <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US      <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE  <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK      <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE    <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT    <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR         <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE    <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~

```

3.21 pandas

```
(
    df_pd
    .pipe(lambda x: x.assign(
        DATE = pd.to_datetime(x['DATE'])))
    .pipe(lambda x: x.info())
)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICENO        14967 non-null  int64
1   DATE             14967 non-null  datetime64[ns]
2   COUNTRY          14967 non-null  object
3   PRODUCTID       14967 non-null  int64
4   SHOP            14967 non-null  object
5   GENDER          14967 non-null  object
6   SIZE_US         14967 non-null  float64
7   SIZE_EUROPE     14967 non-null  object
8   SIZE_UK         14967 non-null  float64
9   UNITPRICE       14967 non-null  int64
10  DISCOUNT       14967 non-null  float64
11  YEAR            14967 non-null  int64
12  MONTH           14967 non-null  int64
13  SALEPRICE       14967 non-null  float64
dtypes: datetime64[ns](1), float64(4), int64(5), object(4)
memory usage: 1.6+ MB
```

```
# converting integer to object
df_pd.INVOICENO = df_pd.INVOICENO.astype(str)
df_pd[['MONTH', 'PRODUCTID']] = df_pd[['MONTH', 'PRODUCTID']].astype(str)
df_pd.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICENO        14967 non-null  object
```

```

1  DATE          14967 non-null  object
2  COUNTRY       14967 non-null  object
3  PRODUCTID     14967 non-null  object
4  SHOP          14967 non-null  object
5  GENDER        14967 non-null  object
6  SIZE_US       14967 non-null  float64
7  SIZE_EUROPE   14967 non-null  object
8  SIZE_UK       14967 non-null  float64
9  UNITPRICE     14967 non-null  int64
10 DISCOUNT     14967 non-null  float64
11 YEAR          14967 non-null  int64
12 MONTH         14967 non-null  object
13 SALEPRICE     14967 non-null  float64
dtypes: float64(4), int64(2), object(8)
memory usage: 1.6+ MB

```

3.22 polars

```

# Change the type of columns
df_pl = df_pl.with_columns(
    pl.col("Date").str.strptime(pl.Date, "%m/%d/%Y")
)
df_pl.head()

```

shape: (5, 14)

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	...	---	---	---	---
i64	date	str	i64	...	f64	i64	i64	f64
52389	2014-01-01	United Kingdom	2152	...	0.0	2014	1	159
52390	2014-01-01	United States	2230	...	0.2	2014	1	159
52391	2014-01-01	Canada	2160	...	0.2	2014	1	119.2
52392	2014-01-01	United States	2234	...	0.0	2014	1	159
52393	2014-01-01	United Kingdom	2222	...	0.0	2014	1	159

```
df_pl.schema
```

```
Schema([('InvoiceNo', Int64), ('Date', Date), ('Country', String), ('ProductID
```

```
# change to character variable
df_pl = df_pl.with_columns(pl.col("InvoiceNo").cast(pl.Utf8))
df_pl.schema
```

```
Schema([('InvoiceNo', String), ('Date', Date), ('Country', String), ('ProductID', String)])
```

```
df_pl = df_pl.with_columns(pl.col(["ProductID", "Month"]).cast(pl.Utf8))
df_pl.schema
```

```
Schema([('InvoiceNo', String), ('Date', Date), ('Country', String), ('ProductID', String)])
```

3.23 Some Other Useful Functions

There are some other useful functions that can be used to explore the dataset for analysis. Some of those useful functions are discussed below.

3.24 dplyr

```
df|> count(YEAR)
```

```
# A tibble: 3 x 2
  YEAR      n
  <dbl> <int>
1  2014  2753
2  2015  4848
3  2016  7366
```

```
df|> count(COUNTRY)
```

```
# A tibble: 4 x 2
  COUNTRY      n
  <chr>      <int>
1 Canada    2952
2 Germany   4392
3 United Kingdom 1737
4 United States 5886
```

```
df|> distinct(COUNTRY)
```

```
# A tibble: 4 x 1
  COUNTRY
  <chr>
1 United Kingdom
2 United States
3 Canada
4 Germany
```

3.25 pandas

```
df_pd['YEAR'].value_counts()
```

```
YEAR
2016    7366
2015    4848
2014    2753
Name: count, dtype: int64
```

```
df_pd['YEAR'].unique()
```

```
array([2014, 2015, 2016], dtype=int64)
```

3.26 polars

```
df_pl.select(pl.col("Year").value_counts())
```

```
shape: (3, 1)
```

Year

struct[2]
{2015, 4848}
{2016, 7366}


```
{2014,2753}
```

```
df_pl.select(pl.col("Year").value_counts()).unnest("Year")
```

shape: (3, 2)

Year	count
---	---
i64	u32
2015	4848
2016	7366
2014	2753

```
df_pl.select(pl.col("Gender").value_counts())
```

shape: (2, 1)

Gender

struct[2]
{"Female",6048}
{"Male",8919}

```
df_pl.select(pl.col("Gender").value_counts()).unnest("Gender")
```

shape: (2, 2)

Gender	count
---	---
str	u32
Male	8919
Female	6048

```
df_pl.group_by("Gender").len()
```

shape: (2, 2)

Gender	len
---	---
str	u32
Male	8919
Female	6048

```
df_pl.group_by("Gender").len().rename({"len": "Total"})
```

shape: (2, 2)

Gender	Total
---	---
str	u32
Male	8919
Female	6048

3.27 Six Verbs for EDA

Table ?? shows the comparable functions in both `dplyr` and `pandas` packages. These functions are very much important to perform exploratory data analysis in both R and Python. `group_by` (`groupby` in `pandas`) and `summarize ()`¹ (`agg ()` in `pandas`) are often used together; therefore, they are in the same group in Table ??.

3.27.1 1st Verb - `filter ()` Function

Filter functions are used to subset a data frame based on rows, meaning that retaining rows that satisfy given conditions. Filtering rows is also called *slicing*² because we obtain a set of elements by filtering.

¹You can also use British spelling - `summarise ()`

²Indexing involves obtaining individual elements.

Table 3.1: Tidyverse and Pandas Equivalent Functions

Verb Number	tidyverse	pandas	polars
1	filter ()	query () or loc () or iloc ()	filter ()
2	arrange ()	sort_values ()	sort ()
3	select ()	filter () or loc ()	select ()
4	rename ()	rename ()	rename ()
5	mutate ()	assign ()	with_columns ()
6	group_by ()	groupby ()	group_by ()
6	summarize ()	agg ()	agg ()

3.28 dplyr

```
df |> filter (YEAR == "2015")
```

```
# A tibble: 4,848 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP  GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>      <chr>      <chr>      <chr> <chr>  <dbl> <chr>
1  54725 2015-01-01 United States 2187    US7  Male    9.5 42-43
2  54726 2015-01-01 United States 2174    US3  Male    8.5 41-42
3  54727 2015-01-01 United States 2240    US11 Male    9 42
4  54728 2015-01-01 Germany      2220    GER2 Male    10 43
5  54729 2015-01-01 United Kingd~ 2199    UK5  Male    9.5 42-43
6  54730 2015-01-01 Canada      2169    CAN7 Female  7 37-38
7  54731 2015-01-01 Germany      2188    GER1 Female  9.5 40
8  54732 2015-01-02 United Kingd~ 2155    UK5  Female  10 40-41
9  54733 2015-01-02 United States 2173    US5  Female  9 39-40
10 54734 2015-01-02 Germany      2222    GER3 Female  7.5 38
# i 4,838 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>
```

```
df |> filter (COUNTRY %in% c("United States", "Canada"))
```

```
# A tibble: 8,838 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP  GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>      <chr>      <chr>      <chr> <chr>  <dbl> <chr>
1  52390 2014-01-01 United States 2230    US15 Male   11.5 44-45
2  52391 2014-01-01 Canada      2160    CAN7 Male    9.5 42-43
```

```

3     52392 2014-01-01 United States 2234     US6   Female     9.5 40
4     52394 2014-01-01 United States 2173     US15  Male     10.5 43-44
5     52396 2014-01-02 Canada        2238     CAN5   Male      10   43
6     52397 2014-01-02 United States 2191     US13  Male     10.5 43-44
7     52399 2014-01-02 United States 2197     US1   Male      10   43
8     52399 2014-01-02 United States 2213     US11  Female     9.5 40
9     52399 2014-01-02 United States 2206     US2   Female     9.5 40
10    52400 2014-01-02 United States 2152     US15  Male      8    41
# i 8,828 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>

```

```
df |> filter (COUNTRY == "United States", YEAR == "2016")
```

```

# A tibble: 2,935 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP  GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr>        <chr>    <chr> <chr>   <dbl> <chr>
1     59206 2016-01-02 United States 2186     US13  Female     8   38-39
2     59209 2016-01-02 United States 2193     US14  Female     9   39-40
3     59213 2016-01-02 United States 2228     US13  Male      9.5 42-43
4     59214 2016-01-02 United States 2177     US12  Female    10.5 41
5     59214 2016-01-02 United States 2236     US6   Male      8.5 41-42
6     59219 2016-01-03 United States 2188     US14  Female     9.5 40
7     59221 2016-01-03 United States 2178     US13  Female     8   38-39
8     59223 2016-01-03 United States 2158     US3   Male      8    41
9     59225 2016-01-03 United States 2236     US13  Male      8    41
10    59226 2016-01-03 United States 2207     US14  Male     14   47
# i 2,925 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>

```

```
df |> filter (COUNTRY == "United States", YEAR %in% c("2015","2016"))
```

```

# A tibble: 4,859 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP  GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr>        <chr>    <chr> <chr>   <dbl> <chr>
1     54725 2015-01-01 United States 2187     US7   Male      9.5 42-43
2     54726 2015-01-01 United States 2174     US3   Male      8.5 41-42
3     54727 2015-01-01 United States 2240     US11  Male      9    42
4     54733 2015-01-02 United States 2173     US5   Female     9   39-40
5     54738 2015-01-02 United States 2226     US3   Male     10   43

```

```

6   54739 2015-01-02 United States 2199    US11 Male    9.5 42-43
7   54742 2015-01-03 United States 2209     US6 Male     10  43
8   54743 2015-01-03 United States 2238    US15 Female    7.5 38
9   54745 2015-01-04 United States 2214    US12 Male    10.5 43-44
10  54749 2015-01-04 United States 2162    US15 Male     9.5 42-43
# i 4,849 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>

```

```
df |> filter (COUNTRY %in% c("United States", "Canada"), YEAR == "2014")
```

```

# A tibble: 1,649 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP  GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr>        <chr>    <chr> <chr>  <dbl> <chr>
1   52390 2014-01-01 United States 2230    US15 Male    11.5 44-45
2   52391 2014-01-01 Canada      2160    CAN7 Male     9.5 42-43
3   52392 2014-01-01 United States 2234     US6 Female    9.5 40
4   52394 2014-01-01 United States 2173    US15 Male    10.5 43-44
5   52396 2014-01-02 Canada      2238    CAN5 Male     10  43
6   52397 2014-01-02 United States 2191    US13 Male    10.5 43-44
7   52399 2014-01-02 United States 2197     US1 Male     10  43
8   52399 2014-01-02 United States 2213    US11 Female    9.5 40
9   52399 2014-01-02 United States 2206     US2 Female    9.5 40
10  52400 2014-01-02 United States 2152    US15 Male     8  41
# i 1,639 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>

```

3.29 pandas

```
df_pd.query("YEAR == 2015")
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
2753	54725	1/1/2015	United States	...	2015	1	179.0
2754	54726	1/1/2015	United States	...	2015	1	169.0
2755	54727	1/1/2015	United States	...	2015	1	116.1
2756	54728	1/1/2015	Germany	...	2015	1	129.0
2757	54729	1/1/2015	United Kingdom	...	2015	1	139.0
...	...	...	...	...	...	...	...

7596	59192	12/31/2015	United States	...	2015	12	79.5
7597	59193	12/31/2015	United States	...	2015	12	139.0
7598	59194	12/31/2015	Germany	...	2015	12	159.0
7599	59195	12/31/2015	Germany	...	2015	12	149.0
7600	59196	12/31/2015	United States	...	2015	12	179.0

[4848 rows x 14 columns]

```
df_pd.query('COUNTRY== "United States" | COUNTRY == "Canada"')
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
7	52396	1/2/2014	Canada	...	2014	1	169.0
...	...	...	...	...	...	...	...
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0
14964	65775	12/31/2016	Canada	...	2016	12	125.3

[8838 rows x 14 columns]

```
df_pd.query("COUNTRY in ['United States', 'Canada']")
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
7	52396	1/2/2014	Canada	...	2014	1	169.0
...	...	...	...	...	...	...	...
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0
14964	65775	12/31/2016	Canada	...	2016	12	125.3

[8838 rows x 14 columns]

```
df_pd.query("COUNTRY== 'United States' & YEAR== 2016")
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
7610	59206	1/2/2016	United States	...	2016	1	132.3
7613	59209	1/2/2016	United States	...	2016	1	127.2
7617	59213	1/2/2016	United States	...	2016	1	125.3
7618	59214	1/2/2016	United States	...	2016	1	151.2
7619	59214	1/2/2016	United States	...	2016	1	151.2
...	...	...	...	...	...	...	...
14956	65767	12/31/2016	United States	...	2016	12	139.0
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

[2935 rows x 14 columns]

```
df_pd.query("COUNTRY== 'United States' & YEAR in [2015,2016]")
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
2753	54725	1/1/2015	United States	...	2015	1	179.0
2754	54726	1/1/2015	United States	...	2015	1	169.0
2755	54727	1/1/2015	United States	...	2015	1	116.1
2761	54733	1/2/2015	United States	...	2015	1	179.0
2766	54738	1/2/2015	United States	...	2015	1	199.0
...	...	...	...	...	...	...	...
14956	65767	12/31/2016	United States	...	2016	12	139.0
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

[4859 rows x 14 columns]

```
df_pd[df_pd['COUNTRY'] == "United States"]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0

8	52397	1/2/2014	United States	...	2014	1	139.0
10	52399	1/2/2014	United States	...	2014	1	129.0
...	...	...	...	...	...	...	...
14956	65767	12/31/2016	United States	...	2016	12	139.0
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

[5886 rows x 14 columns]

```
df_pd.loc[(df_pd['COUNTRY']=="United States")]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
8	52397	1/2/2014	United States	...	2014	1	139.0
10	52399	1/2/2014	United States	...	2014	1	129.0
...	...	...	...	...	...	...	...
14956	65767	12/31/2016	United States	...	2016	12	139.0
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

[5886 rows x 14 columns]

```
df_pd.loc[df_pd['COUNTRY'].isin(["United States", "Canada"])]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
7	52396	1/2/2014	Canada	...	2014	1	169.0
...	...	...	...	...	...	...	...
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

14964	65775	12/31/2016	Canada	...	2016	12	125.3
-------	-------	------------	--------	-----	------	----	-------

[8838 rows x 14 columns]

```
df_pd.loc[df_pd['COUNTRY']\
.isin(["United States", "Canada"]) & (df_pd['YEAR']==2014)]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
7	52396	1/2/2014	Canada	...	2014	1	169.0
...	...	...	...	...	...	...	...
2739	54713	12/30/2014	United States	...	2014	12	189.0
2745	54718	12/31/2014	Canada	...	2014	12	151.2
2746	54719	12/31/2014	United States	...	2014	12	199.0
2748	54721	12/31/2014	Canada	...	2014	12	74.5
2749	54722	12/31/2014	United States	...	2014	12	118.3

[1649 rows x 14 columns]

```
df_pd.loc[(df_pd['COUNTRY']=="United States") & (df_pd ["YEAR"] ==2014)]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
8	52397	1/2/2014	United States	...	2014	1	139.0
10	52399	1/2/2014	United States	...	2014	1	129.0
...	...	...	...	...	...	...	...
2731	54705	12/29/2014	United States	...	2014	12	179.0
2734	54708	12/30/2014	United States	...	2014	12	159.0
2739	54713	12/30/2014	United States	...	2014	12	189.0
2746	54719	12/31/2014	United States	...	2014	12	199.0
2749	54722	12/31/2014	United States	...	2014	12	118.3

[1027 rows x 14 columns]

```
df_pd.loc[df_pd['COUNTRY'] == "United States", :]
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
1	52390	1/1/2014	United States	...	2014	1	159.2
3	52392	1/1/2014	United States	...	2014	1	159.0
5	52394	1/1/2014	United States	...	2014	1	159.0
8	52397	1/2/2014	United States	...	2014	1	139.0
10	52399	1/2/2014	United States	...	2014	1	129.0
...	...	...	...	...	...	...	...
14956	65767	12/31/2016	United States	...	2016	12	139.0
14959	65770	12/31/2016	United States	...	2016	12	119.2
14960	65771	12/31/2016	United States	...	2016	12	189.0
14961	65772	12/31/2016	United States	...	2016	12	129.0
14963	65774	12/31/2016	United States	...	2016	12	149.0

[5886 rows x 14 columns]

```
df_pd.loc[
    df_pd['COUNTRY']=='United States',
    ['COUNTRY', "UNITPRICE", "SALEPRICE"]]
```

	COUNTRY	UNITPRICE	SALEPRICE
1	United States	199	159.2
3	United States	159	159.0
5	United States	159	159.0
8	United States	139	139.0
10	United States	129	129.0
...	...	...	...
14956	United States	139	139.0
14959	United States	149	119.2
14960	United States	189	189.0
14961	United States	129	129.0
14963	United States	149	149.0

[5886 rows x 3 columns]

3.30 polars

```
df_pl.filter(pl.col("Year") == 2015)
```

```
shape: (4_848, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	...	---	---	---	---
str	date	str	str	f64	i64	str	f64	
54725	2015-01-01	United States	2187	...	0.0	2015	1	179
54726	2015-01-01	United States	2174	...	0.0	2015	1	169
54727	2015-01-01	United States	2240	...	0.1	2015	1	116
54728	2015-01-01	Germany	2220	...	0.0	2015	1	129.
54729	2015-01-01	United Kingdom	2199	...	0.0	2015	1	13
...	...	...	...	...	...	...	...	
59192	2015-12-31	United States	2179	...	0.5	2015	12	79
59193	2015-12-31	United States	2148	...	0.0	2015	12	13
59194	2015-12-31	Germany	2209	...	0.0	2015	12	159.
59195	2015-12-31	Germany	2238	...	0.0	2015	12	149.
59196	2015-12-31	United States	2173	...	0.0	2015	12	17

```
df_pl.filter(pl.col("Country").is_in(["United States", "Canada"]))
```

```
shape: (8_838, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	...	---	---	---	---
str	date	str	str	f64	i64	str	f64	
52390	2014-01-01	United States	2230	...	0.2	2014	1	159
52391	2014-01-01	Canada	2160	...	0.2	2014	1	119.2
52392	2014-01-01	United States	2234	...	0.0	2014	1	159
52394	2014-01-01	United States	2173	...	0.0	2014	1	159
52396	2014-01-02	Canada	2238	...	0.0	2014	1	169.0
...	...	...	...	...	...	...	...	
65770	2016-12-31	United States	2178	...	0.2	2016	12	119
65771	2016-12-31	United States	2209	...	0.0	2016	12	189
65772	2016-12-31	United States	2168	...	0.0	2016	12	129
65774	2016-12-31	United States	2181	...	0.0	2016	12	149
65775	2016-12-31	Canada	2203	...	0.3	2016	12	125.3

```
df_pl.filter(
    (pl.col("Country") == "United States") & (pl.col("Year") == 2015)
)
```

shape: (1_924, 14)

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	---	---	---	---	---
str	date	str	str	f64	i64	str	f64	
54725	2015-01-01	United States	2187	...	0.0	2015	1	179
54726	2015-01-01	United States	2174	...	0.0	2015	1	169
54727	2015-01-01	United States	2240	...	0.1	2015	1	116
54733	2015-01-02	United States	2173	...	0.0	2015	1	179
54738	2015-01-02	United States	2226	...	0.0	2015	1	199
...	...	...	...	...	...	...	...	...
59190	2015-12-31	United States	2213	...	0.3	2015	12	104
59191	2015-12-31	United States	2211	...	0.0	2015	12	129
59192	2015-12-31	United States	2179	...	0.5	2015	12	79.
59193	2015-12-31	United States	2148	...	0.0	2015	12	139
59196	2015-12-31	United States	2173	...	0.0	2015	12	179

```
df_pl.filter(
    (pl.col("Country") == "United States") & (pl.col("Year").is_in([2015, 2016])
)
```

shape: (4_859, 14)

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	---	---	---	---	---
str	date	str	str	f64	i64	str	f64	
54725	2015-01-01	United States	2187	...	0.0	2015	1	179
54726	2015-01-01	United States	2174	...	0.0	2015	1	169
54727	2015-01-01	United States	2240	...	0.1	2015	1	116
54733	2015-01-02	United States	2173	...	0.0	2015	1	179
54738	2015-01-02	United States	2226	...	0.0	2015	1	199
...	...	...	...	...	...	...	...	...
65767	2016-12-31	United States	2147	...	0.0	2016	12	139
65770	2016-12-31	United States	2178	...	0.2	2016	12	119

65771	2016-12-31	United States	2209	...	0.0	2016	12	189
65772	2016-12-31	United States	2168	...	0.0	2016	12	129
65774	2016-12-31	United States	2181	...	0.0	2016	12	149

```
df_pl.filter(
  (pl.col("Country").is_in(["United States", "Canada"])) & (pl.col("Year")=
)
```

shape: (1_649, 14)

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	---	---	---	---	
str	date	str	str	f64	i64	str	f64	
52390	2014-01-01	United States	2230	...	0.2	2014	1	159
52391	2014-01-01	Canada	2160	...	0.2	2014	1	119.2
52392	2014-01-01	United States	2234	...	0.0	2014	1	159
52394	2014-01-01	United States	2173	...	0.0	2014	1	159
52396	2014-01-02	Canada	2238	...	0.0	2014	1	169.0
...	...	...	...	...	...	...	...	
54713	2014-12-30	United States	2241	...	0.0	2014	12	189
54718	2014-12-31	Canada	2160	...	0.2	2014	12	151.2
54719	2014-12-31	United States	2189	...	0.0	2014	12	199
54721	2014-12-31	Canada	2219	...	0.5	2014	12	74.5
54722	2014-12-31	United States	2173	...	0.3	2014	12	118

3.30.1 2nd Verb - arrange () Function

In arrange functions, we order the rows of a data frame by the values of given columns. It is like sorting or ordering the data.

3.31 dplyr

```
df |>
  arrange (DATE)
```

```
# A tibble: 14,967 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr>        <chr>    <chr> <chr>    <dbl> <chr>
1     52389 2014-01-01 United Kingd~ 2152      UK2   Male      11  44
2     52390 2014-01-01 United States 2230      US15  Male     11.5 44-45
3     52391 2014-01-01 Canada      2160      CAN7  Male      9.5 42-43
4     52392 2014-01-01 United States 2234      US6   Female     9.5 40
5     52393 2014-01-01 United Kingd~ 2222      UK4   Female     9  39-40
6     52394 2014-01-01 United States 2173      US15  Male     10.5 43-44
7     52395 2014-01-02 Germany      2200      GER2  Female     9  39-40
8     52396 2014-01-02 Canada      2238      CAN5  Male      10  43
9     52397 2014-01-02 United States 2191      US13  Male     10.5 43-44
10    52398 2014-01-02 United Kingd~ 2237      UK1   Female     9  39-40
# i 14,957 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>
```

```
df |>
  arrange(desc (DATE))
```

```
# A tibble: 14,967 x 14
  INVOICENO DATE      COUNTRY      PRODUCTID SHOP GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr>        <chr>    <chr> <chr>    <dbl> <chr>
1     65765 2016-12-31 Canada      2199      CAN5  Male      11  44
2     65766 2016-12-31 United Kingd~ 2202      UK1   Male      9.5 42-43
3     65767 2016-12-31 United States 2147      US15  Male      9.5 42-43
4     65768 2016-12-31 Germany      2205      GER1  Female     7.5 38
5     65769 2016-12-31 Germany      2210      GER2  Male     10.5 43-44
6     65770 2016-12-31 United States 2178      US13  Female     8  38-39
7     65771 2016-12-31 United States 2209      US15  Male     9  42
8     65772 2016-12-31 United States 2168      US13  Male     8  41
9     65773 2016-12-31 United Kingd~ 2154      UK2   Male      9.5 42-43
10    65774 2016-12-31 United States 2181      US12  Female    12  42-43
# i 14,957 more rows
# i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
#   YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>
```

```
df |>
  arrange(MONTH, SALEPRICE)
```

```
# A tibble: 14,967 x 14
```

	INVOICENO	DATE	COUNTRY	PRODUCTID	SHOP	GENDER	SIZE_US	SIZE_EUROPE
	<dbl>	<date>	<chr>	<chr>	<chr>	<chr>	<dbl>	<chr>
1	52414	2014-01-05	Germany	2239	GER2	Female	8.5	39
2	52533	2014-01-25	Canada	2151	CAN5	Female	9	39-40
3	52539	2014-01-26	United Kingd~	2227	UK1	Female	7.5	38
4	52548	2014-01-27	United Kingd~	2224	UK3	Male	8.5	41-42
5	54734	2015-01-02	Germany	2222	GER3	Female	7.5	38
6	54772	2015-01-06	Germany	2159	GER1	Male	12	45
7	54864	2015-01-16	Germany	2239	GER2	Female	7.5	38
8	54989	2015-01-28	Canada	2152	CAN6	Female	8	38-39
9	59220	2016-01-03	Canada	2202	CAN7	Male	10.5	43-44
10	59242	2016-01-04	United States	2231	US9	Female	9	39-40

i 14,957 more rows
i 6 more variables: SIZE_UK <dbl>, UNITPRICE <dbl>, DISCOUNT <dbl>,
YEAR <dbl>, MONTH <dbl>, SALEPRICE <dbl>

3.32 pandas

```
df_pd.sort_values(by=['DATE'])
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
0	52389	1/1/2014	United Kingdom	...	2014	1	159.0
1	52390	1/1/2014	United States	...	2014	1	159.2
2	52391	1/1/2014	Canada	...	2014	1	119.2
3	52392	1/1/2014	United States	...	2014	1	159.0
4	52393	1/1/2014	United Kingdom	...	2014	1	159.0
...	...	...	...	...	...	...	...
12785	63807	9/9/2016	Germany	...	2016	9	189.0
12786	63808	9/9/2016	Germany	...	2016	9	129.0
12787	63809	9/9/2016	United States	...	2016	9	179.0
12789	63811	9/9/2016	United States	...	2016	9	125.3
12779	63802	9/9/2016	United States	...	2016	9	125.1

[14967 rows x 14 columns]

```
df_pd.sort_values(by=['DATE'], ascending = False)
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
12778	63802	9/9/2016	United States	...	2016	9	139.0

12784	63806	9/9/2016	Germany	...	2016	9	139.0
12773	63797	9/9/2016	Germany	...	2016	9	90.3
12772	63796	9/9/2016	Germany	...	2016	9	111.3
12771	63795	9/9/2016	Germany	...	2016	9	152.1
...	...	...	...	...	...	...	...
5	52394	1/1/2014	United States	...	2014	1	159.0
4	52393	1/1/2014	United Kingdom	...	2014	1	159.0
3	52392	1/1/2014	United States	...	2014	1	159.0
2	52391	1/1/2014	Canada	...	2014	1	119.2
0	52389	1/1/2014	United Kingdom	...	2014	1	159.0

[14967 rows x 14 columns]

```
df_pd.sort_values(by=['MONTH', 'SALEPRICE'])
```

	INVOICENO	DATE	COUNTRY	...	YEAR	MONTH	SALEPRICE
33	52414	1/5/2014	Germany	...	2014	1	64.5
177	52533	1/25/2014	Canada	...	2014	1	64.5
185	52539	1/26/2014	United Kingdom	...	2014	1	64.5
194	52548	1/27/2014	United Kingdom	...	2014	1	64.5
2762	54734	1/2/2015	Germany	...	2015	1	64.5
...	...	...	...	...	...	...	...
13245	64219	9/29/2016	United Kingdom	...	2016	9	199.0
13246	64220	9/29/2016	United States	...	2016	9	199.0
13248	64222	9/29/2016	United States	...	2016	9	199.0
13251	64224	9/29/2016	Germany	...	2016	9	199.0
13272	64244	9/30/2016	United States	...	2016	9	199.0

[14967 rows x 14 columns]

3.33 polars

```
df_pl.sort("Date") # default asecending
```

shape: (14_967, 14)

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	S
---	---	---	---	---	---	---	---	---
str	date	str	str	f64	i64	str	f64	

52389	2014-01-01	United Kingdom	2152	...	0.0	2014	1	159.0
52390	2014-01-01	United States	2230	...	0.2	2014	1	159.0
52391	2014-01-01	Canada	2160	...	0.2	2014	1	119.2
52392	2014-01-01	United States	2234	...	0.0	2014	1	159.0
52393	2014-01-01	United Kingdom	2222	...	0.0	2014	1	159.0
...	...	...	...	...	...	...	...	...
65773	2016-12-31	United Kingdom	2154	...	0.0	2016	12	13.0
65774	2016-12-31	United States	2181	...	0.0	2016	12	14.0
65775	2016-12-31	Canada	2203	...	0.3	2016	12	125.3
65776	2016-12-31	Germany	2231	...	0.0	2016	12	199.0
65777	2016-12-31	Germany	2156	...	0.1	2016	12	125.0

```
df_pl.sort(['Month', 'SalePrice'])
```

```
shape: (14_967, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	SalePrice
---	---	---	---	---	---	---	---	---
str	date	str	str	f64	i64	str	f64	f64
59355	2016-01-12	Canada	2196	...	0.5	2016	1	64.5
59274	2016-01-07	Canada	2216	...	0.5	2016	1	64.5
59307	2016-01-09	Canada	2216	...	0.5	2016	1	64.5
52414	2014-01-05	Germany	2239	...	0.5	2014	1	64.5
59290	2016-01-08	Canada	2216	...	0.5	2016	1	64.5
...	...	...	...	...	...	...	...	...
63870	2016-09-12	United States	2220	...	0.0	2016	9	199.0
54015	2014-09-10	Germany	2162	...	0.0	2014	9	199.0
57738	2015-09-17	Germany	2198	...	0.0	2015	9	199.0
57733	2015-09-16	United States	2175	...	0.0	2015	9	199.0
54088	2014-09-22	Germany	2172	...	0.0	2014	9	199.0

```
df_pl.sort(['Month', 'SalePrice'], descending = [True, False])
```

```
shape: (14_967, 14)
```

InvoiceNo	Date	Country	ProductID	...	Discount	Year	Month	SalePrice
---	---	---	---	---	---	---	---	---

str	date	str	str	f64	i64	str	f64
64100	2016-09-23	United Kingdom	2233	...	0.5	2016	9
63793	2016-09-08	Canada	2202	...	0.5	2016	9
57757	2015-09-19	Canada	2196	...	0.5	2015	9
64006	2016-09-19	Germany	2209	...	0.5	2016	9
57532	2015-09-02	Canada	2202	...	0.5	2015	9
...	...	...	...	...	...	...	...
54875	2015-01-17	United States	2218	...	0.0	2015	1
59654	2016-01-30	United States	2224	...	0.0	2016	1
59470	2016-01-18	Canada	2208	...	0.0	2016	1
54859	2015-01-15	Germany	2198	...	0.0	2015	1
59464	2016-01-18	Germany	2170	...	0.0	2016	1

3.33.1 3rd Verb - select () Function

Select functions help to select or obtain columns from the data frame. When there are a lot of columns in our dataset, select functions become very useful.

3.34 dplyr

```
df |> select (DATE, UNITPRICE, DISCOUNT)
```

```
# A tibble: 14,967 x 3
  DATE          UNITPRICE DISCOUNT
  <date>          <dbl>     <dbl>
1 2014-01-01      159         0
2 2014-01-01      199        0.2
3 2014-01-01      149        0.2
4 2014-01-01      159         0
5 2014-01-01      159         0
6 2014-01-01      159         0
7 2014-01-02      179         0
8 2014-01-02      169         0
9 2014-01-02      139         0
10 2014-01-02      149         0
# i 14,957 more rows
```

```
df |> select(1:2, 5:8)
```

```
# A tibble: 14,967 x 6
  INVOICENO DATE      SHOP GENDER SIZE_US SIZE_EUROPE
    <dbl> <date>    <chr> <chr>   <dbl> <chr>
1     52389 2014-01-01 UK2    Male     11    44
2     52390 2014-01-01 US15   Male    11.5  44-45
3     52391 2014-01-01 CAN7   Male     9.5  42-43
4     52392 2014-01-01 US6    Female   9.5   40
5     52393 2014-01-01 UK4    Female   9     39-40
6     52394 2014-01-01 US15   Male    10.5  43-44
7     52395 2014-01-02 GER2   Female   9     39-40
8     52396 2014-01-02 CAN5   Male    10     43
9     52397 2014-01-02 US13   Male    10.5  43-44
10    52398 2014-01-02 UK1    Female   9     39-40
# i 14,957 more rows
```

```
df |>
  select(starts_with('SIZE'))
```

```
# A tibble: 14,967 x 3
  SIZE_US SIZE_EUROPE SIZE_UK
    <dbl> <chr>         <dbl>
1     11    44           10.5
2    11.5  44-45         11
3     9.5  42-43         9
4     9.5   40        7.5
5     9    39-40         7
6    10.5  43-44        10
7     9    39-40         7
8     10    43        9.5
9    10.5  43-44        10
10     9    39-40         7
# i 14,957 more rows
```

```
df |>
  select(ends_with('PRICE'))
```

```
# A tibble: 14,967 x 2
  UNITPRICE SALEPRICE
```

```

      <dbl>      <dbl>
1      159      159
2      199      159.
3      149      119.
4      159      159
5      159      159
6      159      159
7      179      179
8      169      169
9      139      139
10     149      149
# i 14,957 more rows

```

```

df |>
  select(contains("_"))

```

```

# A tibble: 14,967 x 3
  SIZE_US SIZE_EUROPE SIZE_UK
  <dbl> <chr>      <dbl>
1     11    44        10.5
2    11.5 44-45         11
3     9.5 42-43         9
4     9.5 40         7.5
5     9    39-40         7
6    10.5 43-44        10
7     9    39-40         7
8    10    43        9.5
9    10.5 43-44        10
10     9    39-40         7
# i 14,957 more rows

```

```

df |>
  select(matches("SIZE"))

```

```

# A tibble: 14,967 x 3
  SIZE_US SIZE_EUROPE SIZE_UK
  <dbl> <chr>      <dbl>
1     11    44        10.5
2    11.5 44-45         11
3     9.5 42-43         9
4     9.5 40         7.5

```

```

5      9    39-40      7
6     10.5 43-44     10
7      9    39-40      7
8     10    43       9.5
9     10.5 43-44     10
10     9    39-40      7
# i 14,957 more rows

```

```

df |>
  select(matches("PRICE$"))

```

```

# A tibble: 14,967 x 2
  UNITPRICE SALEPRICE
    <dbl>     <dbl>
1     159     159
2     199    159.
3     149    119.
4     159     159
5     159     159
6     159     159
7     179     179
8     169     169
9     139     139
10    149     149
# i 14,957 more rows

```

```

# starts with letter S
df |>
  select(matches("^S"))

```

```

# A tibble: 14,967 x 5
  SHOP  SIZE_US SIZE_EUROPE SIZE_UK SALEPRICE
  <chr>  <dbl>  <chr>         <dbl>     <dbl>
1 UK2    11    44          10.5     159
2 US15   11.5  44-45        11      159.
3 CAN7    9.5  42-43         9      119.
4 US6    9.5   40         7.5     159
5 UK4     9   39-40         7      159
6 US15   10.5  43-44        10      159
7 GER2    9   39-40         7      179
8 CAN5   10   43         9.5     169

```

```

  9 US13      10.5 43-44      10      139
10 UK1       9   39-40      7       149
# i 14,957 more rows

```

```

df |>
  select(where(is.character))

```

```

# A tibble: 14,967 x 5
  COUNTRY      PRODUCTID SHOP  GENDER SIZE_EUROPE
  <chr>      <chr>    <chr> <chr>  <chr>
1 United Kingdom 2152    UK2   Male   44
2 United States  2230    US15  Male   44-45
3 Canada        2160    CAN7  Male   42-43
4 United States  2234    US6   Female 40
5 United Kingdom 2222    UK4   Female 39-40
6 United States  2173    US15  Male   43-44
7 Germany       2200    GER2  Female 39-40
8 Canada        2238    CAN5  Male   43
9 United States  2191    US13  Male   43-44
10 United Kingdom 2237    UK1   Female 39-40
# i 14,957 more rows

```

```

df |>
  select(where(is.numeric))

```

```

# A tibble: 14,967 x 8
  INVOICENO SIZE_US SIZE_UK UNITPRICE DISCOUNT YEAR MONTH SALEPRICE
  <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <dbl> <dbl>    <dbl>
1    52389     11    10.5      159      0   2014     1      159
2    52390    11.5     11      199     0.2  2014     1     159.
3    52391     9.5     9       149     0.2  2014     1     119.
4    52392     9.5     7.5      159      0   2014     1      159
5    52393     9       7       159      0   2014     1      159
6    52394    10.5     10      159      0   2014     1      159
7    52395     9       7       179      0   2014     1      179
8    52396    10      9.5      169      0   2014     1      169
9    52397    10.5     10      139      0   2014     1      139
10   52398     9       7       149      0   2014     1      149
# i 14,957 more rows

```

```
df |>
  select(MONTH, YEAR, everything())
```

```
# A tibble: 14,967 x 14
  MONTH YEAR INVOICENO DATE      COUNTRY      PRODUCTID SHOP GENDER SIZE_US
  <dbl> <dbl>   <dbl> <date>   <chr>      <chr>      <chr> <chr>   <dbl>
1     1   2014     52389 2014-01-01 United Kingd~ 2152      UK2   Male     11
2     1   2014     52390 2014-01-01 United States 2230      US15  Male    11.5
3     1   2014     52391 2014-01-01 Canada      2160      CAN7  Male     9.5
4     1   2014     52392 2014-01-01 United States 2234      US6   Female   9.5
5     1   2014     52393 2014-01-01 United Kingd~ 2222      UK4   Female    9
6     1   2014     52394 2014-01-01 United States 2173      US15  Male    10.5
7     1   2014     52395 2014-01-02 Germany      2200      GER2  Female    9
8     1   2014     52396 2014-01-02 Canada      2238      CAN5  Male     10
9     1   2014     52397 2014-01-02 United States 2191      US13  Male    10.5
10    1   2014     52398 2014-01-02 United Kingd~ 2237      UK1   Female    9
# i 14,957 more rows
# i 5 more variables: SIZE_EUROPE <chr>, SIZE_UK <dbl>, UNITPRICE <dbl>,
#   DISCOUNT <dbl>, SALEPRICE <dbl>
```

```
# any_of () vs all_of ()
df |>
  select(any_of(c("PRICE", "SIZE")))
```

```
# A tibble: 14,967 x 0
```

```
# Dropping columns
df |>
  select(-DATE)
```

```
# A tibble: 14,967 x 13
  INVOICENO COUNTRY      PRODUCTID SHOP GENDER SIZE_US SIZE_EUROPE SIZE_UK
  <dbl> <chr>      <chr>      <chr> <chr>   <dbl> <chr>      <dbl>
1     52389 United Kingdom 2152      UK2   Male     11  44         10.5
2     52390 United States 2230      US15  Male    11.5 44-45       11
3     52391 Canada      2160      CAN7  Male     9.5 42-43        9
4     52392 United States 2234      US6   Female   9.5 40         7.5
5     52393 United Kingdom 2222      UK4   Female    9  39-40        7
6     52394 United States 2173      US15  Male    10.5 43-44       10
7     52395 Germany      2200      GER2  Female    9  39-40        7
```

```

8      52396 Canada      2238      CAN5 Male      10  43      9.5
9      52397 United States 2191      US13 Male      10.5 43-44      10
10     52398 United Kingdom 2237      UK1  Female      9  39-40      7
# i 14,957 more rows
# i 5 more variables: UNITPRICE <dbl>, DISCOUNT <dbl>, YEAR <dbl>, MONTH <dbl>,
#      SALEPRICE <dbl>

```

3.35 pandas

```
df_pd['DATE']
```

```

0          1/1/2014
1          1/1/2014
2          1/1/2014
3          1/1/2014
4          1/1/2014
...
14962     12/31/2016
14963     12/31/2016
14964     12/31/2016
14965     12/31/2016
14966     12/31/2016
Name: DATE, Length: 14967, dtype: object

```

```
df_pd[['DATE', 'UNITPRICE']]
```

```

          DATE  UNITPRICE
0      1/1/2014         159
1      1/1/2014         199
2      1/1/2014         149
3      1/1/2014         159
4      1/1/2014         159
...
14962  12/31/2016         139
14963  12/31/2016         149
14964  12/31/2016         179
14965  12/31/2016         199
14966  12/31/2016         139

```

```
[14967 rows x 2 columns]
```



```
df_pd.loc[:, ['DATE', 'UNITPRICE']]
```

	DATE	UNITPRICE
0	1/1/2014	159
1	1/1/2014	199
2	1/1/2014	149
3	1/1/2014	159
4	1/1/2014	159
...	...	...
14962	12/31/2016	139
14963	12/31/2016	149
14964	12/31/2016	179
14965	12/31/2016	199
14966	12/31/2016	139

[14967 rows x 2 columns]

```
df_pd.iloc[:, 5:8]
```

	GENDER	SIZE_US	SIZE_EUROPE
0	Male	11.0	44
1	Male	11.5	44-45
2	Male	9.5	42-43
3	Female	9.5	40
4	Female	9.0	39-40
...	...	...	...
14962	Male	9.5	42-43
14963	Female	12.0	42-43
14964	Male	10.5	43-44
14965	Female	9.5	40
14966	Female	6.5	37

[14967 rows x 3 columns]

```
df_pd.iloc[:, [3, 5, 8]]
```

	PRODUCTID	GENDER	SIZE_UK
0	2152	Male	10.5
1	2230	Male	11.0
2	2160	Male	9.0

3	2234	Female	7.5
4	2222	Female	7.0
...	...	...	...
14962	2154	Male	9.0
14963	2181	Female	10.0
14964	2203	Male	10.0
14965	2231	Female	7.5
14966	2156	Female	4.5

[14967 rows x 3 columns]

```
df_pd.filter(['YEAR','SALEPRICE', 'DISCOUNT', 'UNITPRICE'])
```

	YEAR	SALEPRICE	DISCOUNT	UNITPRICE
0	2014	159.0	0.0	159
1	2014	159.2	0.2	199
2	2014	119.2	0.2	149
3	2014	159.0	0.0	159
4	2014	159.0	0.0	159
...	...	...	...	...
14962	2016	139.0	0.0	139
14963	2016	149.0	0.0	149
14964	2016	125.3	0.3	179
14965	2016	199.0	0.0	199
14966	2016	125.1	0.1	139

[14967 rows x 4 columns]

```
df_pd.filter(['YEAR','SALEPRICE', 'DISCOUNT', 'UNITPRICE'])
```

	YEAR	SALEPRICE	DISCOUNT	UNITPRICE
0	2014	159.0	0.0	159
1	2014	159.2	0.2	199
2	2014	119.2	0.2	149
3	2014	159.0	0.0	159
4	2014	159.0	0.0	159
...	...	...	...	...
14962	2016	139.0	0.0	139
14963	2016	149.0	0.0	149
14964	2016	125.3	0.3	179
14965	2016	199.0	0.0	199

```
14966    2016      125.1      0.1      139
```

```
[14967 rows x 4 columns]
```

```
#RegularExpression(Regex)
df_pd.filter(regex = "PRICE$") #Ends with Price
```

	UNITPRICE	SALEPRICE
0	159	159.0
1	199	159.2
2	149	119.2
3	159	159.0
4	159	159.0
...	...	...
14962	139	139.0
14963	149	149.0
14964	179	125.3
14965	199	199.0
14966	139	125.1

```
[14967 rows x 2 columns]
```

```
df_pd.filter(regex = "^SIZE") #Starts with SIZE
```

```
Empty DataFrame
```

```
Columns: []
```

```
Index: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
```

```
[14967 rows x 0 columns]
```

```
df_pd.filter(regex = "PRICE") #Contains the word Price
```

	UNITPRICE	SALEPRICE
0	159	159.0
1	199	159.2
2	149	119.2
3	159	159.0
4	159	159.0
...	...	...
14962	139	139.0

14963	149	149.0
14964	179	125.3
14965	199	199.0
14966	139	125.1

[14967 rows x 2 columns]

```
df_pd.select_dtypes('object')
```

	INVOICENO	DATE	COUNTRY	...	GENDER	SIZE	EUROPE	MONTH
0	52389	1/1/2014	United Kingdom	...	Male	44		1
1	52390	1/1/2014	United States	...	Male	44-45		1
2	52391	1/1/2014	Canada	...	Male	42-43		1
3	52392	1/1/2014	United States	...	Female	40		1
4	52393	1/1/2014	United Kingdom	...	Female	39-40		1
...	...	...	...	...	...	...	...	...
14962	65773	12/31/2016	United Kingdom	...	Male	42-43		12
14963	65774	12/31/2016	United States	...	Female	42-43		12
14964	65775	12/31/2016	Canada	...	Male	43-44		12
14965	65776	12/31/2016	Germany	...	Female	40		12
14966	65777	12/31/2016	Germany	...	Female	37		12

[14967 rows x 8 columns]

```
df_pd.select_dtypes('int')
```

	UNITPRICE	YEAR
0	159	2014
1	199	2014
2	149	2014
3	159	2014
4	159	2014
...	...	...
14962	139	2016
14963	149	2016
14964	179	2016
14965	199	2016
14966	139	2016

[14967 rows x 2 columns]

```
df_pd.loc[:,df_pd.columns.str.startswith('SIZE')]
```

	SIZE_US	SIZE_EUROPE	SIZE_UK
0	11.0	44	10.5
1	11.5	44-45	11.0
2	9.5	42-43	9.0
3	9.5	40	7.5
4	9.0	39-40	7.0
...	...	...	...
14962	9.5	42-43	9.0
14963	12.0	42-43	10.0
14964	10.5	43-44	10.0
14965	9.5	40	7.5
14966	6.5	37	4.5

[14967 rows x 3 columns]

```
df_pd.loc[:,df_pd.columns.str.contains('PRICE')]
```

	UNITPRICE	SALEPRICE
0	159	159.0
1	199	159.2
2	149	119.2
3	159	159.0
4	159	159.0
...	...	...
14962	139	139.0
14963	149	149.0
14964	179	125.3
14965	199	199.0
14966	139	125.1

[14967 rows x 2 columns]

```
df_pd.loc[:,df_pd.columns.str.endswith('PRICE')]
```

	UNITPRICE	SALEPRICE
0	159	159.0
1	199	159.2
2	149	119.2

```

3          159      159.0
4          159      159.0
...
14962      139      139.0
14963      149      149.0
14964      179      125.3
14965      199      199.0
14966      139      125.1

```

```
[14967 rows x 2 columns]
```

```
# Dropping columns
df_pd.drop(columns=['SIZE_EUROPE', 'SIZE_UK'], axis=1)
```

```

      INVOICENO      DATE      COUNTRY ... YEAR MONTH SALEPRICE
0          52389  1/1/2014  United Kingdom ... 2014      1      159.0
1          52390  1/1/2014   United States ... 2014      1      159.2
2          52391  1/1/2014         Canada ... 2014      1      119.2
3          52392  1/1/2014   United States ... 2014      1      159.0
4          52393  1/1/2014  United Kingdom ... 2014      1      159.0
...
14962      65773 12/31/2016  United Kingdom ... 2016     12      139.0
14963      65774 12/31/2016   United States ... 2016     12      149.0
14964      65775 12/31/2016         Canada ... 2016     12      125.3
14965      65776 12/31/2016         Germany ... 2016     12      199.0
14966      65777 12/31/2016         Germany ... 2016     12      125.1

```

```
[14967 rows x 12 columns]
```

```
# Dropping columns
df_pd.drop(columns=['SIZE_EUROPE', 'SIZE_UK'], axis=1)\
      .pipe(lambda x: x.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   INVOICENO    14967 non-null  object
1   DATE         14967 non-null  object
2   COUNTRY      14967 non-null  object

```

```

3   PRODUCTID    14967 non-null object
4   SHOP         14967 non-null object
5   GENDER       14967 non-null object
6   SIZE_US      14967 non-null float64
7   UNITPRICE    14967 non-null int64
8   DISCOUNT    14967 non-null float64
9   YEAR         14967 non-null int64
10  MONTH        14967 non-null object
11  SALEPRICE    14967 non-null float64
dtypes: float64(3), int64(2), object(7)
memory usage: 1.4+ MB

```

```

# Rearranging columns
# Sorting Alphabetically
df_pd.reindex(sorted(df_pd.columns), axis =1)

```

	COUNTRY	DATE	DISCOUNT	...	SIZE_US	UNITPRICE	YEAR
0	United Kingdom	1/1/2014	0.0	...	11.0	159	2014
1	United States	1/1/2014	0.2	...	11.5	199	2014
2	Canada	1/1/2014	0.2	...	9.5	149	2014
3	United States	1/1/2014	0.0	...	9.5	159	2014
4	United Kingdom	1/1/2014	0.0	...	9.0	159	2014
...	...	...	...	...	...	...	...
14962	United Kingdom	12/31/2016	0.0	...	9.5	139	2016
14963	United States	12/31/2016	0.0	...	12.0	149	2016
14964	Canada	12/31/2016	0.3	...	10.5	179	2016
14965	Germany	12/31/2016	0.0	...	9.5	199	2016
14966	Germany	12/31/2016	0.1	...	6.5	139	2016

[14967 rows x 14 columns]

```

# Rearranging columns
# Sorting As You Want (ASY)
col_first = ['YEAR','MONTH']
col_rest = df_pd.columns.difference(col_first, sort=False).to_list()
df_pd2 = df_pd [col_first +col_rest]
df_pd2.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 14 columns):

```

#	Column	Non-Null Count	Dtype
0	YEAR	14967 non-null	int64
1	MONTH	14967 non-null	object
2	INVOICENO	14967 non-null	object
3	DATE	14967 non-null	object
4	COUNTRY	14967 non-null	object
5	PRODUCTID	14967 non-null	object
6	SHOP	14967 non-null	object
7	GENDER	14967 non-null	object
8	SIZE_US	14967 non-null	float64
9	SIZE_EUROPE	14967 non-null	object
10	SIZE_UK	14967 non-null	float64
11	UNITPRICE	14967 non-null	int64
12	DISCOUNT	14967 non-null	float64
13	SALEPRICE	14967 non-null	float64

dtypes: float64(4), int64(2), object(8)

memory usage: 1.6+ MB

3.36 polars

```
df_pl.select(["Date", 'UnitPrice', "Discount"])
```

shape: (14_967, 3)

Date	UnitPrice	Discount
---	---	---
date	i64	f64
2014-01-01	159	0.0
2014-01-01	199	0.2
2014-01-01	149	0.2
2014-01-01	159	0.0
2014-01-01	159	0.0
...	...	...
2016-12-31	139	0.0
2016-12-31	149	0.0
2016-12-31	179	0.3
2016-12-31	199	0.0
2016-12-31	139	0.1


```
col_to_select = [c for c in df_pl.columns if c.startswith("Size")]
df_pl.select(pl.col(col_to_select))
```

shape: (14_967, 3)

Size (US)	Size (Europe)	Size (UK)
---	---	---
f64	str	f64
11.0	44	10.5
11.5	44-45	11.0
9.5	42-43	9.0
9.5	40	7.5
9.0	39-40	7.0
...	...	...
9.5	42-43	9.0
12.0	42-43	10.0
10.5	43-44	10.0
9.5	40	7.5
6.5	37	4.5

```
col_to_select = [c for c in df_pl.columns if c.endswith("Price")]
df_pl.select(pl.col(col_to_select))
```

shape: (14_967, 2)

UnitPrice	SalePrice
---	---
i64	f64
159	159.0
199	159.2
149	119.2
159	159.0
159	159.0
...	...
139	139.0
149	149.0

179	125.3
199	199.0
139	125.1

```
col_to_select = [c for c in df_pl.columns if "_" in c]
df_pl.select(pl.col(col_to_select))
```

shape: (0, 0)


```
col_to_select = [c for c in df_pl.columns if "Size" in c]
df_pl.select(pl.col(col_to_select))
```

shape: (14_967, 3)

Size (US)	Size (Europe)	Size (UK)
---	---	---
f64	str	f64
11.0	44	10.5
11.5	44-45	11.0
9.5	42-43	9.0
9.5	40	7.5
9.0	39-40	7.0
...	...	...
9.5	42-43	9.0
12.0	42-43	10.0
10.5	43-44	10.0
9.5	40	7.5
6.5	37	4.5

```
col_to_select = [c for c in df_pl.columns if c.startswith("S")]
df_pl.select(pl.col(col_to_select))
```

shape: (14_967, 5)

--	--	--	--	--

Shop	Size (US)	Size (Europe)	Size (UK)	SalePrice
---	---	---	---	---
str	f64	str	f64	f64
UK2	11.0	44	10.5	159.0
US15	11.5	44-45	11.0	159.2
CAN7	9.5	42-43	9.0	119.2
US6	9.5	40	7.5	159.0
UK4	9.0	39-40	7.0	159.0
...	...	...	...	...
UK2	9.5	42-43	9.0	139.0
US12	12.0	42-43	10.0	149.0
CAN6	10.5	43-44	10.0	125.3
GER1	9.5	40	7.5	199.0
GER1	6.5	37	4.5	125.1

```
df_pl.select(pl.col(pl.Utf8)) # all character columns
```

```
shape: (14_967, 7)
```

InvoiceNo	Country	ProductID	Shop	Gender	Size (Europe)	Month
---	---	---	---	---	---	---
str	str	str	str	str	str	
52389	United Kingdom	2152	UK2	Male	44	1
52390	United States	2230	US15	Male	44-45	1
52391	Canada	2160	CAN7	Male	42-43	1
52392	United States	2234	US6	Female	40	1
52393	United Kingdom	2222	UK4	Female	39-40	1
...	...	...	...	...	...	...
65773	United Kingdom	2154	UK2	Male	42-43	12
65774	United States	2181	US12	Female	42-43	12
65775	Canada	2203	CAN6	Male	43-44	12
65776	Germany	2231	GER1	Female	40	12
65777	Germany	2156	GER1	Female	37	12

```
df_pl.select(pl.col(pl.Int64)) # all numeric columns
```

```
shape: (14_967, 2)
```

UnitPrice	Year
---	---
i64	i64
159	2014
199	2014
149	2014
159	2014
159	2014
...	...
139	2016
149	2016
179	2016
199	2016
139	2016

```
cols = ["Month", "Year"] + [c for c in df_pl.columns if c not in ["Month", "Year"]]
df_pl.select(cols)
```

shape: (14_967, 14)

Month	Year	InvoiceNo	Date	...	Size (UK)	UnitPrice	Discount	SalePrice
---	---	---	---	...	---	---	---	---
str	i64	str	date	...	f64	i64	f64	f64
1	2014	52389	2014-01-01	...	10.5	159	0.0	159.0
1	2014	52390	2014-01-01	...	11.0	199	0.2	159.2
1	2014	52391	2014-01-01	...	9.0	149	0.2	119.2
1	2014	52392	2014-01-01	...	7.5	159	0.0	159.0
1	2014	52393	2014-01-01	...	7.0	159	0.0	159.0
...	...	...	...	...	...	...	...	...
12	2016	65773	2016-12-31	...	9.0	139	0.0	139.0
12	2016	65774	2016-12-31	...	10.0	149	0.0	149.0
12	2016	65775	2016-12-31	...	10.0	179	0.3	125.3
12	2016	65776	2016-12-31	...	7.5	199	0.0	199.0
12	2016	65777	2016-12-31	...	4.5	139	0.1	125.1

3.36.1 4th Verb - rename () Function

3.37 dplyr

```
df |>
  rename(INVOICE = INVOICENO,
         PRODUCT = PRODUCTID) |>
  glimpse()
```

Rows: 14,967

Columns: 14

```
$ INVOICE      <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE         <date> 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-0~
$ COUNTRY      <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCT      <chr> "2152", "2230", "2160", "2234", "2222", "2173", "2200", "2~
$ SHOP         <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~
$ GENDER       <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US      <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE  <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK      <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE    <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT    <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR         <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE    <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~
```

3.38 pandas

```
(df_pd.rename(columns = {"PRODUCTID": "PRODUCT", "INVOICENO": "INVOICE"})
  .pipe(lambda x: x.info())
)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 14967 entries, 0 to 14966
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	INVOICE	14967 non-null	object
1	DATE	14967 non-null	object

```

2   COUNTRY      14967 non-null  object
3   PRODUCT      14967 non-null  object
4   SHOP         14967 non-null  object
5   GENDER       14967 non-null  object
6   SIZE_US      14967 non-null  float64
7   SIZE_EUROPE  14967 non-null  object
8   SIZE_UK      14967 non-null  float64
9   UNITPRICE    14967 non-null  int64
10  DISCOUNT    14967 non-null  float64
11  YEAR         14967 non-null  int64
12  MONTH        14967 non-null  object
13  SALEPRICE    14967 non-null  float64
dtypes: float64(4), int64(2), object(8)
memory usage: 1.6+ MB

```

3.39 polars

```

df_pl = df_pl.rename({"ProductID": "PRODUCTID", "InvoiceNo": "INVOICE"})
df_pl.schema

```

```
Schema([('INVOICE', String), ('Date', Date), ('Country', String), ('PRODUCTID',
```

3.39.1 5th Verb - mutate () Function

3.40 dplyr

```

df |>
  mutate(NECOLUMN = 5,
         SALESPRICE2 = UNITPRICE*(1-DISCOUNT)) |>
  glimpse()

```

```
Rows: 14,967
```

```
Columns: 16
```

```

$ INVOICENO   <dbl> 52389, 52390, 52391, 52392, 52393, 52394, 52395, 52396, 52~
$ DATE       <date> 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-01, 2014-01-0~
$ COUNTRY    <chr> "United Kingdom", "United States", "Canada", "United State~
$ PRODUCTID  <chr> "2152", "2230", "2160", "2234", "2222", "2173", "2200", "2~
$ SHOP       <chr> "UK2", "US15", "CAN7", "US6", "UK4", "US15", "GER2", "CAN5~

```

```

$ GENDER      <chr> "Male", "Male", "Male", "Female", "Female", "Male", "Femal~
$ SIZE_US     <dbl> 11.0, 11.5, 9.5, 9.5, 9.0, 10.5, 9.0, 10.0, 10.5, 9.0, 10.~
$ SIZE_EUROPE <chr> "44", "44-45", "42-43", "40", "39-40", "43-44", "39-40", "~
$ SIZE_UK     <dbl> 10.5, 11.0, 9.0, 7.5, 7.0, 10.0, 7.0, 9.5, 10.0, 7.0, 9.5,~
$ UNITPRICE   <dbl> 159, 199, 149, 159, 159, 159, 179, 169, 139, 149, 129, 169~
$ DISCOUNT   <dbl> 0.0, 0.2, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.1~
$ YEAR        <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
$ MONTH       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ SALEPRICE   <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~
$ NECOLUMN    <dbl> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5~
$ SALESPRICE2 <dbl> 159.0, 159.2, 119.2, 159.0, 159.0, 159.0, 179.0, 169.0, 13~

```

3.41 pandas

```

df_pd['NEWCOLUMN'] = 5
df_pd.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICENO        14967 non-null  object
1   DATE             14967 non-null  object
2   COUNTRY          14967 non-null  object
3   PRODUCTID        14967 non-null  object
4   SHOP             14967 non-null  object
5   GENDER           14967 non-null  object
6   SIZE_US          14967 non-null  float64
7   SIZE_EUROPE      14967 non-null  object
8   SIZE_UK          14967 non-null  float64
9   UNITPRICE        14967 non-null  int64
10  DISCOUNT        14967 non-null  float64
11  YEAR             14967 non-null  int64
12  MONTH            14967 non-null  object
13  SALEPRICE        14967 non-null  float64
14  NEWCOLUMN        14967 non-null  int64
dtypes: float64(4), int64(3), object(8)
memory usage: 1.7+ MB

```

```
df_pd.drop(columns = ['NEWCOLUMN'], axis = 1, inplace = True)
df_pd.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICENO        14967 non-null  object
1   DATE             14967 non-null  object
2   COUNTRY          14967 non-null  object
3   PRODUCTID        14967 non-null  object
4   SHOP             14967 non-null  object
5   GENDER           14967 non-null  object
6   SIZE_US          14967 non-null  float64
7   SIZE_EUROPE      14967 non-null  object
8   SIZE_UK          14967 non-null  float64
9   UNITPRICE        14967 non-null  int64
10  DISCOUNT        14967 non-null  float64
11  YEAR             14967 non-null  int64
12  MONTH            14967 non-null  object
13  SALEPRICE        14967 non-null  float64
dtypes: float64(4), int64(2), object(8)
memory usage: 1.6+ MB
```

```
df_pd['SALEPRICE2']=df_pd['UNITPRICE']*(1-df_pd['DISCOUNT'])
df_pd.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14967 entries, 0 to 14966
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   INVOICENO        14967 non-null  object
1   DATE             14967 non-null  object
2   COUNTRY          14967 non-null  object
3   PRODUCTID        14967 non-null  object
4   SHOP             14967 non-null  object
5   GENDER           14967 non-null  object
6   SIZE_US          14967 non-null  float64
7   SIZE_EUROPE      14967 non-null  object
```



```

8  SIZE_UK      14967 non-null float64
9  UNITPRICE    14967 non-null int64
10 DISCOUNT    14967 non-null float64
11 YEAR         14967 non-null int64
12 MONTH        14967 non-null object
13 SALEPRICE     14967 non-null float64
14 SALEPRICE2    14967 non-null float64
dtypes: float64(5), int64(2), object(8)
memory usage: 1.7+ MB

```

```

# Using the assign() function
(df_pd[['PRODUCTID', 'UNITPRICE', 'DISCOUNT']]\
  .assign(SALEPRICE3 =lambda x: x.UNITPRICE*(1-x.DISCOUNT)) \
  .head(5)
)

```

	PRODUCTID	UNITPRICE	DISCOUNT	SALEPRICE3
0	2152	159	0.0	159.0
1	2230	199	0.2	159.2
2	2160	149	0.2	119.2
3	2234	159	0.0	159.0
4	2222	159	0.0	159.0

3.42 polars

```

df_pl.with_columns(
    NEWCOLUMN = 5,
    SALEPRICE2 = pl.col('UnitPrice')*(1-pl.col('Discount'))
)

```

shape: (14_967, 16)

INVOICE	Date	Country	PRODUCTID	...	Month	SalePrice	NEWCOLUMN	S
---	---	---	---	...	---	---	---	
str	date	str	str	...	str	f64	i32	f64
52389	2014-01-01	United Kingdom	2152	...	1	159.0	5	159.0
52390	2014-01-01	United Kingdom	2230	...	1	159.2	5	159.2

		States							
52391	2014-01-01	Canada	2160	...	1	119.2	5	119.2	
52392	2014-01-01	United	2234	...	1	159.0	5	159.0	
		States							
52393	2014-01-01	United	2222	...	1	159.0	5	159.0	
		Kingdom							
...	...	...	...	...	...	...	...		
65773	2016-12-31	United	2154	...	12	139.0	5	139.0	
		Kingdom							
65774	2016-12-31	United	2181	...	12	149.0	5	149.0	
		States							
65775	2016-12-31	Canada	2203	...	12	125.3	5	125.3	
65776	2016-12-31	Germany	2231	...	12	199.0	5	199.0	
65777	2016-12-31	Germany	2156	...	12	125.1	5	125.1	

3.42.1 6th Verbs - group_by () and summarize () Functions

Figure ?? presents Split Apply Combine principle in group_by () and summarize () functions.

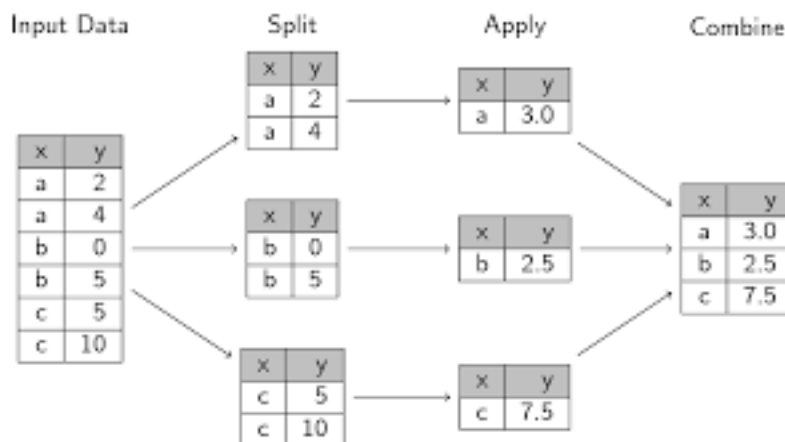


Figure 3.1: Split Apply Combine Principle

3.43 dplyr

```
df |>
  group_by(COUNTRY) |>
  summarize (AVGPRICE = mean(UNITPRICE, na.rm = TRUE))
```

```
# A tibble: 4 x 2
  COUNTRY      AVGPRICE
  <chr>        <dbl>
1 Canada      165.
2 Germany     164.
3 United Kingdom 166.
4 United States 163.
```

```
df |>
  group_by(COUNTRY) |>
  summarize (AVGPRICE = mean(UNITPRICE, na.rm = TRUE),
    AVGSALEPRICE = mean (SALEPRICE, na.rm = TRUE))
```

```
# A tibble: 4 x 3
  COUNTRY      AVGPRICE AVGSALEPRICE
  <chr>        <dbl>        <dbl>
1 Canada      165.          144.
2 Germany     164.          144.
3 United Kingdom 166.          146.
4 United States 163.          144.
```

```
# Summary Statistics
df %>%
  select(UNITPRICE, SALEPRICE) %>%
  summarize(across(where(is.numeric),
    .fns = list(N = ~length(.),
      Mean = mean,
      Std = sd,
      Median = median,
      P25 = ~quantile(., 0.25),
      P75 = ~quantile(., 0.75)
    )
  )) %>%
  pivot_longer(everything(), names_sep='_', names_to=c('variable', '.value'))
```

```
# A tibble: 2 x 7
```

	variable	N	Mean	Std	Median	P25	P75
	<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	UNITPRICE	14967	164.	22.9	159	149	179
2	SALEPRICE	14967	144.	35.2	149	125.	169

3.44 pandas

```
df_pd.groupby(['COUNTRY'])['UNITPRICE'].mean()
```

```
COUNTRY
Canada          164.691057
Germany          164.163934
United Kingdom   165.614853
United States    163.490316
Name: UNITPRICE, dtype: float64
```

```
df_pd.groupby(['COUNTRY'])[['UNITPRICE', 'SALEPRICE']].mean()
```

```
          UNITPRICE  SALEPRICE
COUNTRY
Canada          164.691057    144.228963
Germany          164.163934    143.574658
United Kingdom   165.614853    145.505872
United States    163.490316    143.727421
```

```
df_pd.groupby(['COUNTRY'])[['UNITPRICE', 'SALEPRICE']] \
    .agg(np.mean)
```

```
          UNITPRICE  SALEPRICE
COUNTRY
Canada          164.691057    144.228963
Germany          164.163934    143.574658
United Kingdom   165.614853    145.505872
United States    163.490316    143.727421
```

```
df_pd.groupby(['COUNTRY'])[['UNITPRICE', 'SALEPRICE']] \
    .agg("mean")
```

	UNITPRICE	SALEPRICE
COUNTRY		
Canada	164.691057	144.228963
Germany	164.163934	143.574658
United Kingdom	165.614853	145.505872
United States	163.490316	143.727421

```
df_pd.groupby(['COUNTRY']) [['UNITPRICE', 'SALEPRICE']] \
    .agg(AVG_UNITPRICE = ("UNITPRICE", "mean"),
        AVG_LISTPRICE = ("SALEPRICE", "mean"))
```

	AVG_UNITPRICE	AVG_LISTPRICE
COUNTRY		
Canada	164.691057	144.228963
Germany	164.163934	143.574658
United Kingdom	165.614853	145.505872
United States	163.490316	143.727421

```
df_pd.groupby(['COUNTRY']) [['UNITPRICE', 'SALEPRICE']] \
    .agg(AVG_UNITPRICE = ("UNITPRICE", "mean"),
        AVG_LISTPRICE = ("SALEPRICE", "mean"),
        TOTALN= ("SALEPRICE", "size"), # size function for n
        TOTALOBS= ("SALEPRICE", "count") # count function for n
    )
```

	AVG_UNITPRICE	AVG_LISTPRICE	TOTALN	TOTALOBS
COUNTRY				
Canada	164.691057	144.228963	2952	2952
Germany	164.163934	143.574658	4392	4392
United Kingdom	165.614853	145.505872	1737	1737
United States	163.490316	143.727421	5886	5886

```
# Defining a Function
def percentile(n):
    def percentile_(x):
        return x.quantile(n)
    percentile_.__name__ = 'percentile_{:02.0f}'.format(n*100)
    return percentile_

# Some summary statistics
```

```
df_pd[['UNITPRICE', 'SALEPRICE']] \
    .agg(['count', 'mean', 'std', 'median', percentile(0.25), percentile (0.75)]) \
    .transpose() \
    .reset_index() \
    .rename(columns = {"index": "variables", "percentile_25": "P25", "percentile_75": "P75"}) \
    .round(3)
```

	variables	N	mean	std	median	P25	P75
0	UNITPRICE	14967.0	164.171	22.941	159.0	149.0	179.0
1	SALEPRICE	14967.0	143.988	35.181	149.0	125.1	169.0

```
# Summary Statistics
agg_dict = {
    "N": "count",
    'Mean': "mean",
    "Std. Dev" : "std",
    'P25': lambda x: x.quantile(0.25),
    'Median': 'median',
    'p75': lambda x: x.quantile(0.75)
}

df_pd[['UNITPRICE', 'SALEPRICE']].agg(agg_dict)
```

3.45 polars

```
df_pl.group_by("Country").agg(pl.col("UnitPrice").mean().alias("AvgPrice"))
```

shape: (4, 2)

Country	AvgPrice
---	---
str	f64
United States	163.490316
United Kingdom	165.614853
Germany	164.163934
Canada	164.691057

```
df_pl.groupby("Country").agg(
    [pl.col("UnitPrice").mean().alias("Avgprice"),
     pl.col("SalePrice").mean().alias("AvgSalePrice")]
)
```

shape: (4, 3)

Country	Avgprice	AvgSalePrice
---	---	---
str	f64	f64
Germany	164.163934	143.574658
United States	163.490316	143.727421
Canada	164.691057	144.228963
United Kingdom	165.614853	145.505872

3.46 Reshaping Data

Before we discuss about reshaping of the data, we need to discuss about tidy format of the data. Data can come in many shapes, but not all shapes are useful for data analysis. In most cases, tidy format of the data is most useful for analysis. Therefore, if the data is untidy, we need to make it tidy first. There are three interrelated rules which make a dataset tidy (Wickham et al. 2023). These rules are given below. Figure ?? visually represents tidy principle.

1. Each variable must have its own column
2. Each observation must have its own row
3. Each value must have its own cell

For analysis, many times we need to change the format of our dataset and we call it reshaping. Data come primarily in two shapes - *wide* and *long*. Sometimes *wide* format is called “record” format and *long* format is called “stacked” format. In *wide* format data, there is one row for each subject (units of observation). Data is *long* when there are multiple rows for each subject (units of observations).

This reshaping can be two types - a) long to wide and 2) wide to long. Long-to-wide means reshaping a long data, which has many rows, into wide format, which has many variables. In wide-to-long format, we do otherwise. For analytical purpose, reshaping data is useful; so, we need to know how to do the reshaping.

Whether a given dataset (e.g., Table ??) is in wide or long format depends on our research questions (on what variables we are interested in and how we conceive of our data). If we are interested in variable

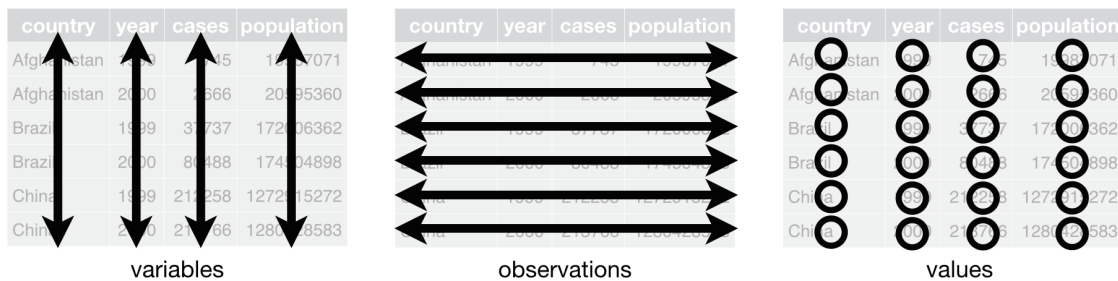


Figure 3.2: Tidy Principle

Table 3.2: Which Format - Long or Wide

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6
23	299	8.6	65	5	7
19	99	13.8	59	5	8
8	19	20.1	61	5	9
NA	194	8.6	69	5	10

Temp and Month variable is the unit of observation, then the dataset in Table ?? is in long format because Month is repeated in multiple rows.

```
airquality = airquality
examp = airquality |>
  slice(1:10)
```

3.46.1 Long-to-Wide Format

To make a long dataset to wide, we can use `pivot_wider()` function from `tidyr` package in R and `pivot()` function from `pandas` in python.

3.47 tidyr

```
tidyr::us_rent_income
```

```
# A tibble: 104 x 5
  GEOID NAME      variable estimate   moe
  <chr> <chr>      <chr>      <dbl> <dbl>
1 01     Alabama income    24476  136
2 01     Alabama  rent       747    3
3 02     Alaska  income    32940  508
4 02     Alaska  rent     1200   13
5 04     Arizona income    27517  148
6 04     Arizona  rent       972    4
7 05     Arkansas income    23789  165
8 05     Arkansas  rent       709    5
9 06     California income    29454  109
10 06     California rent     1358    3
# i 94 more rows
```

```
tidyr::us_rent_income |>
  pivot_wider(
    names_from = variable,
    values_from = c(estimate, moe)
  )
```

```
# A tibble: 52 x 6
  GEOID NAME      estimate_income estimate_rent moe_income moe_rent
  <chr> <chr>      <dbl>      <dbl>      <dbl>      <dbl>
1 01     Alabama    24476        747        136         3
2 02     Alaska    32940       1200        508        13
3 04     Arizona    27517        972        148         4
4 05     Arkansas    23789        709        165         5
5 06     California  29454       1358        109         3
6 08     Colorado    32401       1125        109         5
7 09     Connecticut  35326       1123        195         5
8 10     Delaware    31560       1076        247        10
9 11     District of Columbia  43198       1424        681        17
10 12     Florida    25952       1077         70         3
# i 42 more rows
```

3.48 pandas

```
# install palmerpenguins package
# pip install palmerpenguins
import palmerpenguins
penguins = palmerpenguins.load_penguins()
```

```
penguins[["island", "bill_length_mm"]] \
    .pivot(columns = "island", values = "bill_length_mm") \
    .fillna(0)
```

island	Biscoe	Dream	Torgersen
0	0.0	0.0	39.1
1	0.0	0.0	39.5
2	0.0	0.0	40.3
3	0.0	0.0	0.0
4	0.0	0.0	36.7
..	...	...	...
339	0.0	55.8	0.0
340	0.0	43.5	0.0
341	0.0	49.6	0.0
342	0.0	50.8	0.0
343	0.0	50.2	0.0

[344 rows x 3 columns]

3.49 polars

```
import palmerpenguins

penguins = palmerpenguins.load_penguins()
penguins.head()
```

	species	island	bill_length_mm	...	body_mass_g	sex	year
0	Adelie	Torgersen	39.1	...	3750.0	male	2007
1	Adelie	Torgersen	39.5	...	3800.0	female	2007
2	Adelie	Torgersen	40.3	...	3250.0	female	2007
3	Adelie	Torgersen	NaN	...	NaN	NaN	2007

```
4 Adelie Torgersen          36.7 ...      3450.0 female 2007

[5 rows x 8 columns]
```

```
df_penguins = pl.DataFrame(penguins)
```

```
# Long to Wide (pivot)
df_penguins.with_columns(
    pl.arange(0,df_penguins.height).alias("row_num")
).select(pl.col(["row_num", "island","bill_length_mm"]))
).pivot(values = "bill_length_mm", on = "island", index = "row_num") \
    .fill_null (0)
```

```
shape: (344, 4)
```

row_num	Torgersen	Biscoe	Dream
---	---	---	---
i64	f64	f64	f64
0	39.1	0.0	0.0
1	39.5	0.0	0.0
2	40.3	0.0	0.0
3	0.0	0.0	0.0
4	36.7	0.0	0.0
...	...	...	...
339	0.0	0.0	55.8
340	0.0	0.0	43.5
341	0.0	0.0	49.6
342	0.0	0.0	50.8
343	0.0	0.0	50.2

3.49.1 Wide-to-Long Format

To make a wide dataset to long, we can use `pivot_longer()` function from `tidyr` package in R and `melt()` function from `pandas` in python.

3.50 tidyr

```
relig_income
```

```
# A tibble: 18 x 11
  religion `<$10k` ` $10-20k` ` $20-30k` ` $30-40k` ` $40-50k` ` $50-75k` ` $75-100k`
  <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 Agnostic    27        34        60        81        76       137       122
2 Atheist     12        27        37        52        35        70        73
3 Buddhist    27        21        30        34        33        58        62
4 Catholic   418       617       732       670       638      1116      949
5 Don't k~    15        14        15        11        10        35        21
6 Evangel~   575       869      1064       982       881      1486      949
7 Hindu        1         9         7         9        11        34        47
8 Histori~   228       244       236       238       197       223      131
9 Jehovah~    20        27        24        24        21        30        15
10 Jewish     19        19        25        25        30        95        69
11 Mainlin~   289       495       619       655       651     1107      939
12 Mormon     29        40        48        51        56       112        85
13 Muslim      6         7         9        10         9        23        16
14 Orthodox   13        17        23        32        32        47        38
15 Other C~    9         7        11        13        13        14        18
16 Other F~   20        33        40        46        49        63        46
17 Other W~    5         2         3         4         2         7         3
18 Unaffil~   217       299       374       365       341       528      407
# i 3 more variables: ` $100-150k` <dbl>, ` >150k` <dbl>,
#   `Don't know/refused` <dbl>
```

```
relig_income |>
  pivot_longer(
    cols = !c(religion),
    names_to = "income",
    values_to = "count"
  )
```

```
# A tibble: 180 x 3
  religion income      count
  <chr>      <chr>      <dbl>
1 Agnostic <$10k          27
2 Agnostic $10-20k         34
3 Agnostic $20-30k         60
```

```

4 Agnostic $30-40k      81
5 Agnostic $40-50k      76
6 Agnostic $50-75k     137
7 Agnostic $75-100k    122
8 Agnostic $100-150k   109
9 Agnostic >150k       84
10 Agnostic Don't know/refused 96
# i 170 more rows

```

3.51 pandas

```

penguins.melt(value_vars=["bill_length_mm", "bill_depth_mm", "flipper_length_mm",
                          "tail_base_depth_mm"],
              id_vars = ['species', 'island', 'sex', 'year']
            )

```

	species	island	sex	year	variable	value
0	Adelie	Torgersen	male	2007	bill_length_mm	39.1
1	Adelie	Torgersen	female	2007	bill_length_mm	39.5
2	Adelie	Torgersen	female	2007	bill_length_mm	40.3
3	Adelie	Torgersen	NaN	2007	bill_length_mm	NaN
4	Adelie	Torgersen	female	2007	bill_length_mm	36.7
...	...	...	...	...	...	...
1371	Chinstrap	Dream	male	2009	body_mass_g	4000.0
1372	Chinstrap	Dream	female	2009	body_mass_g	3400.0
1373	Chinstrap	Dream	male	2009	body_mass_g	3775.0
1374	Chinstrap	Dream	male	2009	body_mass_g	4100.0
1375	Chinstrap	Dream	female	2009	body_mass_g	3775.0

```
[1376 rows x 6 columns]
```

3.52 polars

```

# Wide to Long (melt() but now polar calls it unpivot() )
df_penguins.with_columns(
    pl.arange(0, df_penguins.height).alias("row_num")
).select(pl.col(["row_num", "island", "bill_length_mm"]))
).pivot(values = "bill_length_mm", on = "island", index = "row_num") \
    .unpivot(index = "row_num", value_name = "bill_length_mm")

```

shape: (1_032, 3)

row_num	variable	bill_length_mm
---	---	---
i64	str	f64
0	Torgersen	39.1
1	Torgersen	39.5
2	Torgersen	40.3
3	Torgersen	null
4	Torgersen	36.7
...	...	...
339	Dream	55.8
340	Dream	43.5
341	Dream	49.6
342	Dream	50.8
343	Dream	50.2

3.53 Merging Datasets

Many times, for analysis purposes, we need to join two datasets. This process is also called merging³. There are different types of joining. So, it is important to learn about those joining techniques. In Figure ?? shows the joining technique and functions using `dplyr` in R. Below all of these joining functions are explained.

- `left_join()`: The merged dataset contains **all** observations from the **first** (or left) dataset and only **matched** observations from the **second** (or right) dataset
- `right_join()`: The merged dataset contains only **matched** observations from the **first** (or left) dataset and **all** observations from the **second** (or right) dataset
- `inner_join()`: The merged dataset contains *only* **matched** observations from both datasets
- `semi_join()`: The merged dataset contains **matched** observations from the **first** (or left) dataset. Please note that `semi_join()` differs from `inner_join()` in that `inner_join()` will return one row of first dataset (x) for each matching row of second dataset (y), whereas `semi_join()` will never duplicate rows of x.
- `full_join()`: The merged dataset contains **all** observations from both datasets

³In database context, it is “merging”, but commonly it is called “joining”.

Table 3.3: Joining Functions - dplyr vs pandas

dplyr	pandas	Description
<code>left_join()</code>	<code>pd.merge(df1, df2, on='key', how='left')</code>	Join matching rows from df2 to df1, keeping all rows from df1.
<code>right_join()</code>	<code>pd.merge(df1, df2, on='key', how='right')</code>	Join matching rows from df1 to df2, keeping all rows from df2.
<code>inner_join()</code>	<code>pd.merge(df1, df2, on='key', how='inner')</code>	Join matching rows from both dataframes (default).
<code>full_join()</code>	<code>pd.merge(df1, df2, on='key', how='outer')</code>	Join all rows from both dataframes, filling missing values with NA.
<code>semi_join()</code>	No direct equivalent, but can be achieved using filtering	Keep rows in df1 where a match exists in df2.
<code>anti_join()</code>	No direct equivalent, but can be achieved using filtering	Keep rows in df1 where no match exists in df2.

- `anti_join()`: The merged dataset contains only **not matched** observations from the **first** (or left) dataset and contains only the variable from the **left** dataset

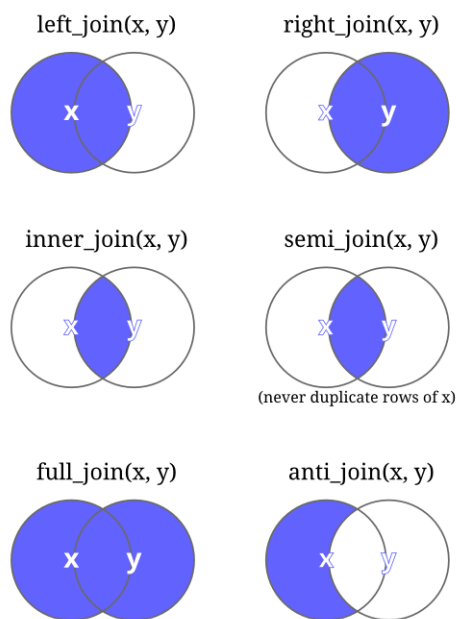


Figure 3.3: Joining Datasets

Table ?? compares the `dplyr` joining functions with equivalent joining functions from `pandas`.

3.54 dplyr

```
data1 = read_csv("https://raw.githubusercontent.com/msharifbd/DATA/refs/heads/main/data1.csv")
glimpse(data1)
```

```

Rows: 6,910
Columns: 14
$ companyname      <chr> "AMERICAN AIRLINES GROUP INC", "AMERICAN AIRLINES GROU~
$ stateincorp      <chr> "DE", "DE", "DE", "DE", "DE", "DE", "DE", "DE", "DE", ~
$ ticker           <chr> "AAL", "AAL", "AAL", "AAL", "AAL", "AAL", "AAL", "AAL"~
$ year             <dbl> 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, ~
$ sic              <dbl> 4512, 4512, 4512, 4512, 4512, 4512, 4512, 4512, 4512, ~
$ totalassets      <dbl> 25175.000, 25438.000, 25088.000, 23848.000, 23510.000,~
$ costofgoodssold <dbl> 20232.000, 16935.000, 18138.000, 20420.000, 20529.000,~
$ netincome        <dbl> -2071.000, -1468.000, -471.000, -1979.000, -1876.000, ~
$ sale             <dbl> 23766.000, 19917.000, 22170.000, 24022.000, 24855.000,~
$ advertising      <dbl> 153.000, 153.000, 165.000, 186.000, 153.000, 166.000, ~
$ sellingadmin     <dbl> 3024.000, 2720.000, 2729.000, 2907.000, 2892.000, 4672~
$ mktvalue         <dbl> 2976.3858, 2571.1835, 2597.5755, 117.3438, 266.5571, 6~
$ commonequity     <dbl> -2935.000, -3489.000, -3945.000, -7111.000, -7987.000,~
$ totalliability   <dbl> 28110.000, 28927.000, 29033.000, 30959.000, 31497.000,~

```

```

data2 = read_csv("https://github.com/msharifbd/DATA/raw/refs/heads/main/DATA2")
glimpse(data2)

```

```

Rows: 43,147
Columns: 13
$ ticker           <chr> "MLP", "MLP", "MLP", "RCPIQ", "RCPIQ", "RCPI~
$ cik_code         <dbl> 63330, 63330, 63330, 776008, 776008, 776008,~
$ auditor          <chr> "Accuity LLP", "Accuity LLP", "Accuity LLP",~
$ INTERNALCONTROL_EFFECTIVE <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "N~
$ year_ended_date  <date> 2017-12-31, 2018-12-31, 2019-12-31, 2004-12~
$ financials_date  <date> 2017-12-31, 2018-12-31, 2019-12-31, 2004-12~
$ fiscal_year      <dbl> 2017, 2018, 2019, 2004, 2005, 2006, 2007, 20~
$ signature_date   <date> 2018-02-23, 2019-03-01, 2020-02-28, 2005-04~
$ NUMBEROFCONTROLWEAKNESS <dbl> 0, 0, 0, 0, 0, 0, 0, 1, 4, 0, 0, 0, 0, 0, 0,~
$ audit_fees       <dbl> 206000, 209000, 213000, 224209, 382454, 4155~
$ non_audit_fees   <dbl> 29000, 30000, 30000, 188081, 92869, 61351, 3~
$ year             <dbl> 2017, 2018, 2019, 2004, 2005, 2006, 2007, 20~
$ big4             <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~

```

```

# left_join
left_join(data1, data2, by = c("ticker", "year"))

```

```

# A tibble: 6,969 x 25
  companyname      stateincorp ticker year  sic totalassets costofgoodssold

```


	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	AMERICAN AIRLINES~	DE	AAL	2008	4512	25175	20232
2	AMERICAN AIRLINES~	DE	AAL	2009	4512	25438	16935
3	AMERICAN AIRLINES~	DE	AAL	2010	4512	25088	18138
4	AMERICAN AIRLINES~	DE	AAL	2011	4512	23848	20420
5	AMERICAN AIRLINES~	DE	AAL	2012	4512	23510	20529
6	AMERICAN AIRLINES~	DE	AAL	2013	4512	42278	19084
7	AMERICAN AIRLINES~	DE	AAL	2014	4512	43771	29511
8	AMERICAN AIRLINES~	DE	AAL	2015	4512	48415	25416
9	AMERICAN AIRLINES~	DE	AAL	2016	4512	51274	25695
10	AMERICAN AIRLINES~	DE	AAL	2017	4512	51396	28262

```

# i 6,959 more rows
# i 18 more variables: netincome <dbl>, sale <dbl>, advertising <dbl>,
#   sellingadmin <dbl>, mktvalue <dbl>, commonequity <dbl>,
#   totalliability <dbl>, cik_code <dbl>, auditor <chr>,
#   INTERNALCONTROL_EFFECTIVE <chr>, year_ended_date <date>,
#   financials_date <date>, fiscal_year <dbl>, signature_date <date>,
#   NUMBEROFCONTROLWEAKNESS <dbl>, audit_fees <dbl>, non_audit_fees <dbl>, ...

```

```

# left_join
left_join(data1 |> distinct(ticker, year, .keep_all = TRUE),
          data2 |> distinct(ticker, year, .keep_all = TRUE),
          by = c("ticker", "year")
        )

```

```

# A tibble: 6,910 x 25
  companyname      stateincorp ticker year  sic totalassets costofgoodssold
  <chr>          <chr>      <chr> <dbl> <dbl>    <dbl>          <dbl>
1 AMERICAN AIRLINES~ DE      AAL    2008  4512    25175          20232
2 AMERICAN AIRLINES~ DE      AAL    2009  4512    25438          16935
3 AMERICAN AIRLINES~ DE      AAL    2010  4512    25088          18138
4 AMERICAN AIRLINES~ DE      AAL    2011  4512    23848          20420
5 AMERICAN AIRLINES~ DE      AAL    2012  4512    23510          20529
6 AMERICAN AIRLINES~ DE      AAL    2013  4512    42278          19084
7 AMERICAN AIRLINES~ DE      AAL    2014  4512    43771          29511
8 AMERICAN AIRLINES~ DE      AAL    2015  4512    48415          25416
9 AMERICAN AIRLINES~ DE      AAL    2016  4512    51274          25695
10 AMERICAN AIRLINES~ DE      AAL    2017  4512    51396          28262

```

```

# i 6,900 more rows
# i 18 more variables: netincome <dbl>, sale <dbl>, advertising <dbl>,
#   sellingadmin <dbl>, mktvalue <dbl>, commonequity <dbl>,
#   totalliability <dbl>, cik_code <dbl>, auditor <chr>,

```

```
# INTERNALCONTROL_EFFECTIVE <chr>, year_ended_date <date>,
# financials_date <date>, fiscal_year <dbl>, signature_date <date>,
# NUMBEROFCONTROLWEAKNESS <dbl>, audit_fees <dbl>, non_audit_fees <dbl>, ...
```

3.55 pandas

```
dataset1 = pd.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/refs/heads/main/dataset1.csv")
dataset2 = pd.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/refs/heads/main/dataset2.csv")
```

```
pd.merge(dataset1, dataset2, on=['ticker', 'year'], how='left')
```

	companyname	state	incorp	...	non_audit_fees	big4
0	AMERICAN AIRLINES GROUP INC	DE	...		990000.0	1.0
1	AMERICAN AIRLINES GROUP INC	DE	...		1446000.0	1.0
2	AMERICAN AIRLINES GROUP INC	DE	...		1455000.0	1.0
3	AMERICAN AIRLINES GROUP INC	DE	...		2645000.0	1.0
4	AMERICAN AIRLINES GROUP INC	DE	...		1989000.0	1.0
...	...	...	...	...	...	...
6964	INFOSONICS CORP -OLD	MD	...		NaN	NaN
6965	INFOSONICS CORP -OLD	MD	...		NaN	NaN
6966	INFOSONICS CORP -OLD	MD	...		NaN	NaN
6967	INFOSONICS CORP -OLD	MD	...		NaN	NaN
6968	INFOSONICS CORP -OLD	MD	...		NaN	NaN

[6969 rows x 25 columns]

```
dataset1_drop = dataset1.drop_duplicates(subset=['ticker', 'year'], ignore_index=True)
dataset2_drop = dataset2.drop_duplicates(subset=['ticker', 'year'], ignore_index=True)
pd.merge(dataset1_drop, dataset2_drop, on=['ticker', 'year'], how='left')
```

	companyname	state	incorp	...	non_audit_fees	big4
0	AMERICAN AIRLINES GROUP INC	DE	...		990000.0	1.0
1	AMERICAN AIRLINES GROUP INC	DE	...		1446000.0	1.0
2	AMERICAN AIRLINES GROUP INC	DE	...		1455000.0	1.0
3	AMERICAN AIRLINES GROUP INC	DE	...		2645000.0	1.0
4	AMERICAN AIRLINES GROUP INC	DE	...		1989000.0	1.0
...	...	...	...	...	...	...
6905	INFOSONICS CORP -OLD	MD	...		NaN	NaN
6906	INFOSONICS CORP -OLD	MD	...		NaN	NaN

```

6907      INFOSONICS CORP -OLD      MD ...      NaN      NaN
6908      INFOSONICS CORP -OLD      MD ...      NaN      NaN
6909      INFOSONICS CORP -OLD      MD ...      NaN      NaN

```

[6910 rows x 25 columns]

```

df1 = pd.DataFrame({'id': [1, 2, 3], 'value': ['A', 'B', 'C']})
df2 = pd.DataFrame({'id': [2, 3, 4], 'other_value': ['X', 'Y', 'Z']})

# Left join
left_join = pd.merge(df1, df2, on='id', how='left')

# Inner join
inner_join = pd.merge(df1, df2, on='id')

# Semi join
semi_join = df1[df1['id'].isin(df2['id'])]

# Anti join
anti_join = df1[~df1['id'].isin(df2['id'])]

```

3.56 polars

```

dataset1 = pl.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/refs/heads/main/dataset1.csv")
dataset2 = pl.read_csv("https://raw.githubusercontent.com/msharifbd/DATA/refs/heads/main/dataset2.csv")

```

```
dataset1.join(dataset2, on = ["ticker", 'year'], how = "left")
```

shape: (6_969, 25)

companyname	stateincorp	ticker	year	...	NUMBEROFCONT	audit_fees	non
AMERICAN	DE	AAL	2008	...	0	2578000	990000
AIRLINES							
GROUP INC							
AMERICAN	DE	AAL	2009	...	0	2570000	1446000

AIRLINES									
GROUP INC									
AMERICAN	DE	AAL	2010	...	0		2183000	1455000	
AIRLINES									
GROUP INC									
AMERICAN	DE	AAL	2011	...	0		2551000	2645000	
AIRLINES									
GROUP INC									
AMERICAN	DE	AAL	2012	...	0		3939000	1989000	
AIRLINES									
GROUP INC									
...	...	...	...	...	...	...	...	...	...
INFOSONICS	MD	IFON.1	2013	...	null		null	null	
CORP -OLD									
INFOSONICS	MD	IFON.1	2014	...	null		null	null	
CORP -OLD									
INFOSONICS	MD	IFON.1	2015	...	null		null	null	
CORP -OLD									
INFOSONICS	MD	IFON.1	2016	...	null		null	null	
CORP -OLD									
INFOSONICS	MD	IFON.1	2017	...	null		null	null	
CORP -OLD									

```
dataset1_drop = dataset1.unique(subset=['ticker', 'year'])
dataset2_drop = dataset2.unique(subset=['ticker', 'year'])
```

```
dataset1_drop.join(dataset2_drop, on = ["ticker", 'year'], how = "left")
```

```
shape: (6_910, 25)
```

companyname	stateincorp	ticker	year	...	NUMBEROFCONT	audit_fees	non	
---	---	---	---	ROLWEAKNESS	---	es	---	
str	str	str	i64	---	str	---	i64	
			i64		str			
DOLBY	DE	DLB	2017	...	0	2787800	720107	1
LABORATORIES								
INC								
PLANTRONICS	DE	POLY	2008	...	0	2314915	224806	
INC								
INTEGRATED	DE	INBP	2009	...	null	null	null	

cally applying descriptive statistics and visual techniques, EDA uncovers a dataset's structure, trends, relationships, and irregularities, so that subsequent analyses rely on sound assumptions instead of guesswork. It enables analysts to spot errors and outliers, clarify what variables represent, and pinpoint the most important features, thereby strengthening model performance, interpretability, and the overall value of the conclusions drawn.

Rather than a single step, EDA is an iterative, reflective process that refines research questions, tests the suitability of methods, and ensures that technical work stays aligned with practical goals. Ultimately, EDA is essential: skipping it leaves statistical analysis and machine learning vulnerable to being elaborate but misguided, while using it turns them into dependable, transparent, and practically useful tools.

4 Data Visualization

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Understand the Meaning of Visualization
- Recognize the Importance of Visualization
- Explore Different R and Python Packages for Visualization
- Prepare Different Types of Visualizations for Different Types of Data
- Prepare Interactive Visualization

4.1 What is Visualization?

Visualization refers to the graphical presentation of data. Visualization involves using charts, graphs, maps, and other visualization tools. Visualization enables to understand trends, patterns, and outliers, if any, in the dataset. Visualization is very critical in data analytics process in that it allows complex data to be presented in a way that is easy to understand and interpret. Visualization tools can range from simple charts to more sophisticated advanced tools like interactive dashboards.

4.2 Importance of Visualization

Mastering visualization technique is important because it allows to simplify story telling of complex data, identify pattern, enhance communication and so on.

- **Simplifying Complex Data:** More complex data can be made simpler and understandable using visualization, thus enabling decision makers to understand key insights and make informed decisions.

- **Identifying Trends and Patterns:** Using visualization, patterns and trends can be easily identified in the data, which further help to identify correlation between variables.
- **Improving Communication:** Visualization is very effective to communicate findings from the data to stakeholders in a simple and understandable way. Therefore, users who lack technical expertise can easily understand the main point of data.
- **Supporting Effective Decision Making:** Visualization emphasizes important insights, thus helping organizations make data-driven decisions. Moreover, visualization makes the data more understandable and accessible, thus supporting strategic planning and operational improvements.
- **Detecting Outliers and Anomalies:** Effective visualization tools and techniques make it easy to spot outliers and anomalies in the data. These unusual data points can be important for identifying errors, fraud, or opportunities for further investigation.
- **Engaging Audience:** Interactive and visually appealing visualization can engage audience and enhance effective delivery of main points. Visualization tools like interactive dashboards allow users to explore the data themselves, thus leading to deeper understanding of the data.
- **Encouraging Exploration:** Data visualization tools often allow for interactive exploration of data. Users can drill down into specific data points, filter relevant data, and view different dimensions of the data, thus leading to new insights and discoveries.

4.3 Visualization Packages in R and Python

4.4 R

ggplot2 is a powerful package for visualization in R. In addition, some other packages enhance the functionalities of ggplot2. These packages include - gganimate, ggthemes, ggpubr, ggribbles, ggmap, ggrepel, ggextra, ggpattern, ggcorrplot and so on.

```
# Loading tidyverse package
library(tidyverse)
# Loading dataset
tips = read_csv(
  'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv'
)
```


4.5 Python

In Python, matplotlib and seaborn are two of the powerful packages for visualization. Additionally, plotly, plotnine, altair, and bokeh are some other python packages that enhances visualization in python.

```
# Loading Necessary Python Packages
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# ggplot style
plt.style.use('ggplot')
# Loading dataset
tips = sns.load_dataset('tips')
```

4.6 ggplot - Grammar of Graphics

In ggplot, a plot consists of at least four elements -

1. Data - the data frame
2. Aesthetic Mappings - aesthetic mappings map variable from the data frame to different kinds of aesthetics such as x coordinate, y coordinate, color, shape, size and so on.
3. Coordinate System - the positioning of points
4. Geom - geoms are geometirc objects such as points or lines.

You can also use [cheatsheet](#) of ggplot to know more about the ggplot. Another good source to learn more about visualization in R is [The R Graph Library](#). Similarly, for Python, you can use [The Python Graph Library](#).

4.7 Types of Visualization

4.7.1 Bar Diagram (Bar Plot)

4.7.1.1 One Categorical Variable

4.8 R

```
tips |>
  count (sex) |>
  ggplot(mapping = aes(x = sex, y = n))+
  geom_bar(stat = 'identity', width = 0.5, fill = "orangered3") +
  labs(x = 'Sex', y = 'Total Observations')
```

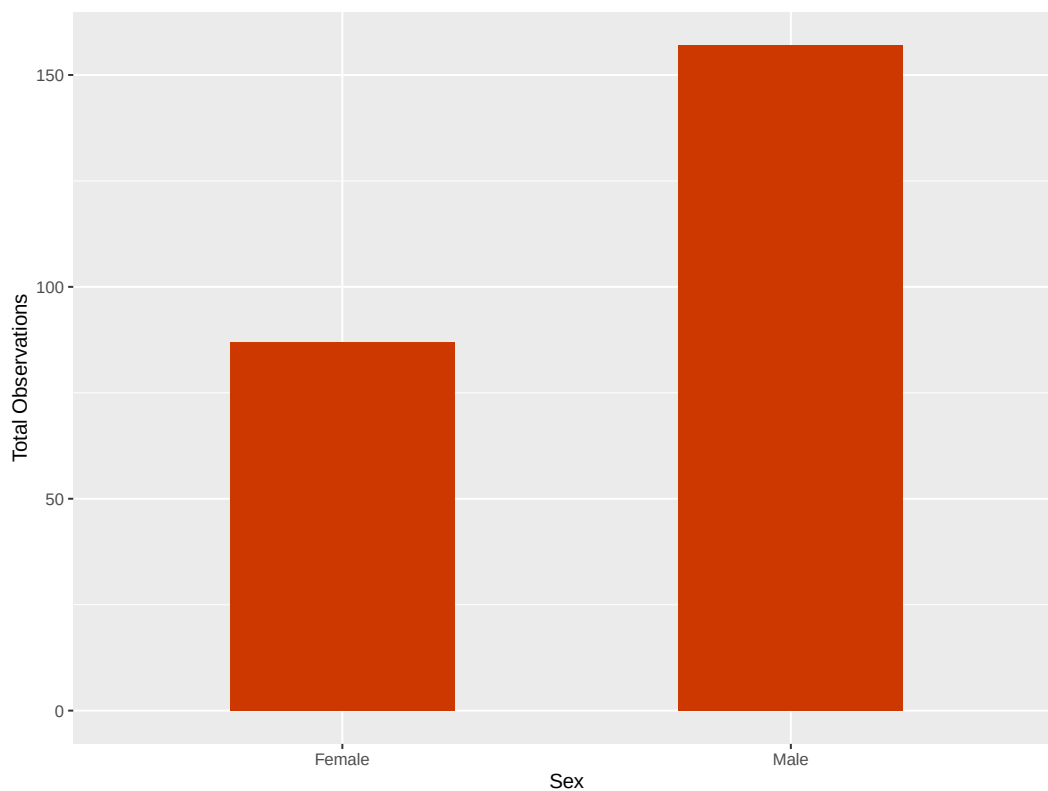


Figure 4.1: Bar Plot of Gender (geom_bar)

Either of the the following code will also produce the same visualization.

```
tips |>
  ggplot(mapping = aes(x = sex))+
  geom_bar(width = 0.5, fill = "maroon") +
  labs(x = 'Sex', y = 'Total Observations')
```

```
tips |>
  ggplot(mapping = aes(x = sex))+
  stat_count(width = 0.5, fill = "maroon") +
  labs(x = 'Sex', y = 'Total Observations')
```

4.9 Python

```
sns.countplot(data = tips, x = "sex", width=0.5)
plt.xlabel('Sex')
plt.ylabel('Total Observations')
```

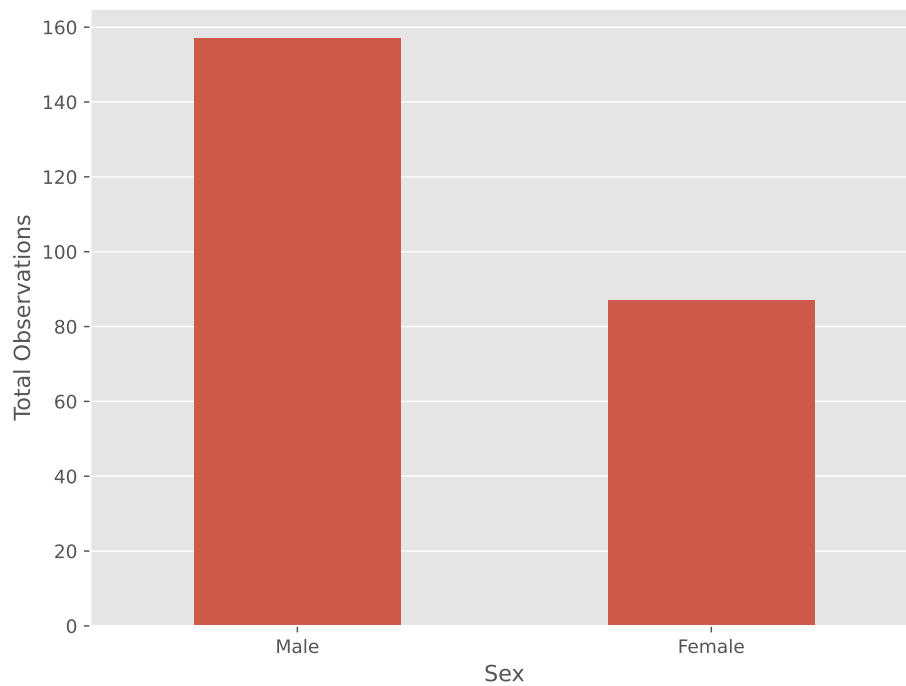


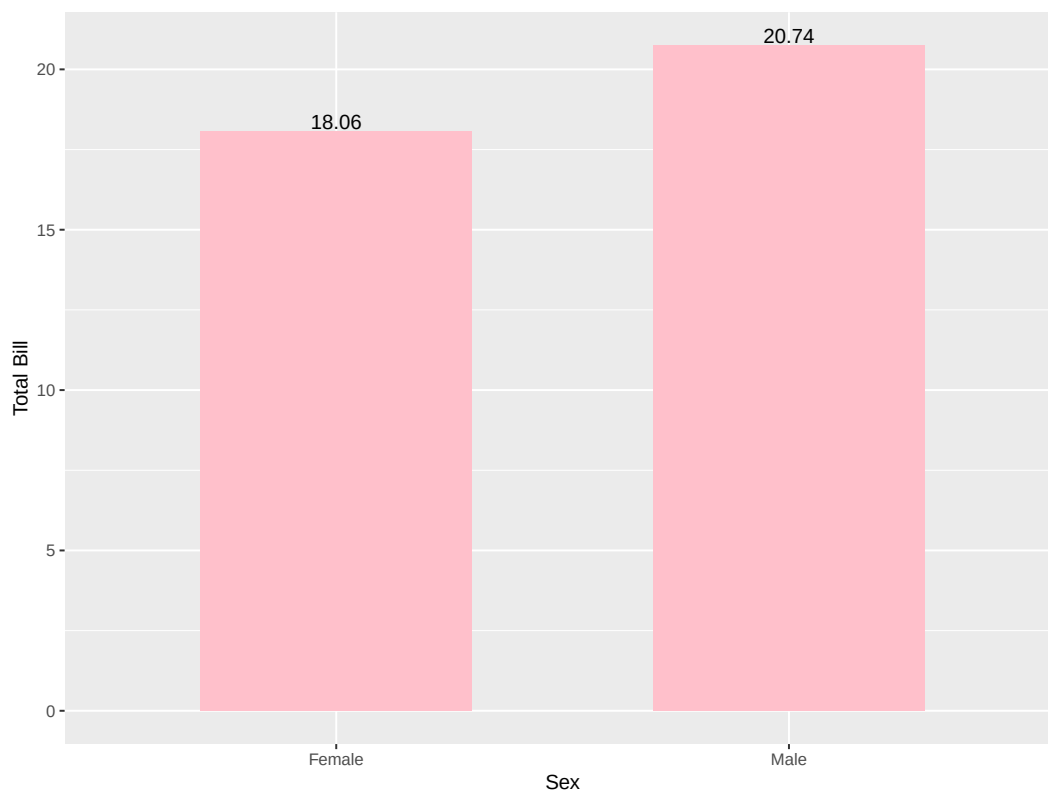
Figure 4.2: Bar Plot of Gender (sns.countplot)

4.9.0.1 One Categorical Variable and One Continuous Variable

Barplot can also be used for two variables - both discrete (categorical) variables or one discrete (categorical) and one continuous variable. Below is bar plot for one discrete (categorical) and one continuous variable.

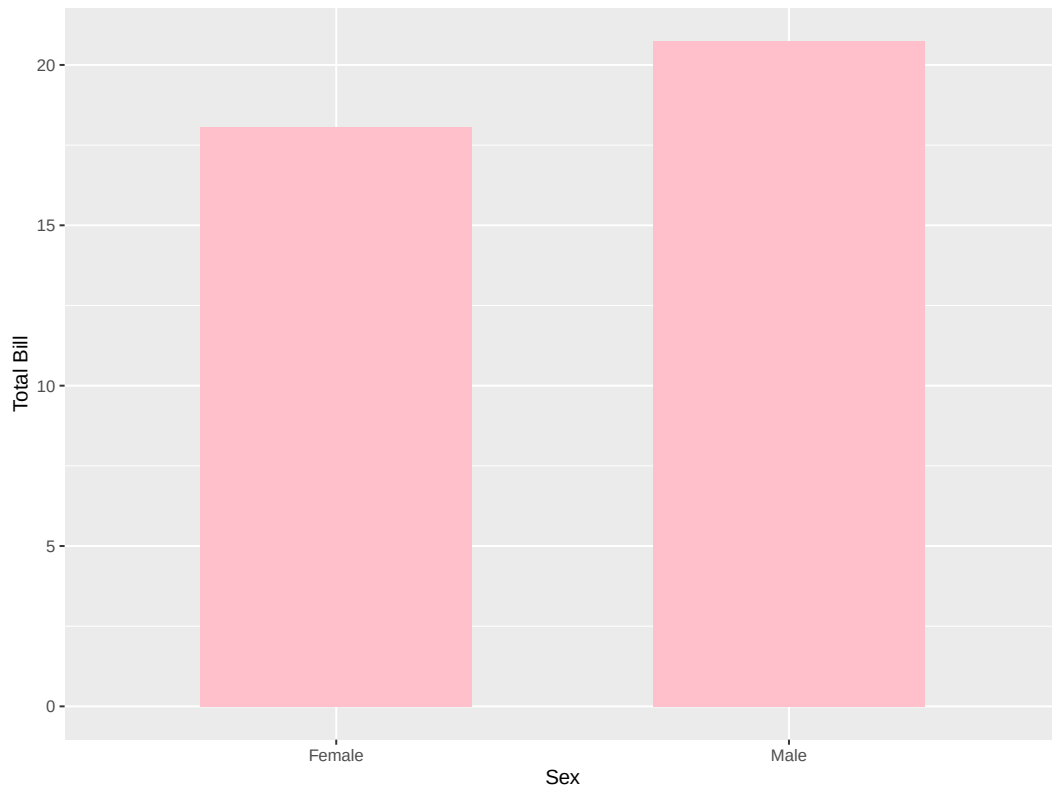
4.10 R

```
tips |>
  group_by(sex) |>
  summarize(total_bill = mean(total_bill)) |>
  ggplot(aes(x = sex, y = total_bill)) +
  geom_col(width = 0.6, fill = "pink") +
  labs(x = "Sex", y = "Total Bill") +
  geom_text(aes(label = round(total_bill, 2)), vjust = -0.2)
```



The following code will produce the same results.

```
tips |>
  ggplot(mapping = aes(x = sex, y = total_bill))+
  geom_bar(stat = 'summary', fun = "mean", position = "dodge",
width = 0.60, fill = "pink") +
  labs(x = "Sex", y = "Total Bill")
```



4.11 Python

```
sns.barplot(data = tips, x = "sex", y = "total_bill",
            width= 0.5,
            errorbar= None)
plt.xlabel('Sex')
plt.ylabel('Total Bill')
```

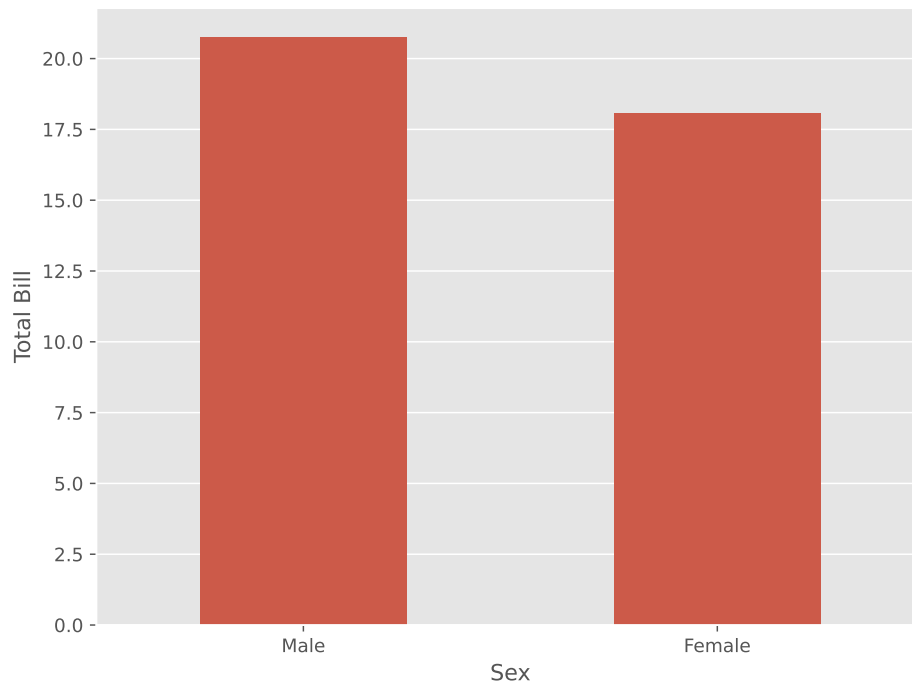


Figure 4.3: Bar Plot of Gender (sns.barplot)

The following code will add text value on the bars in barplot.

```
ax = sns.barplot(data = tips, x = "sex", y = "total_bill",
                 width= 0.5,
                 errorbar= None)

for i in ax.containers:
    ax.bar_label(i,)

plt.xlabel('Sex')
plt.ylabel('Total Bill')
```

4.11.0.1 Two Categorical Variables

Below is a bar plot for both discrete (categorical) variables.

4.12 R

```
tips |>
  count (sex, day) |>
  ggplot(mapping = aes(x = sex, y = n, fill = day))+
  geom_bar(stat = 'identity', position = "dodge") +
  labs(x = "Sex", y = "Total Observations")
```

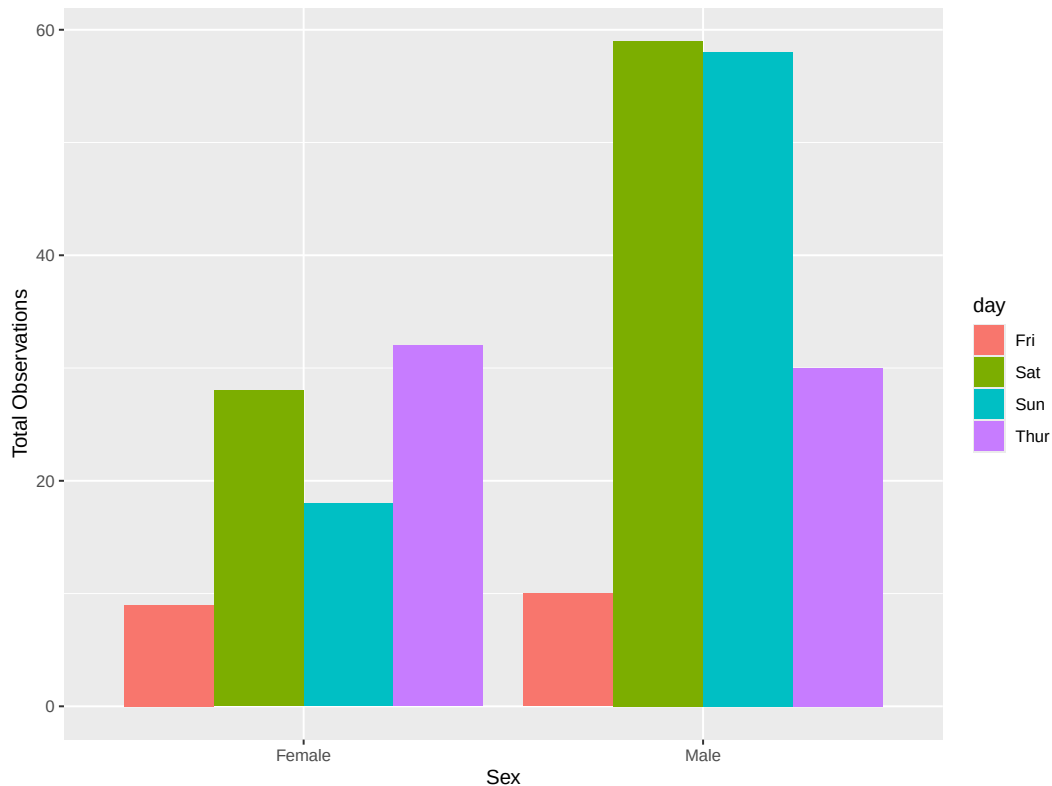
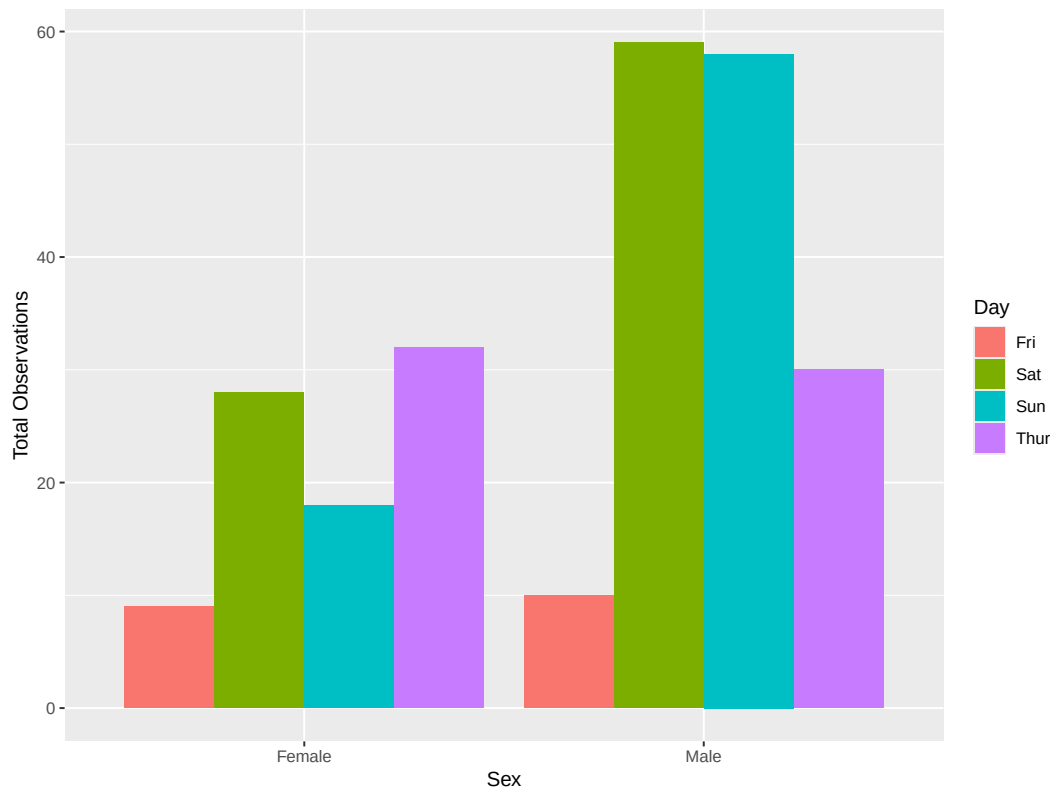


Figure 4.4: Bar Plot of Gender (geom_bar - unstacking bar)

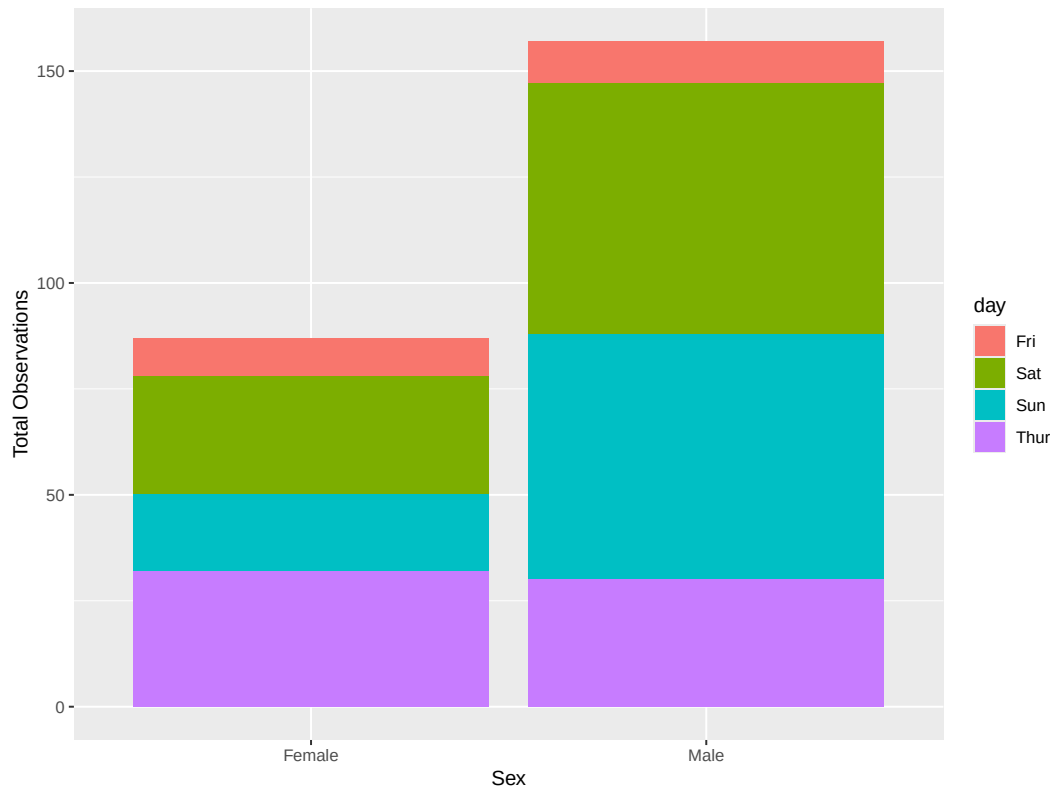
The following code will also produce the same visualization.

```
tips |>
  #count (sex, day) |>
  ggplot(mapping = aes(x = sex, fill = day))+
  geom_bar(stat = 'count', position = "dodge") +
  labs(x = "Sex", y = "Total Observations")
```

```
) , fill = "Day"  
)
```

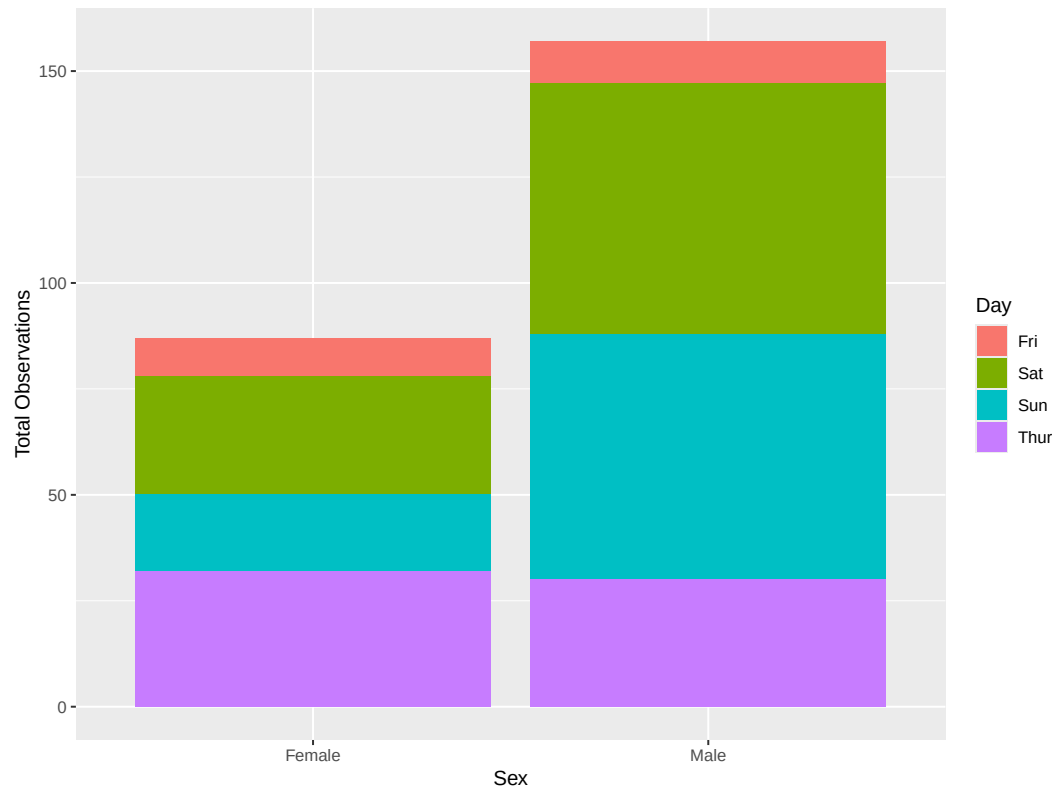


```
tips |>  
  count (sex, day) |>  
  ggplot(mapping = aes(x = sex, y = n, fill = day))+  
  geom_bar(stat = 'identity', position = "stack") + # position = "fill"  
  labs(x = "Sex", y = "Total Observations")
```

The following code will also produce the same visualization.

```
tips |>
  #count (sex, day) |>
  ggplot(mapping = aes(x = sex, fill = day))+
  geom_bar(stat = 'count', position = "stack") + # position = "fill"
  labs(x = "Sex", y = "Total Observations"
        ,fill = "Day"
  )
```



4.13 Python

```
sns.countplot(data = tips, x = "sex", hue = "day")  
plt.xlabel('Sex')  
plt.ylabel('Total Observations')
```

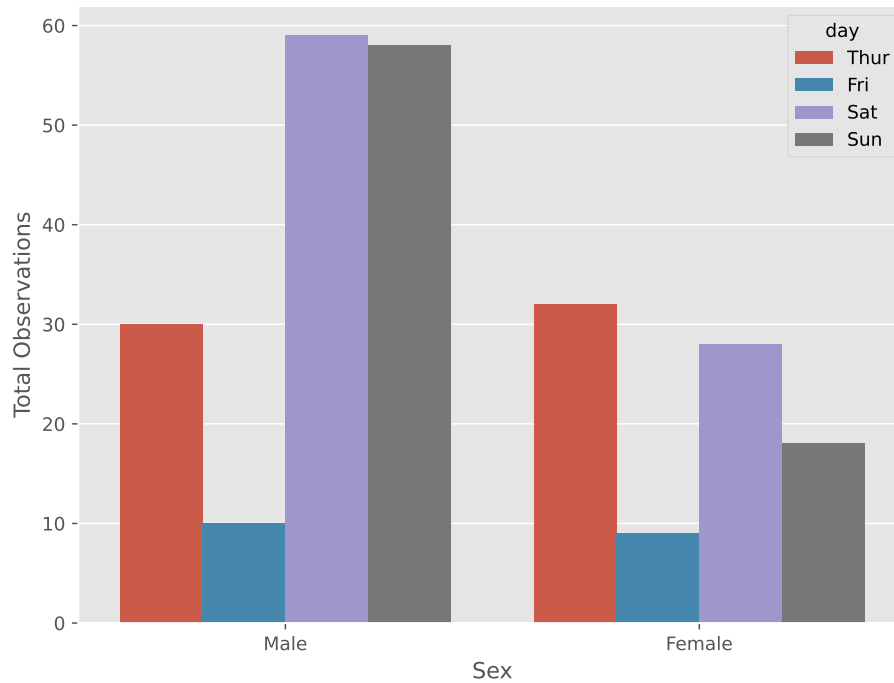


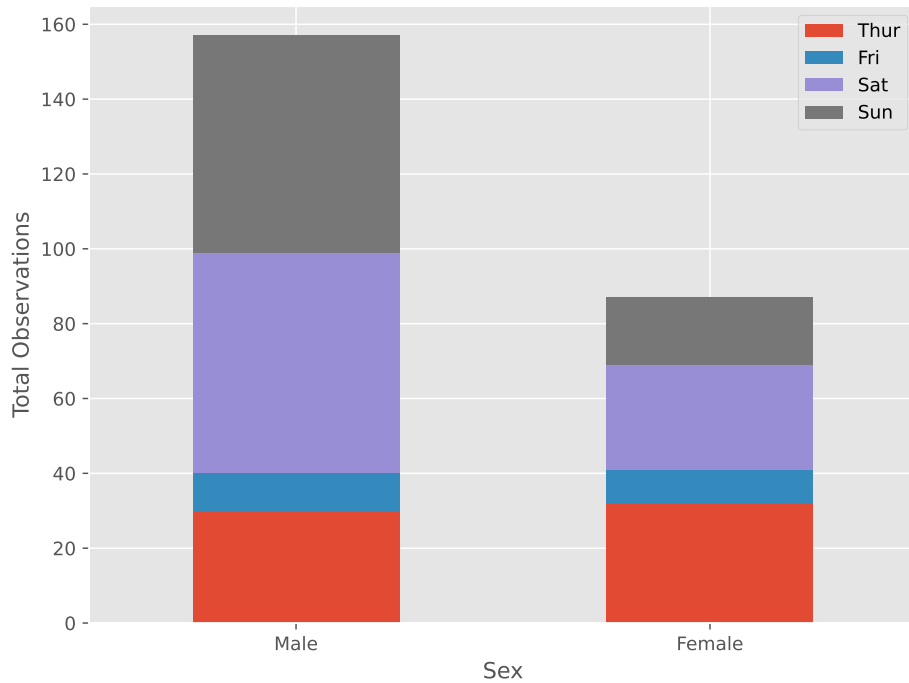
Figure 4.5: Bar Plot of Gender (sns.countplot - unstacking bar)

Stacked barchart cannot be created using seaborn. So, we use alternatives -

```
tips[['sex', 'day']].value_counts().reset_index() \
    .pivot(index = "sex", columns = "day", values = 'count') \
    .plot(kind = "bar", stacked = True)
plt.xticks(rotation = 360)
```

```
(array([0, 1]), [Text(0, 0, 'Male'), Text(1, 0, 'Female')])
```

```
plt.xlabel("Sex")
plt.ylabel("Total Observations")
plt.legend(loc = "upper right")
```

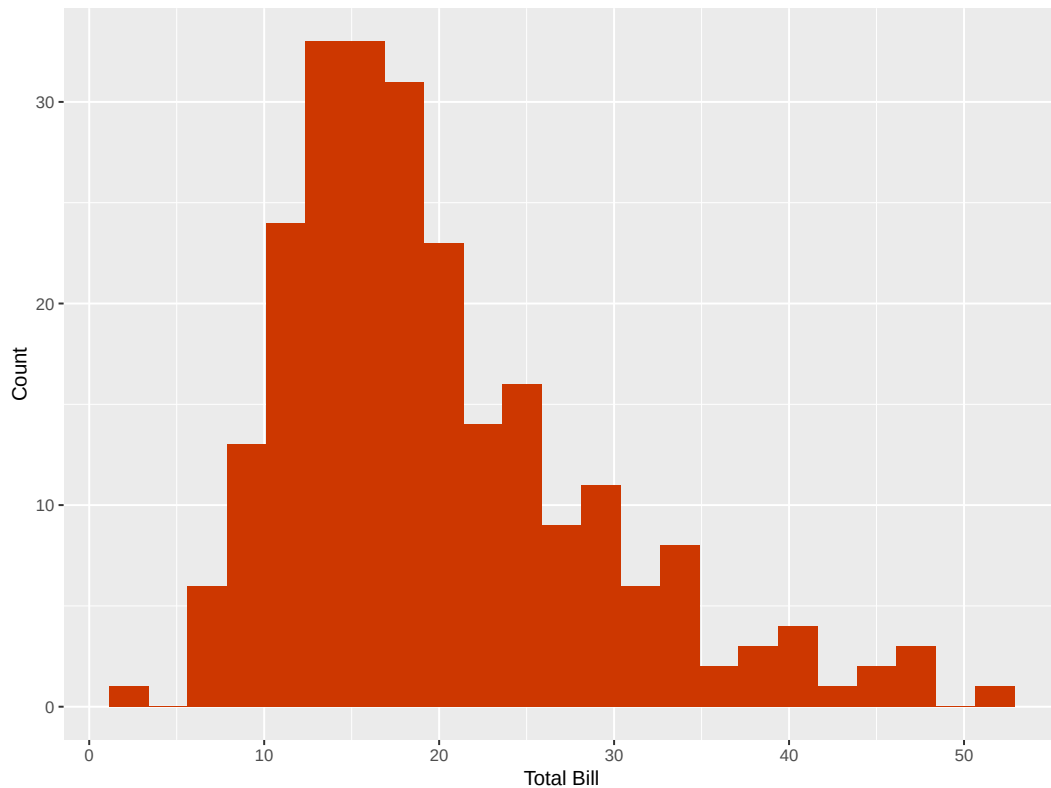


4.13.1 Histogram

4.13.1.1 One Continuous Variable

4.14 R

```
tips |>
  ggplot(aes(x = total_bill)) +
  geom_histogram(binwidth = 2.25, fill = "orangered3") +
  labs(x = "Total Bill", y = "Count")
```

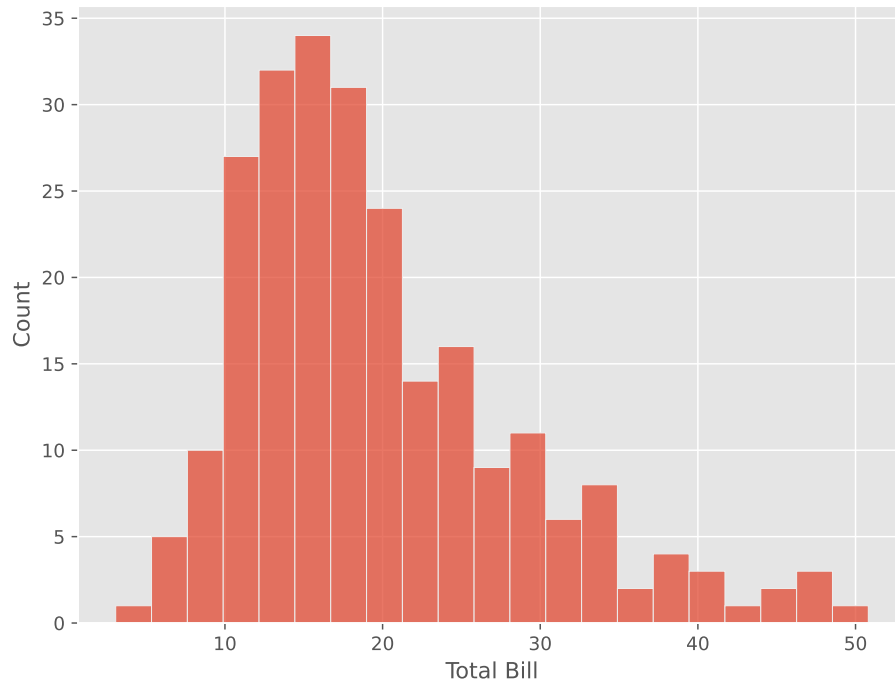


The following code will generate the same results with a little modification -

```
tips |>
  ggplot(aes(x = total_bill)) +
  geom_histogram(binwidth = 2.25, fill = "orangered3", col = "white") +
  labs(x = "Total Bill", y = "Count")
```

4.15 Python

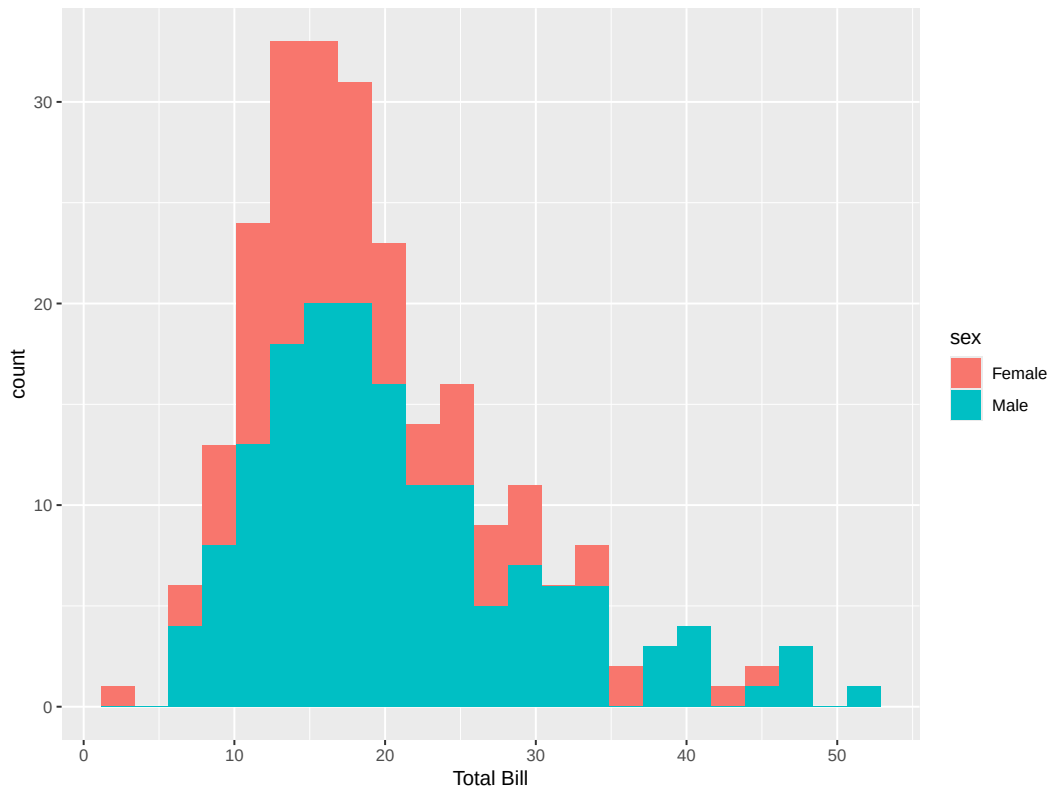
```
sns.histplot(data = tips, x = "total_bill", binwidth=2.25)
plt.xlabel("Total Bill")
plt.ylabel("Count")
```



4.15.0.1 One Continuous and One Categorical Variable

4.16 R

```
tips |>
  ggplot(aes(x = total_bill, fill = sex))+
  geom_histogram(binwidth = 2.25)+
  labs(x = "Total Bill")
```

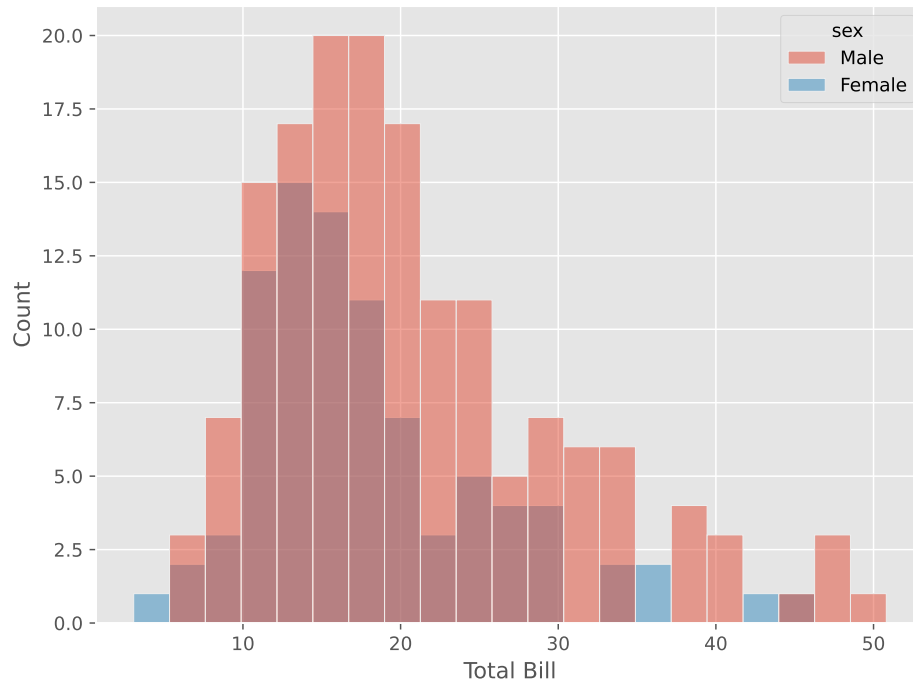


The following code will generate the same results -

```
tips |>
  ggplot(aes(x = total_bill, color = sex)) +
  geom_histogram(binwidth = 2.25)
```

4.17 Python

```
sns.histplot(data = tips, x = "total_bill", hue = "sex", binwidth=2.25)
plt.xlabel("Total Bill")
plt.ylabel("Count")
```



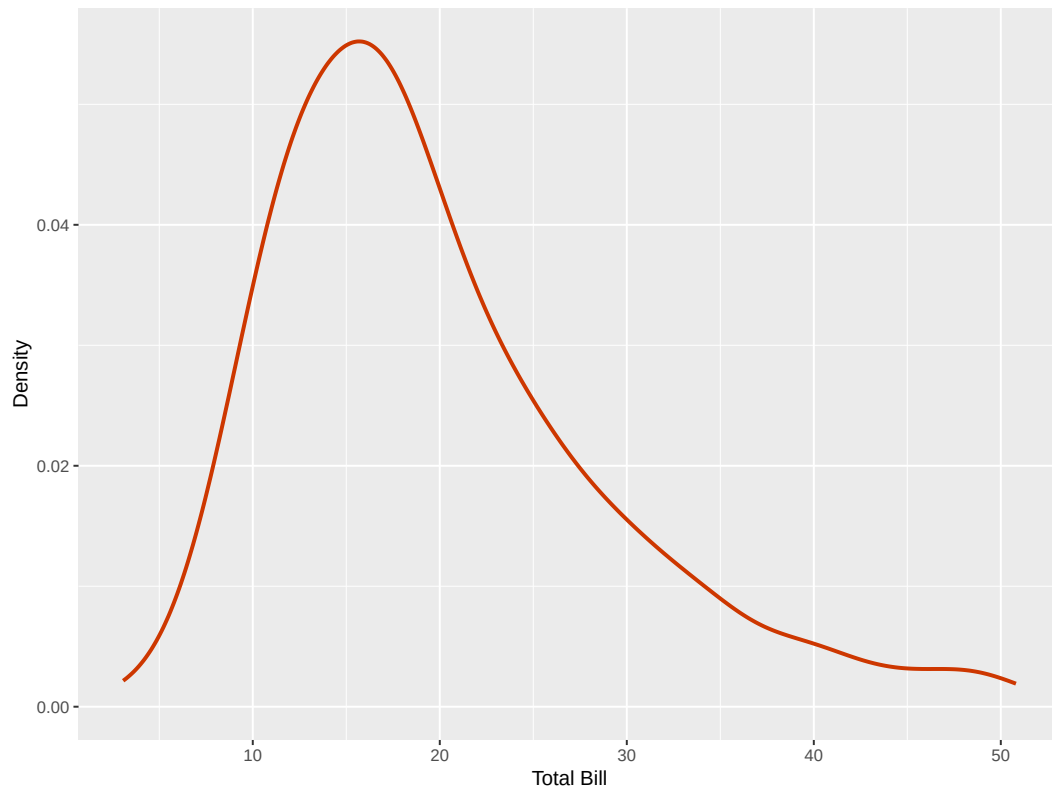
```
sns.FacetGrid(data=tips, col="sex") \
    .map(sns.histplot, "total_bill", binwidth = 2.25)
```

4.17.1 Density Plot

4.17.1.1 One Continuous Variable

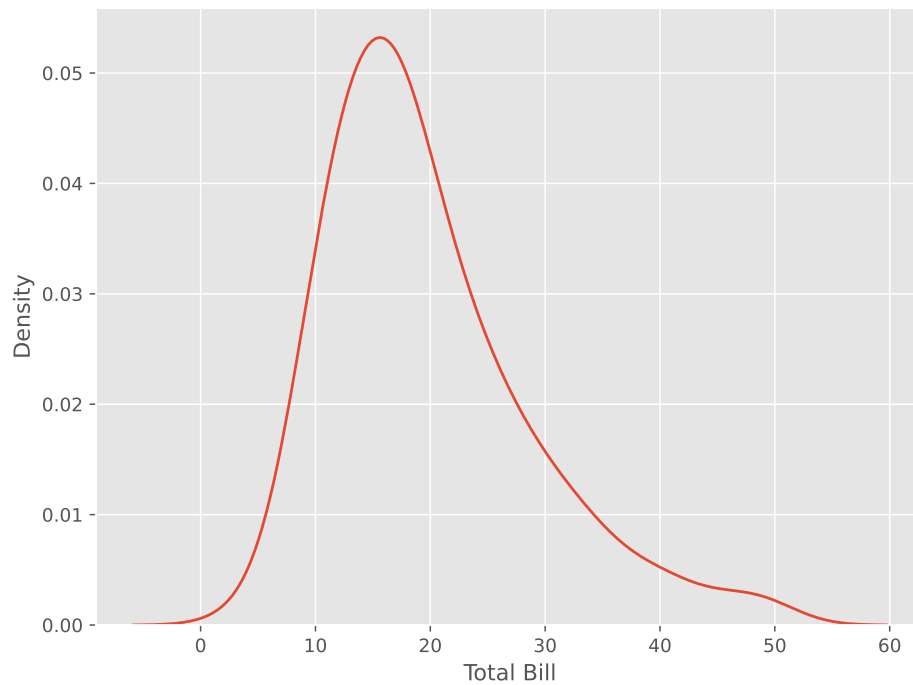
4.18 R

```
tips |>
  ggplot(aes(x = total_bill))+
  geom_density( size = 1, color = "orangered3"
    #adjust = 0.2
  ) +
  labs(x = "Total Bill", y = "Density")
```

4.19 Python

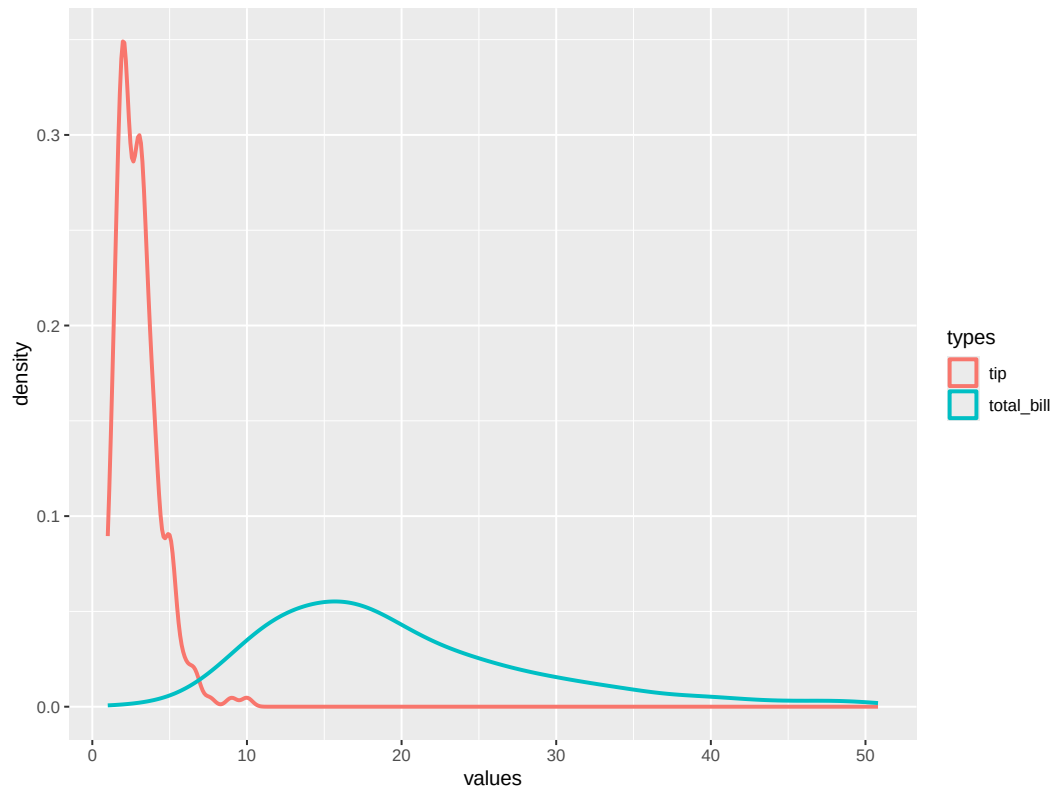
```
sns.kdeplot(data = tips, x = "total_bill"  
            #,bw_adjust = 0.20  
            )  
plt.xlabel("Total Bill")
```



4.19.0.1 Two Continuous Variables

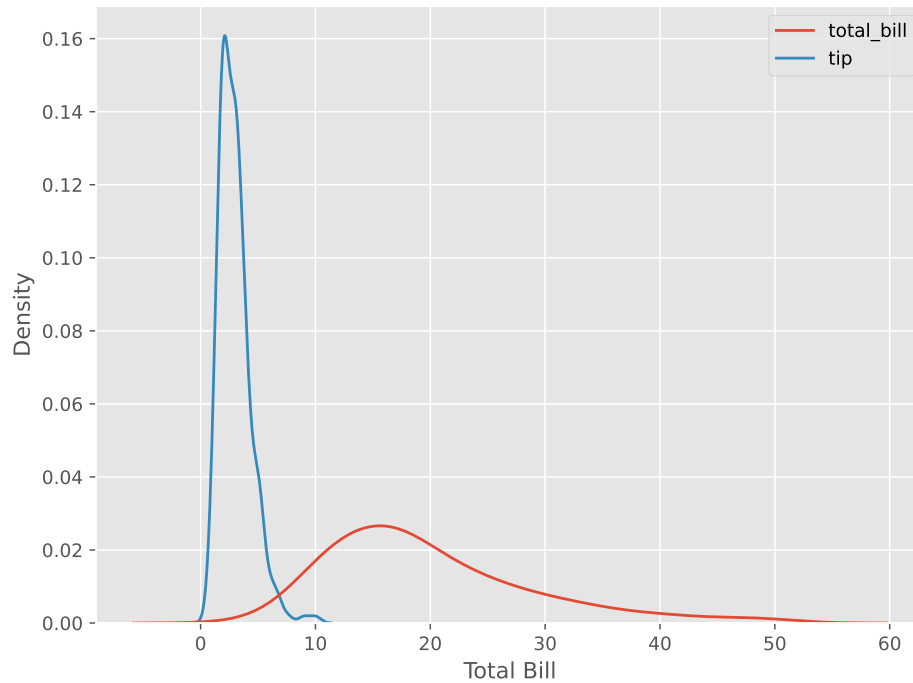
4.20 R

```
tips |>
  select(1:2) |>
  pivot_longer(cols = everything(), names_to = "types", values_to = "values") +
  ggplot(aes(x = values, col = types)) +
  geom_density(size = 1)
```



4.21 Python

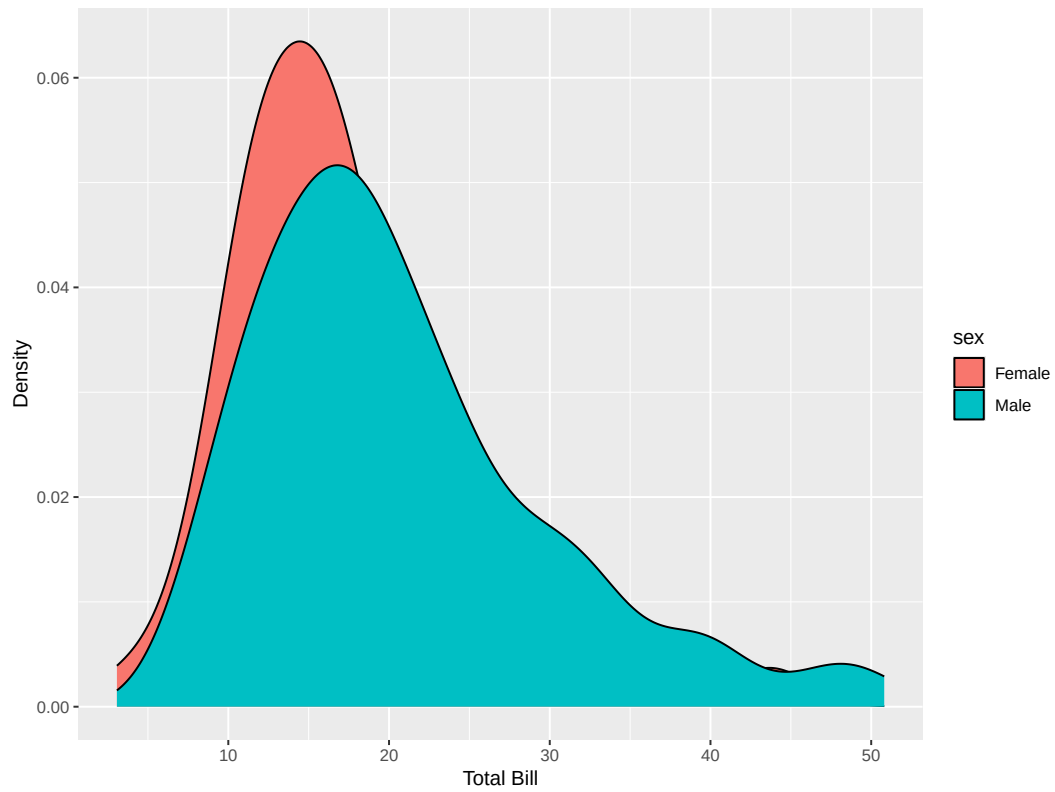
```
sns.kdeplot(data = tips[['total_bill', 'tip']])  
plt.xlabel("Total Bill")
```



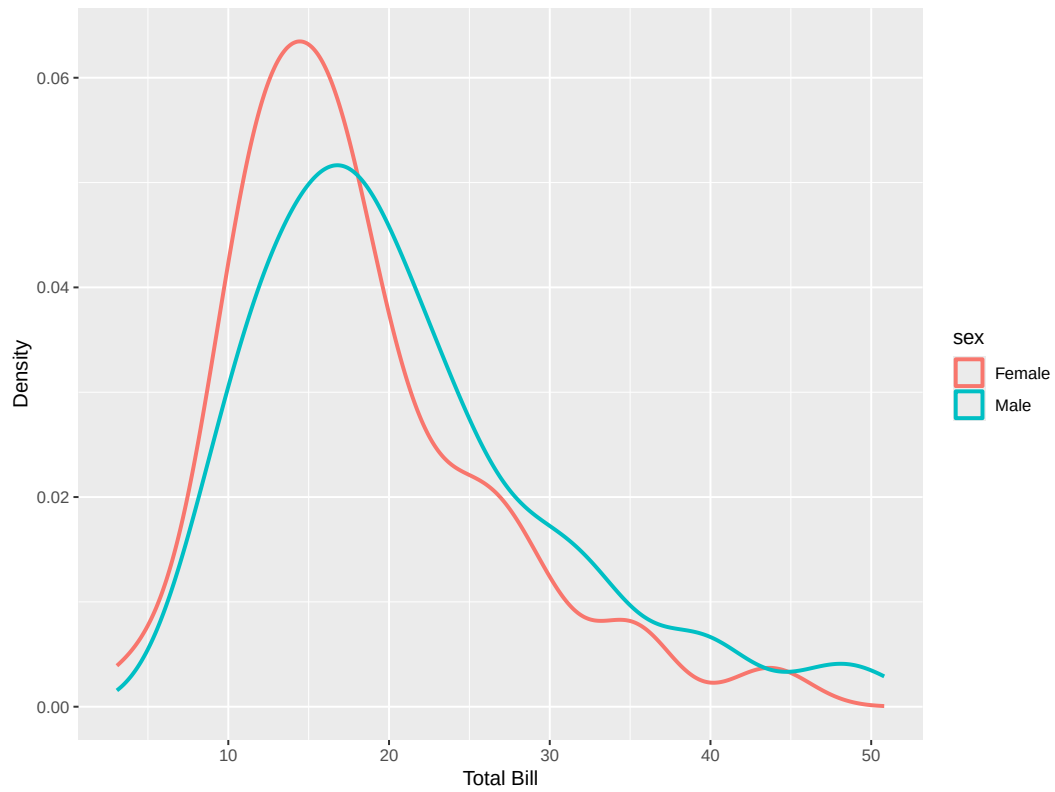
4.21.0.1 One Continuous Variable and One Categorical Variable

4.22 R

```
tips |>
  ggplot(aes(x = total_bill, fill = sex))+
  geom_density(
    #adjust = 0.2
  )+
  labs(x = "Total Bill", y = "Density")
```

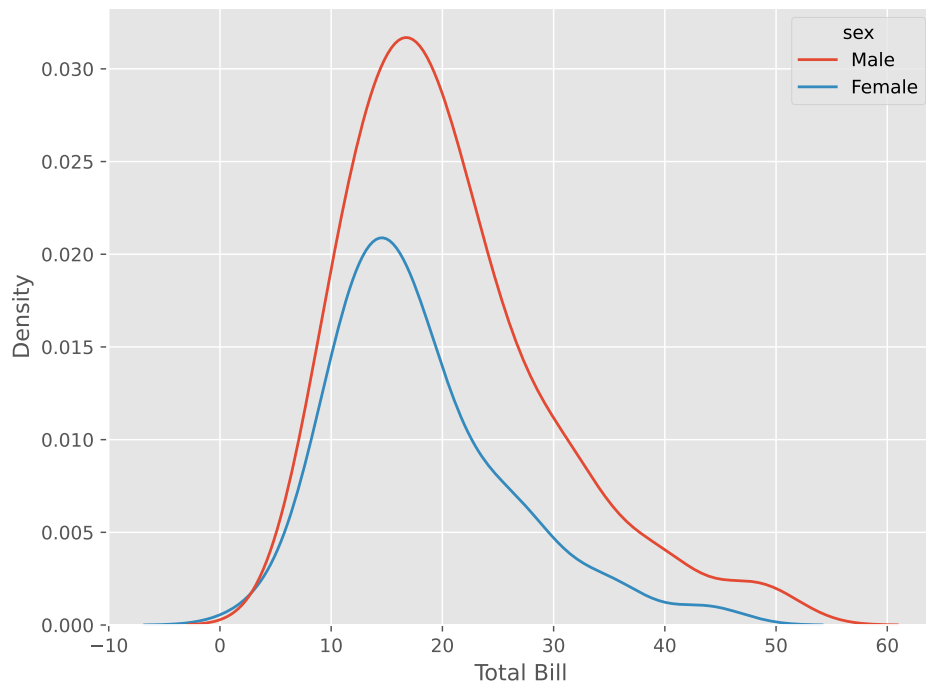


```
tips |>
  ggplot(aes(x = total_bill, color = sex)) +
  geom_density(size = 1
    #adjust = 0.2
  ) +
  labs(x = "Total Bill", y = "Density")
```

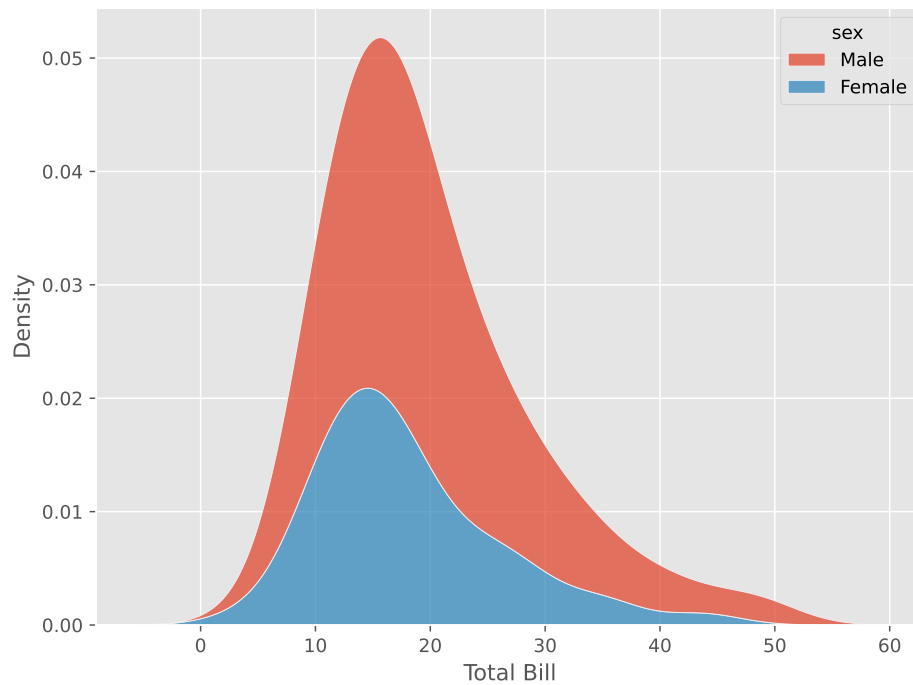


4.23 Python

```
sns.kdeplot(data = tips, x = "total_bill", hue = "sex")  
plt.xlabel("Total Bill")
```



```
sns.kdeplot(data = tips, x = "total_bill", hue = "sex", multiple = "stack")  
plt.xlabel("Total Bill")
```



4.23.1 Point Plot

4.23.1.1 One Categorical and One Continuous Variable

4.24 R

```
tips |>
  ggplot(aes(x = sex, y = total_bill, group = 1)) +
  stat_summary(aes(sex, total_bill), geom = "point", fun.y = mean, size = 2) +
  stat_summary(aes(sex, total_bill), geom = "line", fun.y = mean, size = 1) +
  labs(x = "Sex", y = "Total Bill")
```

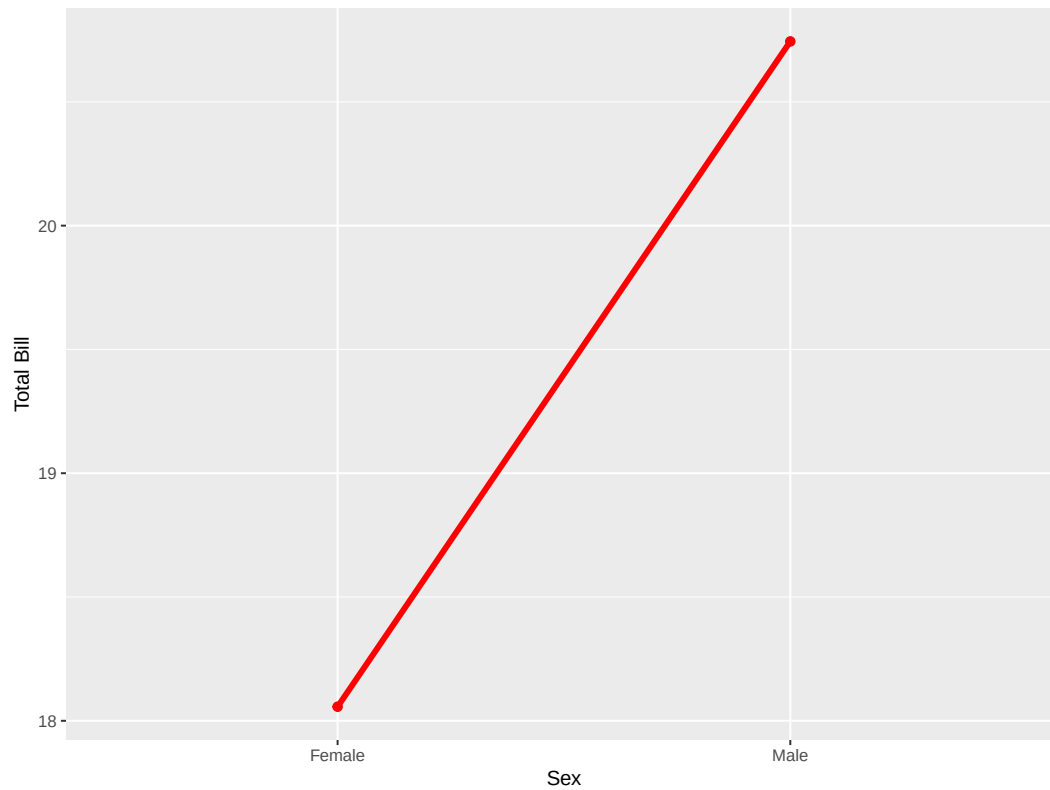
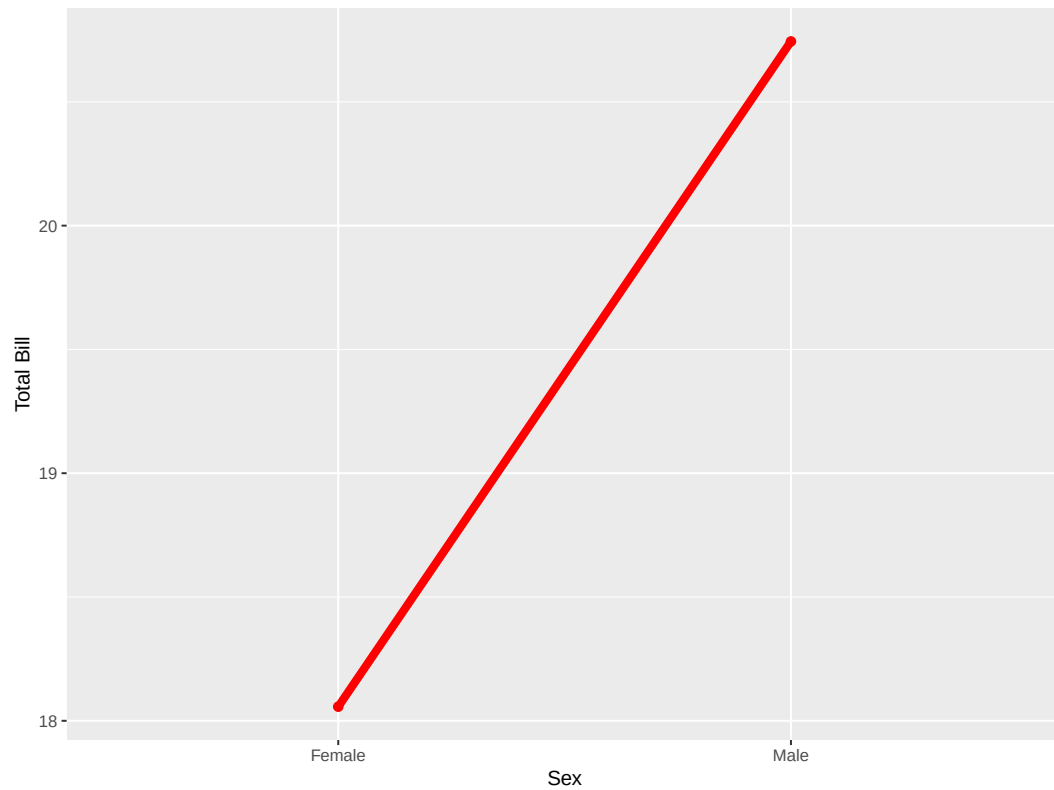



Figure 4.6: Line Plot of Gender (geom_line - mean)

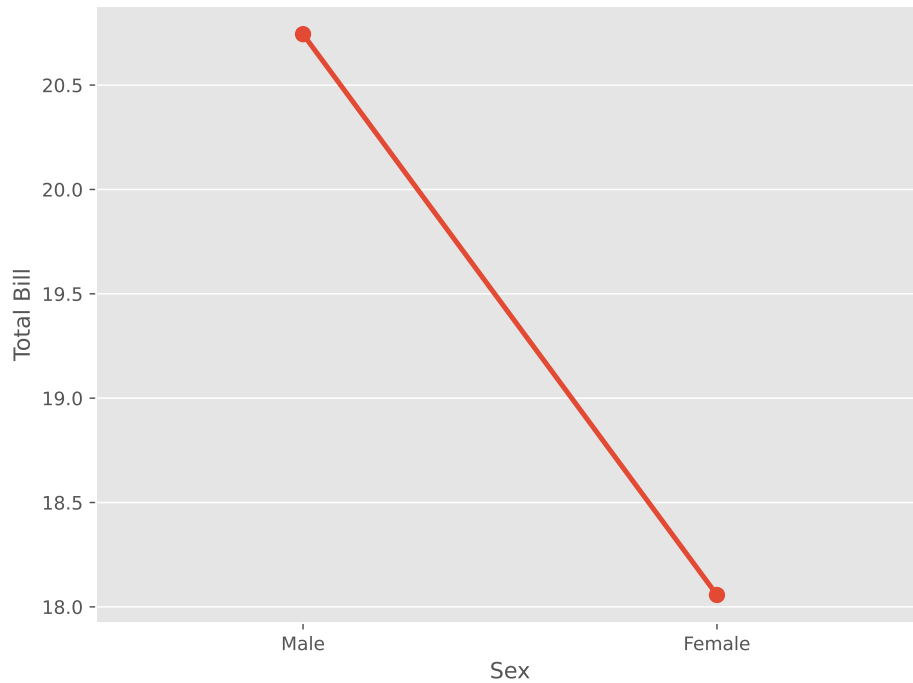
The following code will also produce the same visualization.

```
tips |>
  group_by(sex) |>
  summarize(total_bill = mean(total_bill)) |>
  ggplot(aes(x = sex, y = total_bill, group = 1)) +
  geom_point(col = "red", size = 2) +
  geom_line(col = "red", size = 2.1) +
  labs(x = "Sex", y = "Total Bill")
```



4.25 Python

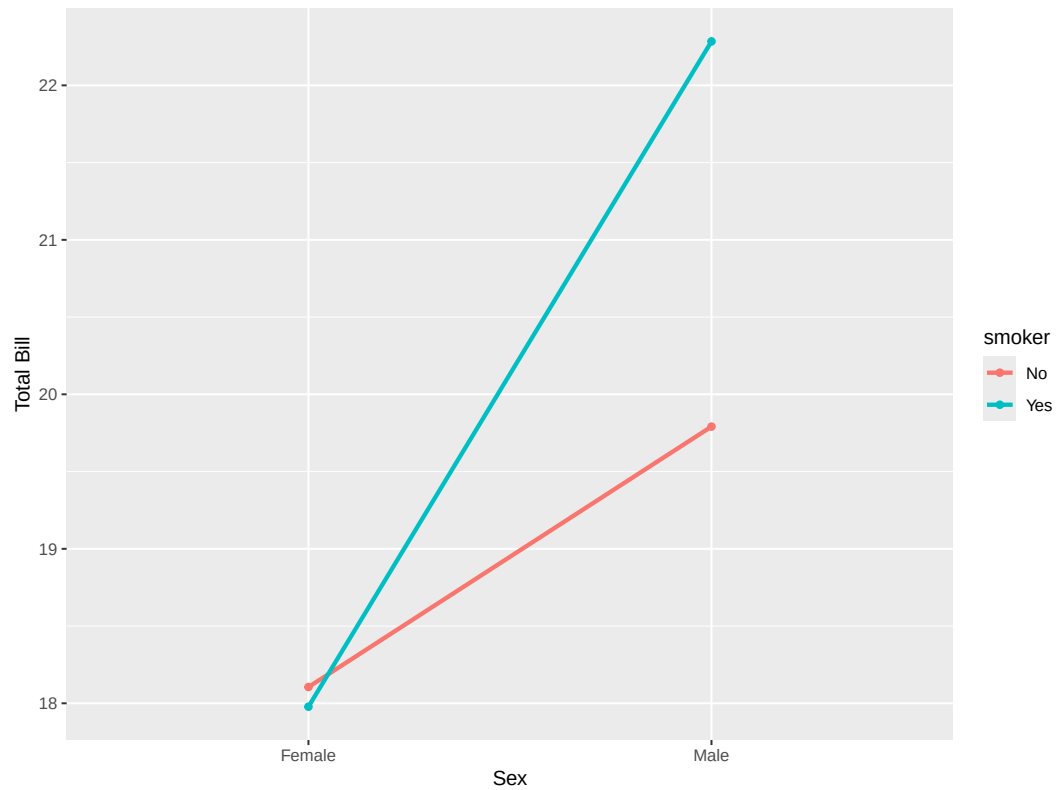
```
sns.pointplot(data = tips, x = "sex", y = "total_bill", errorbar=None)
plt.xlabel('Sex')
plt.ylabel("Total Bill")
```



4.25.0.1 Two Categorical Variables and One Continuous Variable

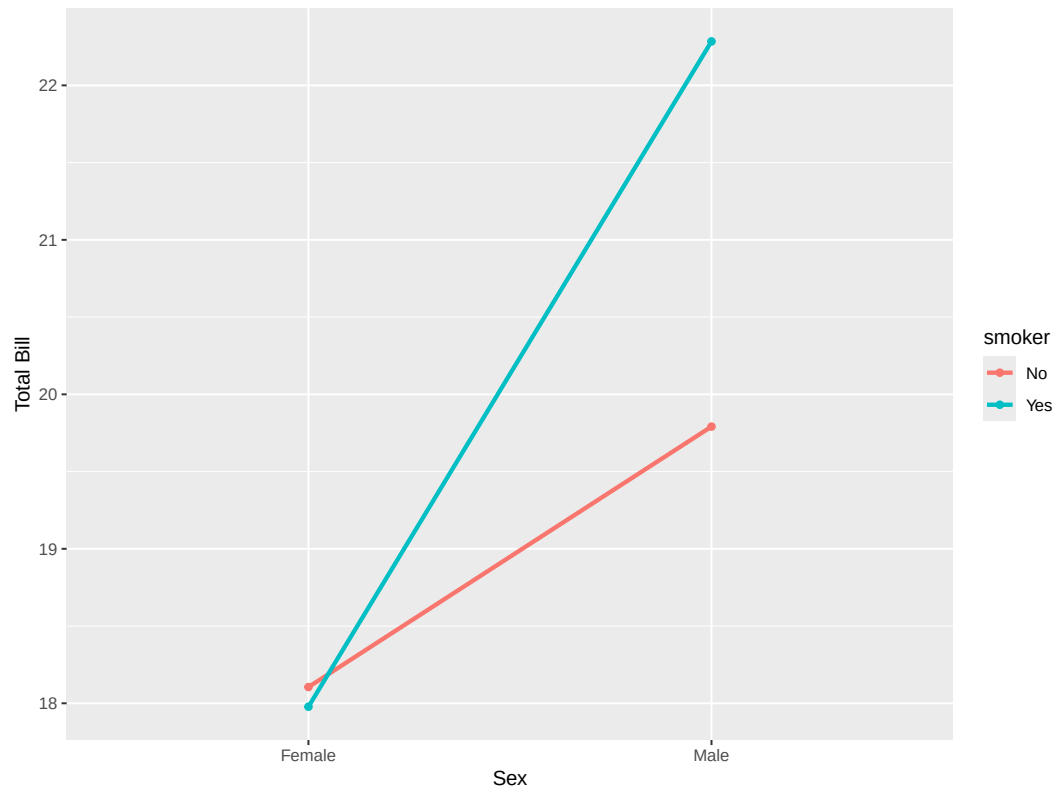
4.26 R

```
tips |>
  ggplot(aes(x = sex, y = total_bill, group = smoker, color = smoker)) +
  stat_summary(aes(x = sex, y = total_bill), geom = "point", fun.y = mean) +
  stat_summary(aes(x = sex, y = total_bill), geom = "line", fun.y = mean, color = "Smoker")
  labs(x = "Sex", y = "Total Bill" #, color = "Smoker")
  )
```



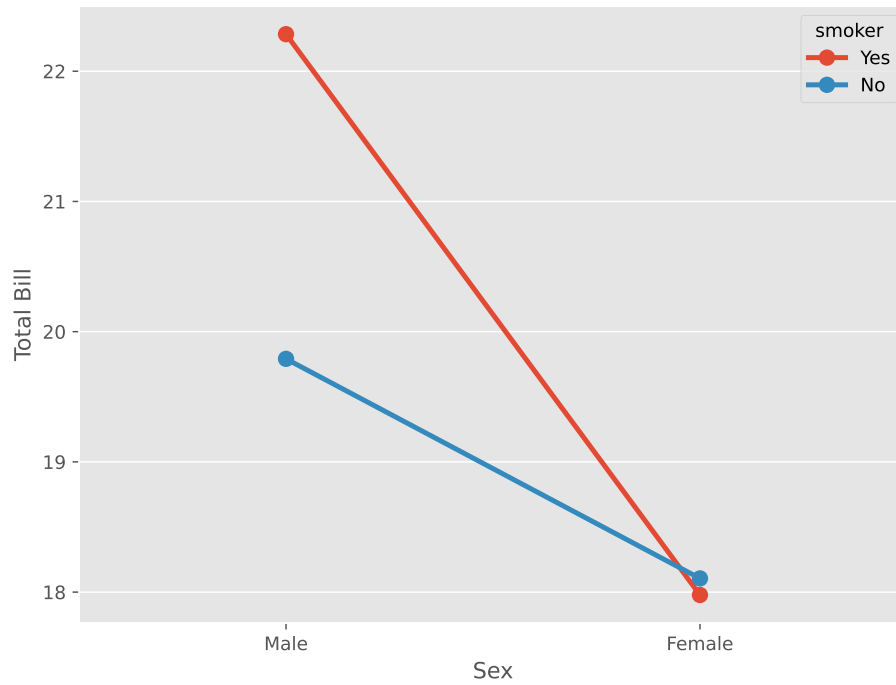
The following code will also produce the same visualization.

```
tips |>
  group_by(sex, smoker) |>
  summarize( total_bill = mean(total_bill)) |>
  ggplot(aes(x = sex, y = total_bill, group = smoker , color = smoker)) +
  geom_point()+
  geom_line(size = 1.1)+
  labs(x = "Sex", y = "Total Bill")
```



4.27 Python

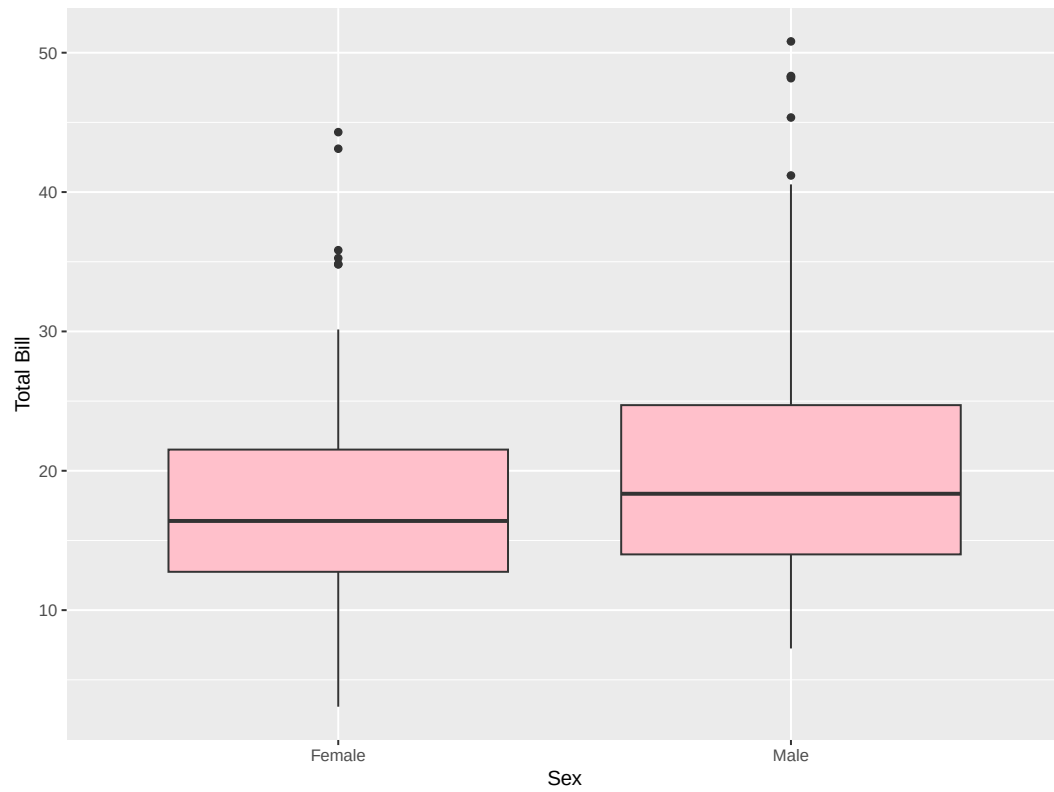
```
sns.pointplot(data = tips, x = "sex", y = "total_bill",  
               hue = "smoker", errorbar= None)  
plt.xlabel("Sex")  
plt.ylabel("Total Bill")
```



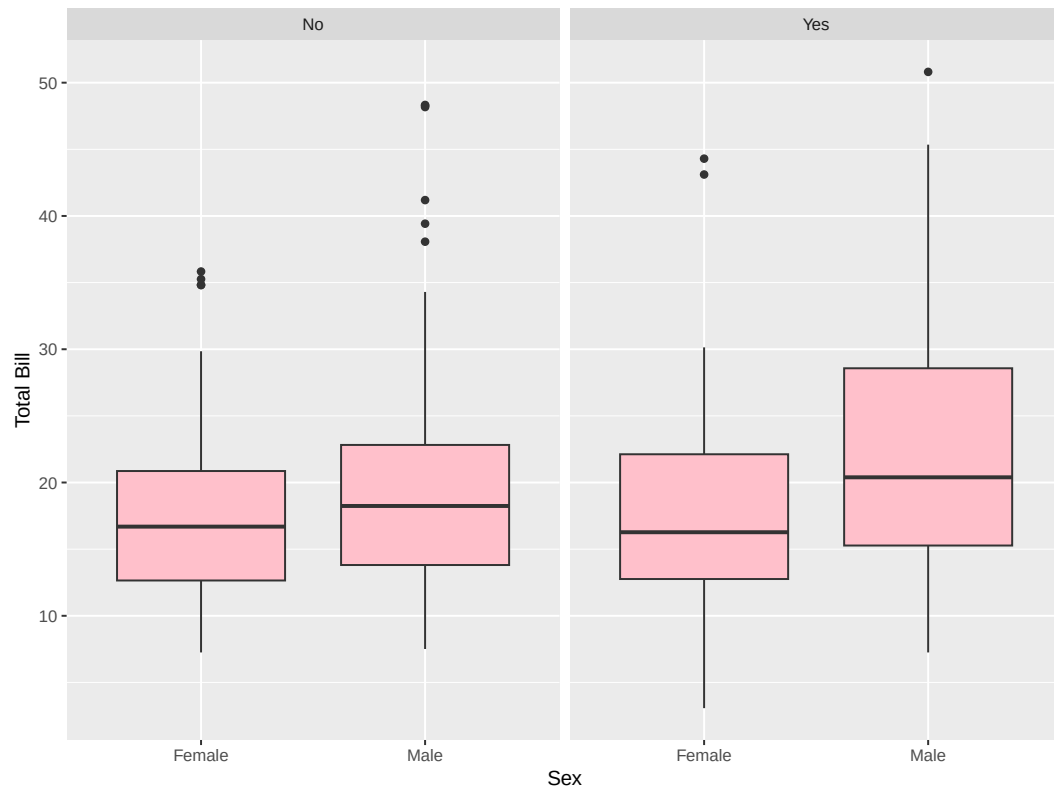
4.27.1 Box Plot

4.28 R

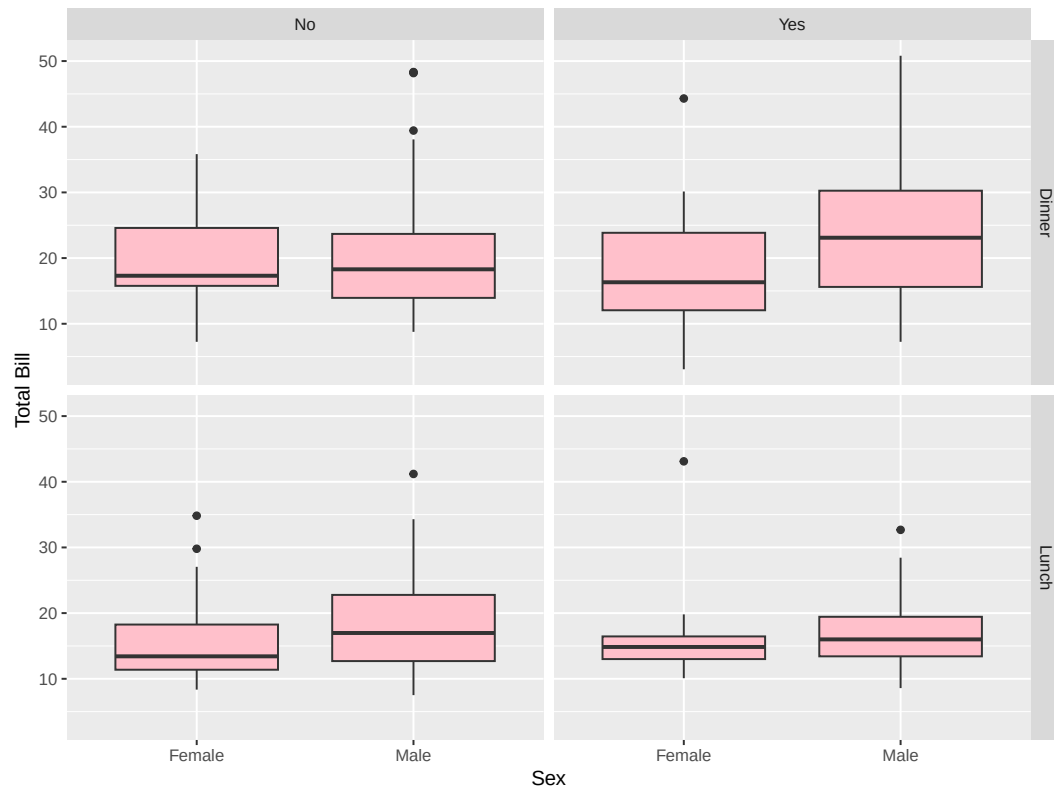
```
tips |>
  ggplot(aes(x = sex, y = total_bill)) +
  geom_boxplot(fill = "pink") +
  labs (x = "Sex", y = "Total Bill")
```



```
tips |>
  ggplot(aes(x = sex, y = total_bill))+
  geom_boxplot(fill = "pink") +
  labs (x = "Sex", y = "Total Bill") +
  facet_wrap(~smoker)
```

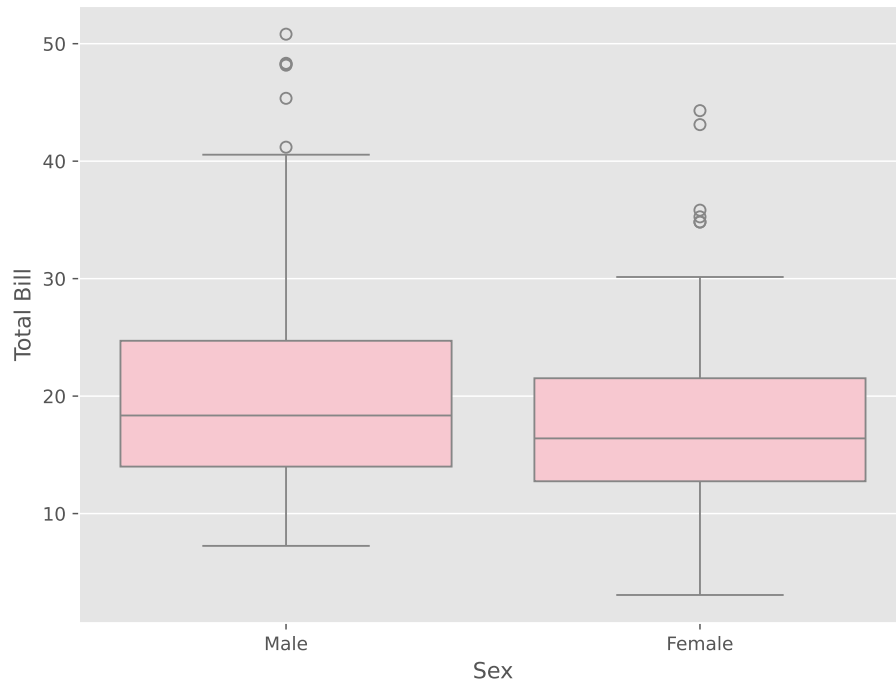


```
tips |>
  ggplot(aes(x = sex, y = total_bill))+
  geom_boxplot(fill = "pink") +
  labs (x = "Sex", y = "Total Bill") +
  facet_grid(time~smoker)
```

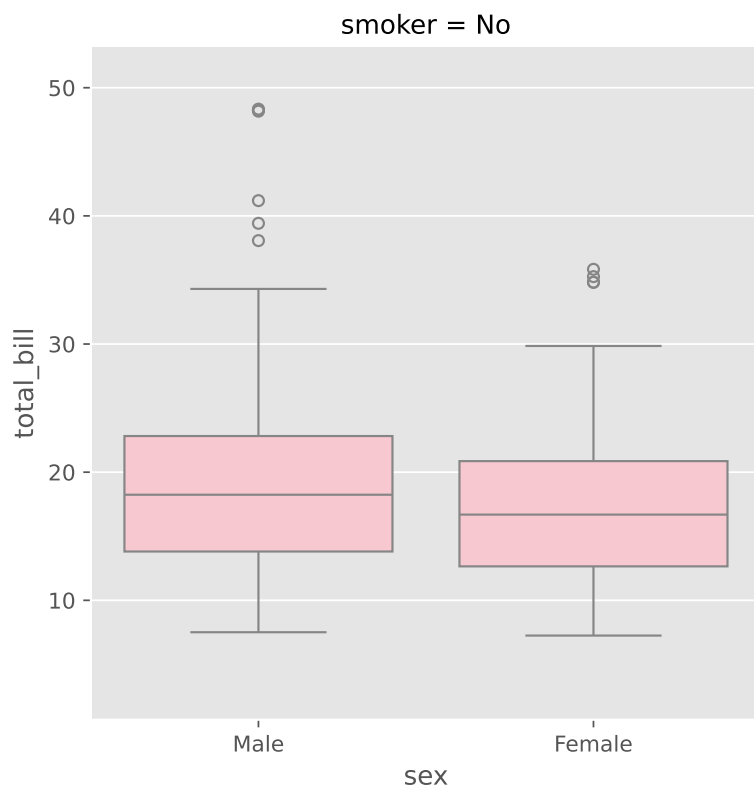
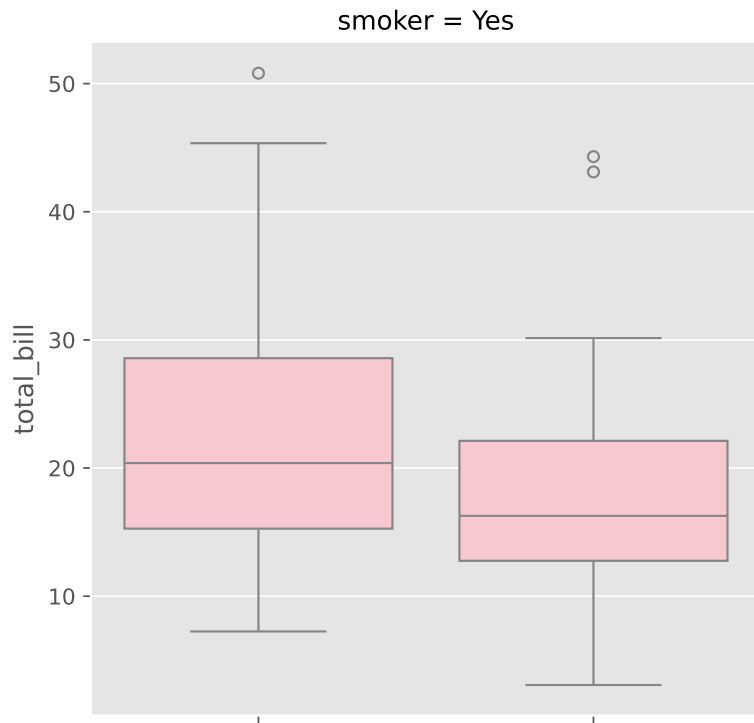



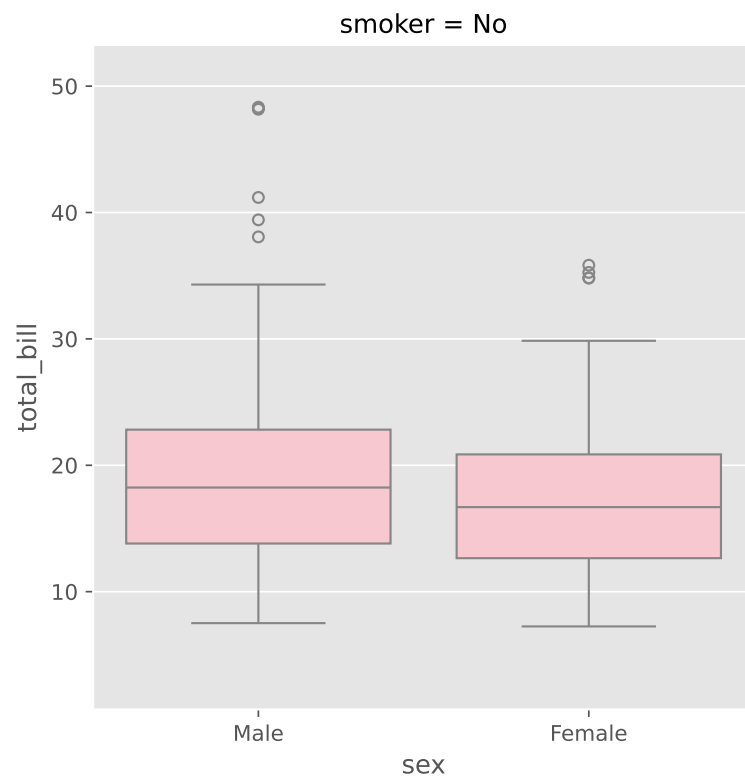
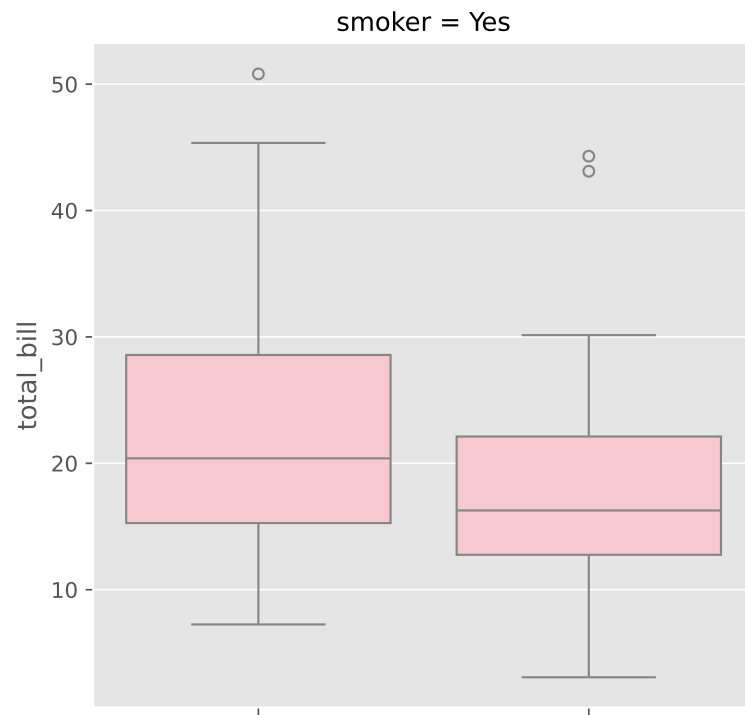
4.29 Python

```
sns.boxplot(data = tips, x = "sex", y = "total_bill", color = "pink")
plt.xlabel("Sex")
plt.ylabel("Total Bill")
```

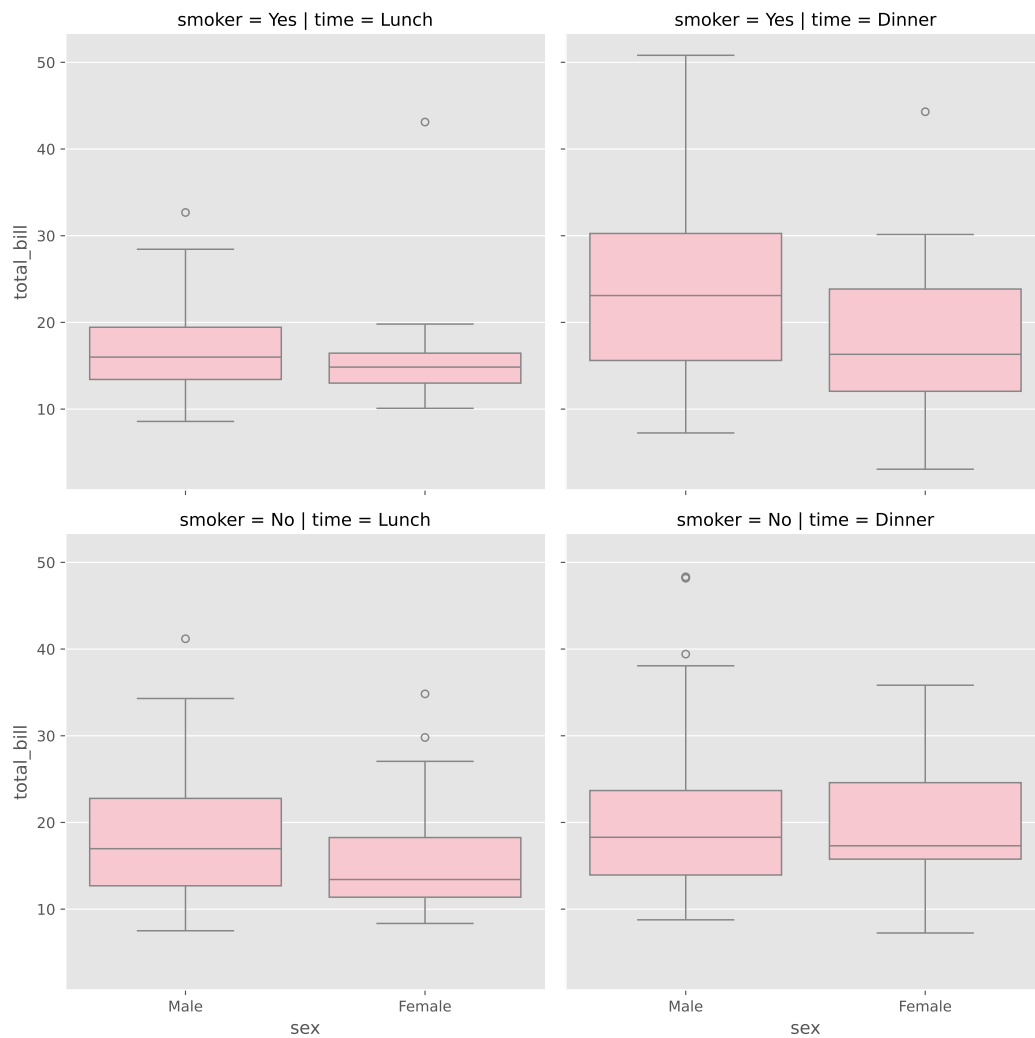


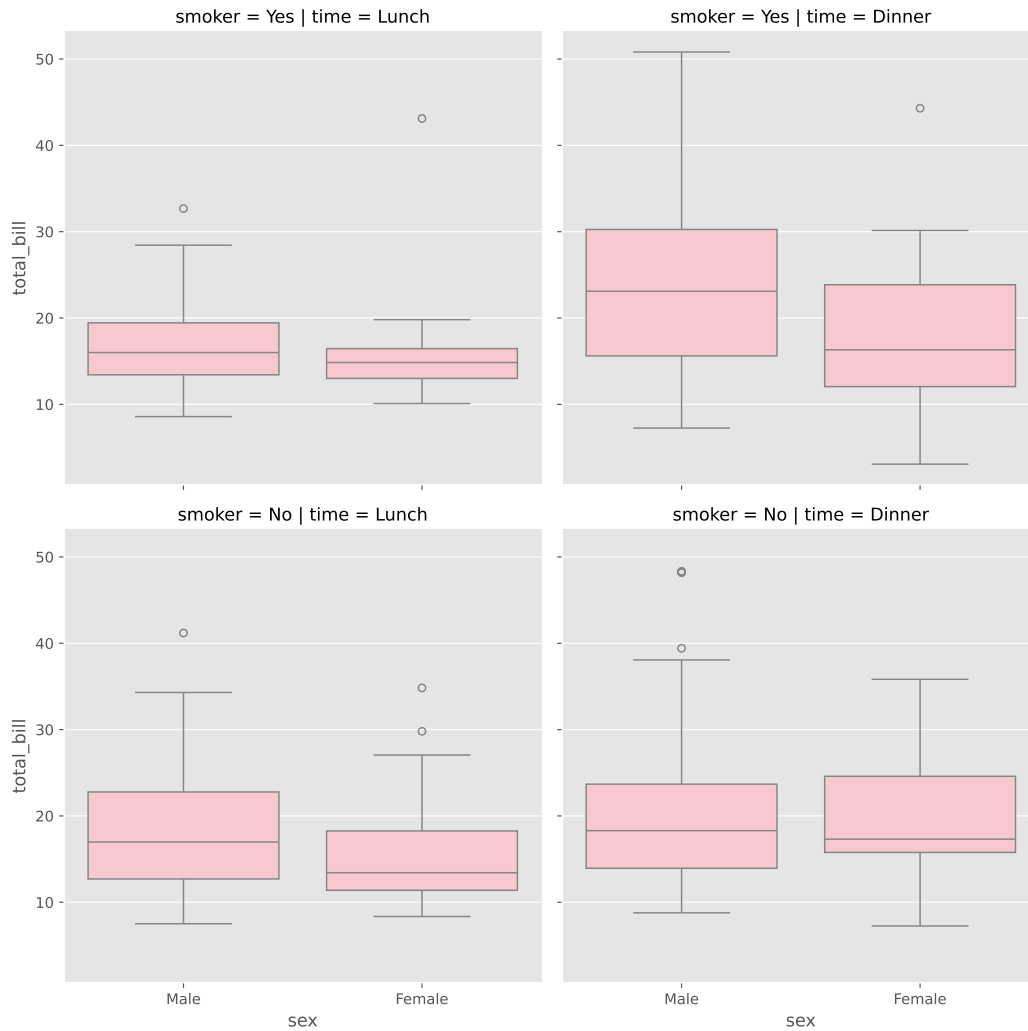
```
sns.catplot(data = tips, x = "sex", y = "total_bill",  
            color = "pink", kind = "box", row = "smoker"  
            )
```





```
sns.catplot(data = tips, x = "sex", y = "total_bill",
            color = "pink", kind = "box", row = "smoker",
            col = "time"
            )
```

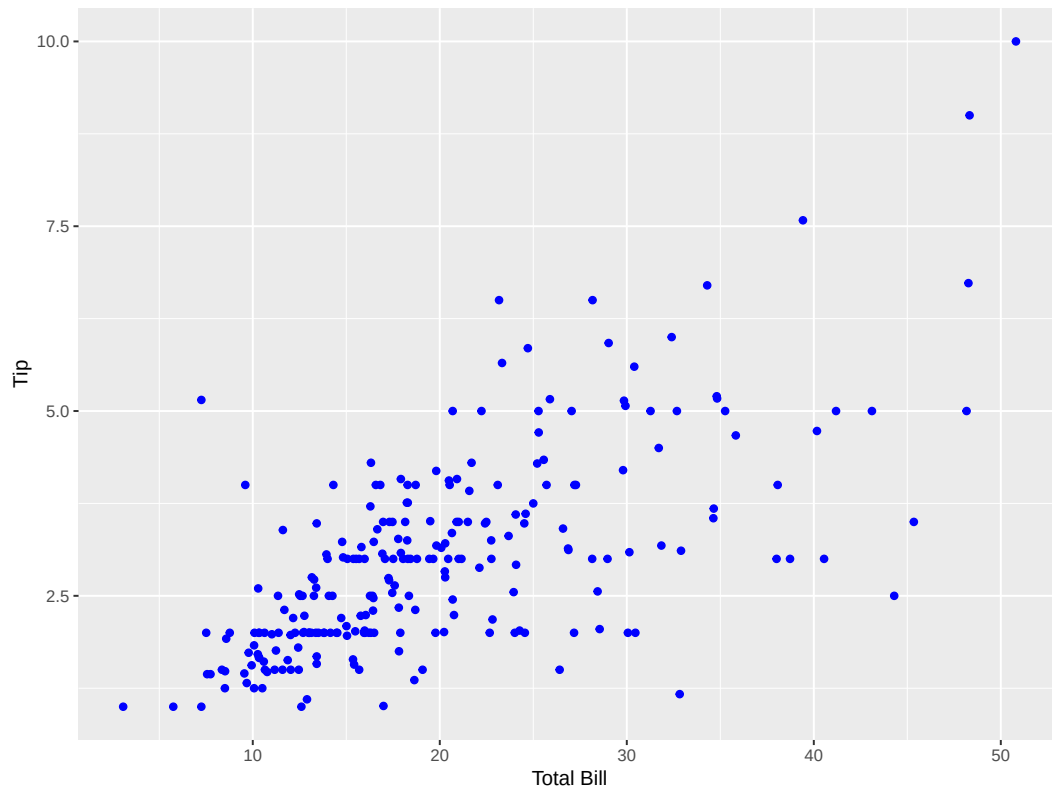




4.29.1 Scatter Plot

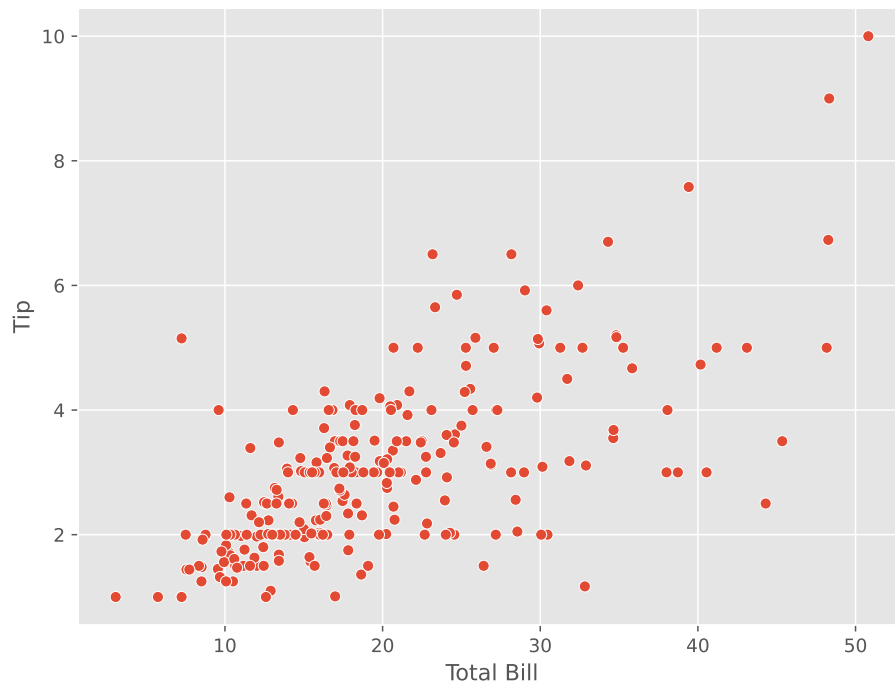
4.30 R

```
tips |>
  ggplot(aes(x = total_bill, y = tip))+
  geom_point(col = "blue")+
  labs(x = "Total Bill", y = "Tip")
```



4.31 Python

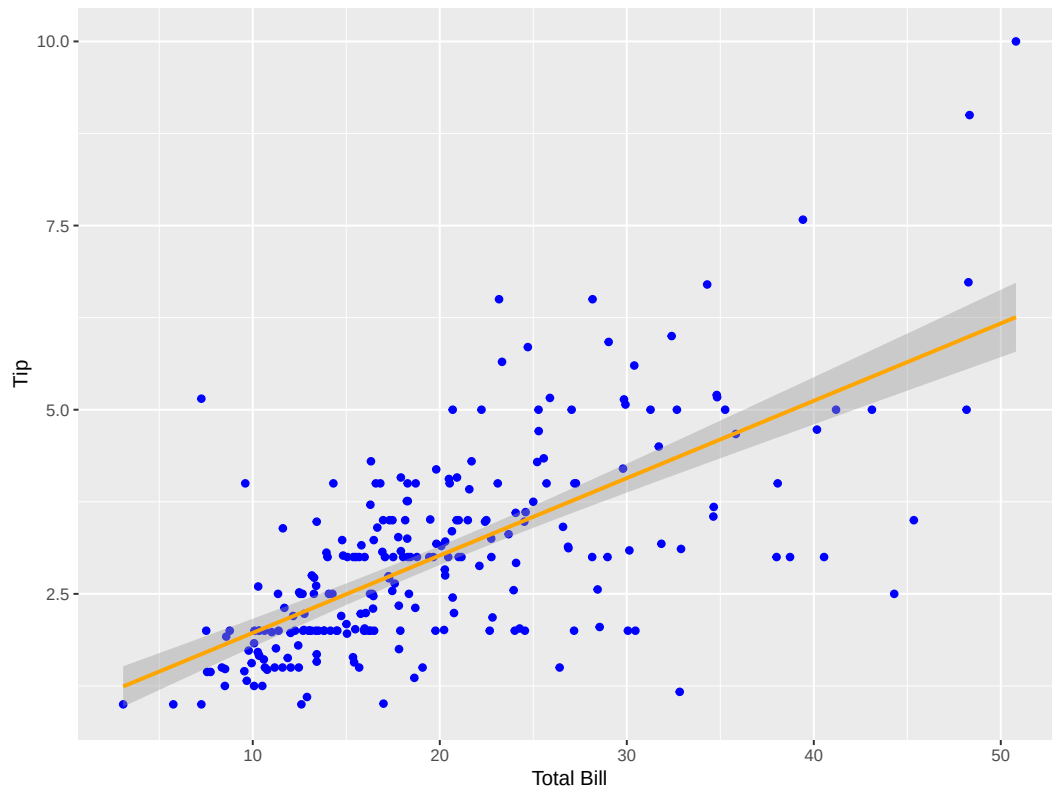
```
sns.scatterplot(data = tips, x = "total_bill", y = "tip")  
plt.xlabel("Total Bill")  
plt.ylabel("Tip")
```



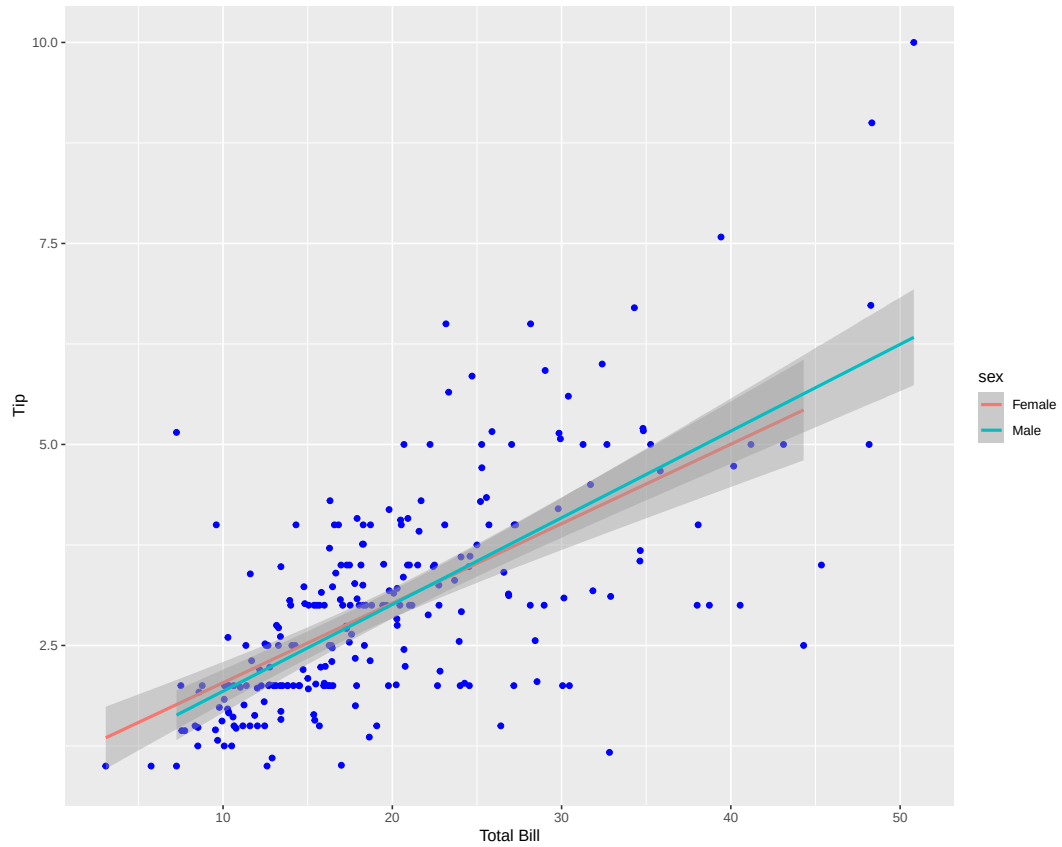
4.31.1 Regression Plot

4.32 R

```
tips |>
  ggplot(aes(x = total_bill, y = tip))+
  geom_point(col = "blue")+
  geom_smooth(method = "lm", col = "orange") +
  labs(x = "Total Bill", y = "Tip")
```

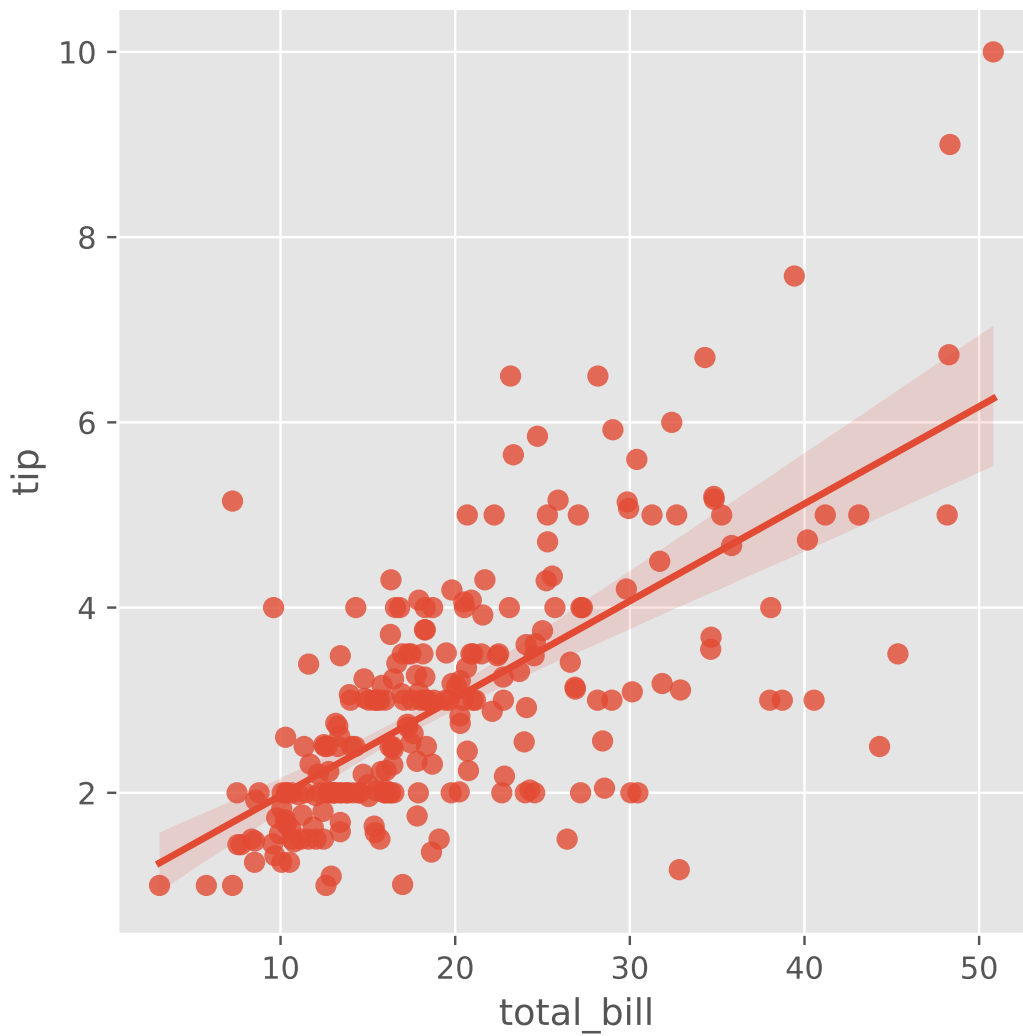



```
tips |>
  ggplot(aes(x = total_bill, y = tip, col = sex))+
  geom_point(col = "blue")+
  geom_smooth(method = "lm") +
  labs(x = "Total Bill", y = "Tip")
```

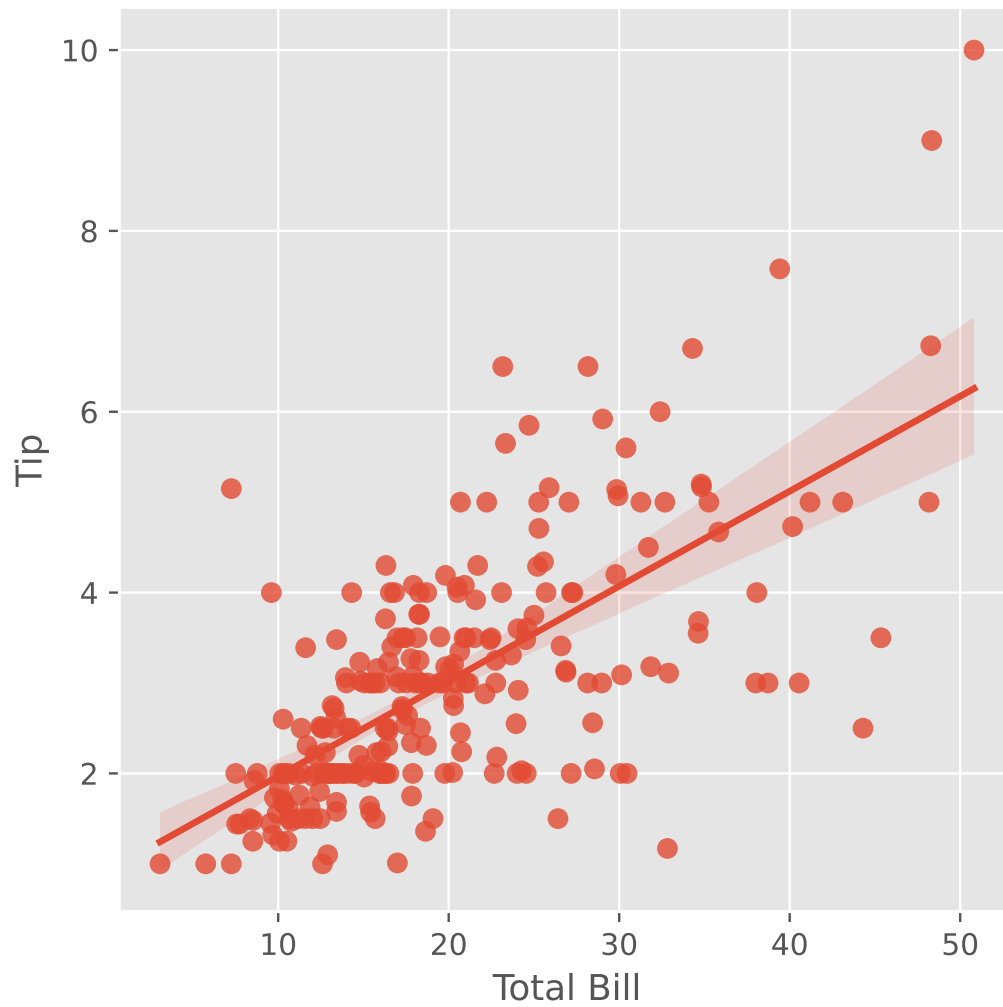


4.33 Python

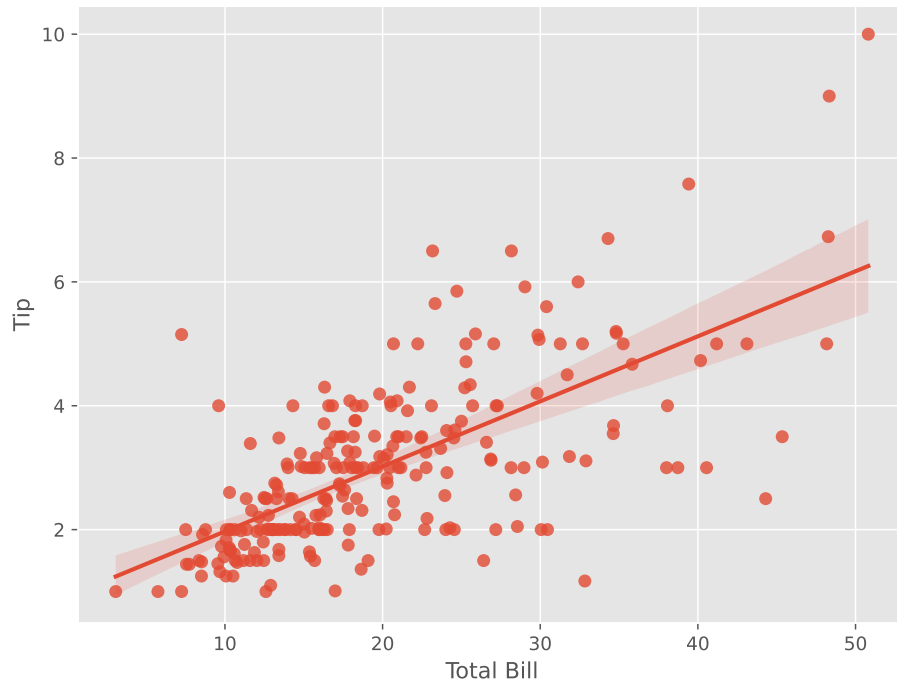
```
sns.lmplot(data = tips, x = "total_bill", y = "tip")
```



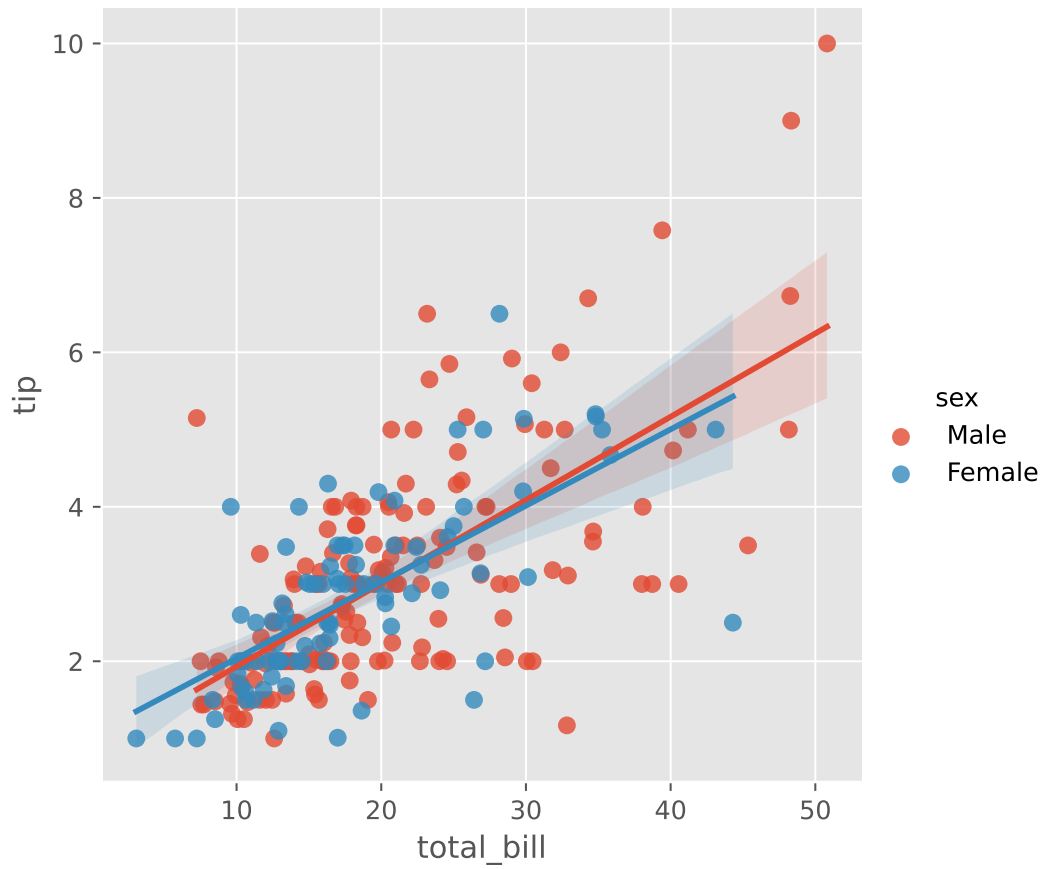
```
plt.xlabel("Total Bill")  
plt.ylabel("Tip")
```



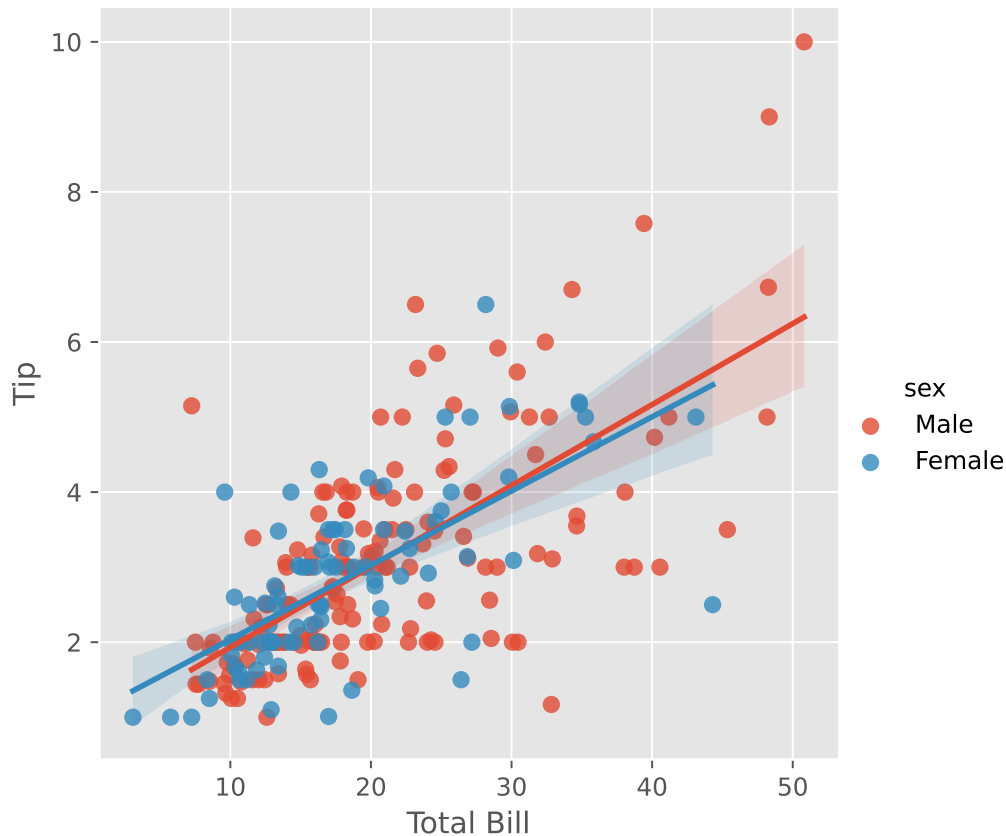
```
sns.regplot(data = tips, x = "total_bill", y = "tip")  
plt.xlabel("Total Bill")  
plt.ylabel("Tip")
```



```
sns.lmplot(data = tips, x = "total_bill", y = "tip", hue = "sex")
```



```
plt.xlabel("Total Bill")  
plt.ylabel("Tip")
```



4.34 Exercises # 01

1. Download student data from the [url](#) and create a pointplot (lineplot) of students average math score (`math.grade`) of gender (`gender`). Please note that the variable `gender` includes a label called `other` in addition to M and F; you should filter out observations of the label `other` before you create visualization.
2. From the dataset in above (question 1), compare, using pointplot (lineplot), the average math (`math.grade`) and science score (`sciences.grade`) of different students based on gender (`gender`). You might need to use `pivot_longer` function to reshape the data frame before visualizing the relation.

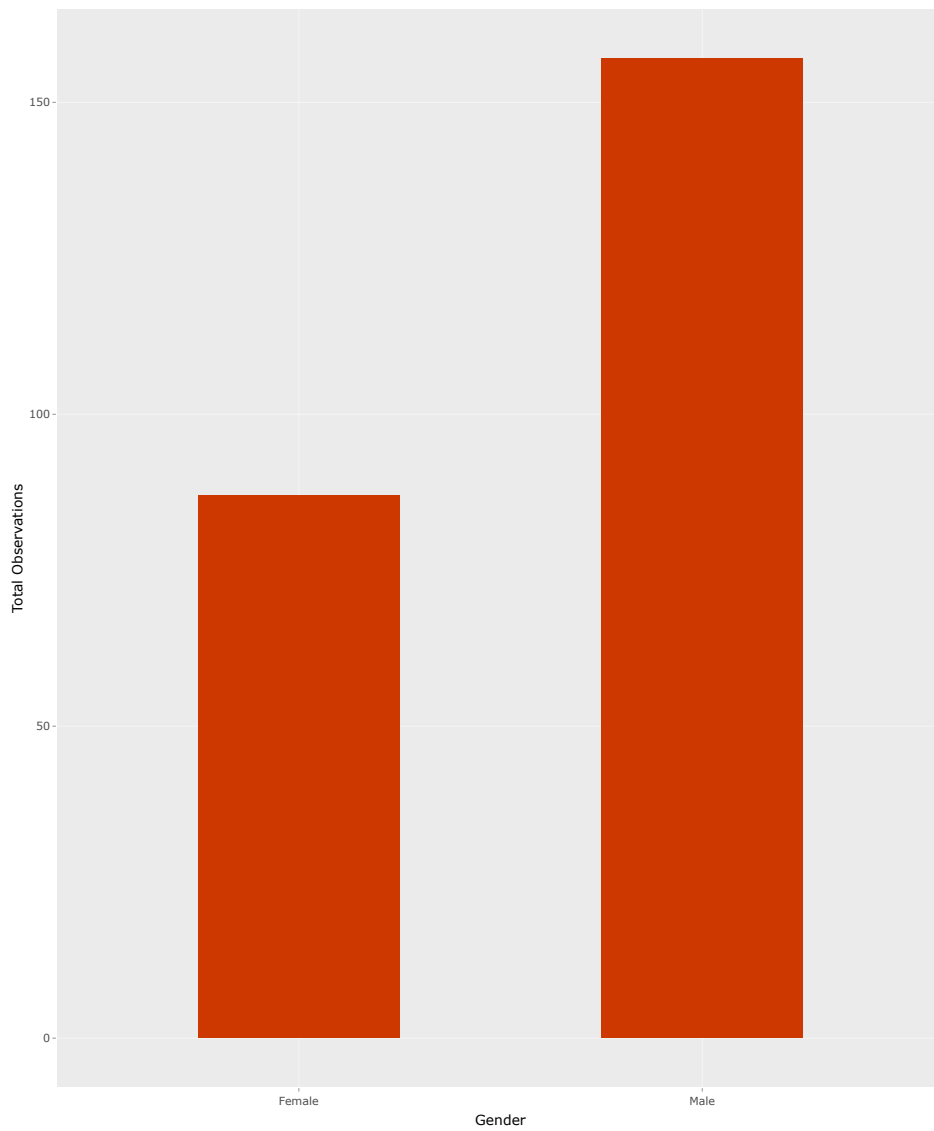
4.35 Interactive Visualization

Interactive Visualization involves graphical presentation of data that permits users to engage with the visual elements directly. Unlike static visualization, interactive visualization allows users to manipulate data, explore different aspects, and customize the visualization in real time. The primary objective of interactive visualization is to make data exploration more intuitive and dynamic. The benefits of interactive visualization include - enhanced engagement, deeper insights, customization, and exploration and discovery.

4.36 R

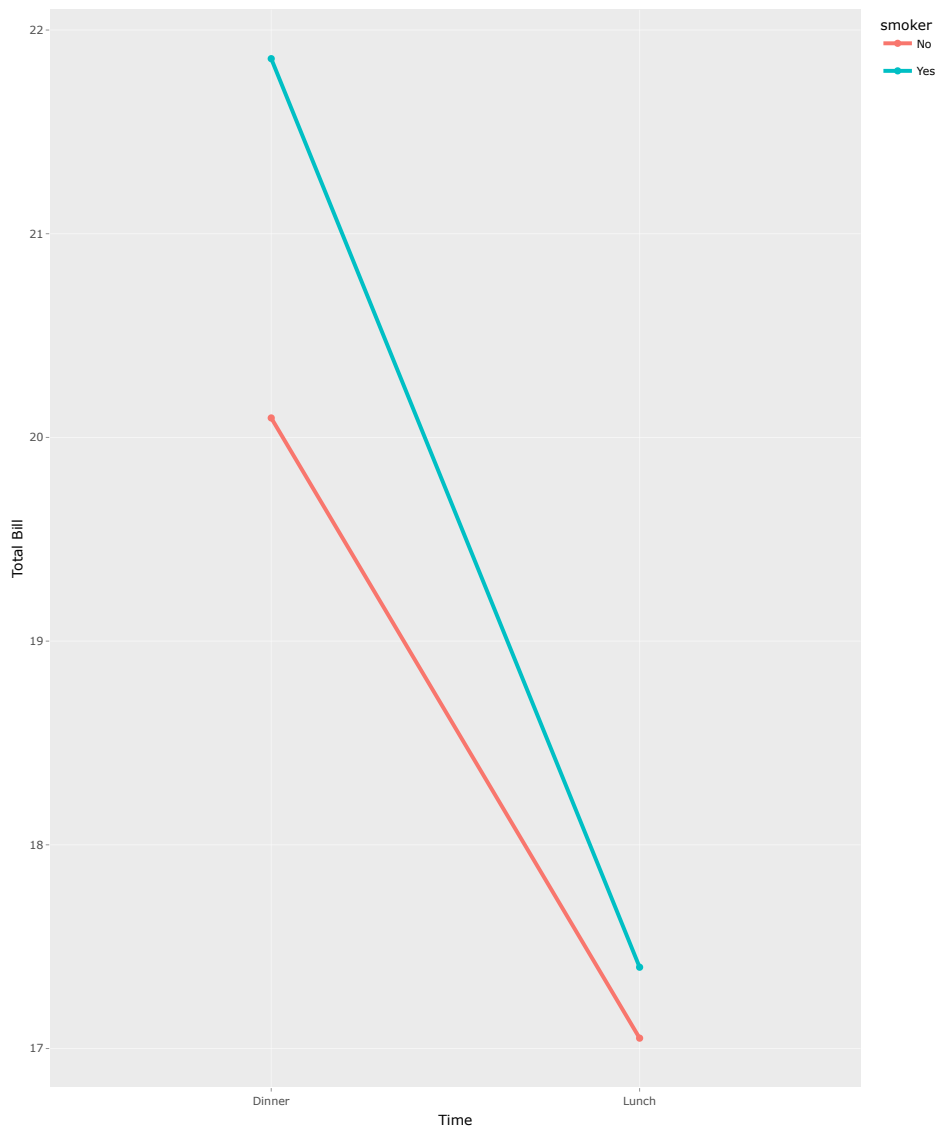
```
library(plotly)
```

```
p = ggplot(data = tips, aes(x = sex)) +  
  geom_bar(width = 0.5, fill = "orangered3") +  
  labs(x = "Gender", y = "Total Observations")  
  
ggplotly(p)
```

```
p2 = tips |>  
  ggplot(aes(x = time, y = total_bill, group = smoker, color = smoker))+  
  stat_summary(aes(x = time, y = total_bill), geom = "point", fun.y = mean)
```

```
    stat_summary(aes(x = time, y = total_bill), geom = "line", fun.y = mean,  
    labs (x = "Time", y = "Total Bill")  
ggplotly(p2)
```



4.37 Python

```
#!pip install plotly
import plotly.express as px
```

```
fig = px.histogram(tips, x = "sex") \
    .update_traces(marker_color = "orangered") \
    .update_xaxes(title = "Sex") \
    .update_yaxes(title = "Count")
fig.show()
```

```
px.histogram(tips, x = "sex", y = "total_bill", histfunc='avg') \
    .update_traces(marker_color = "orangered") \
    .update_xaxes(title = "Sex") \
    .update_yaxes(title = "Average Total Bill") \
    .show()
```

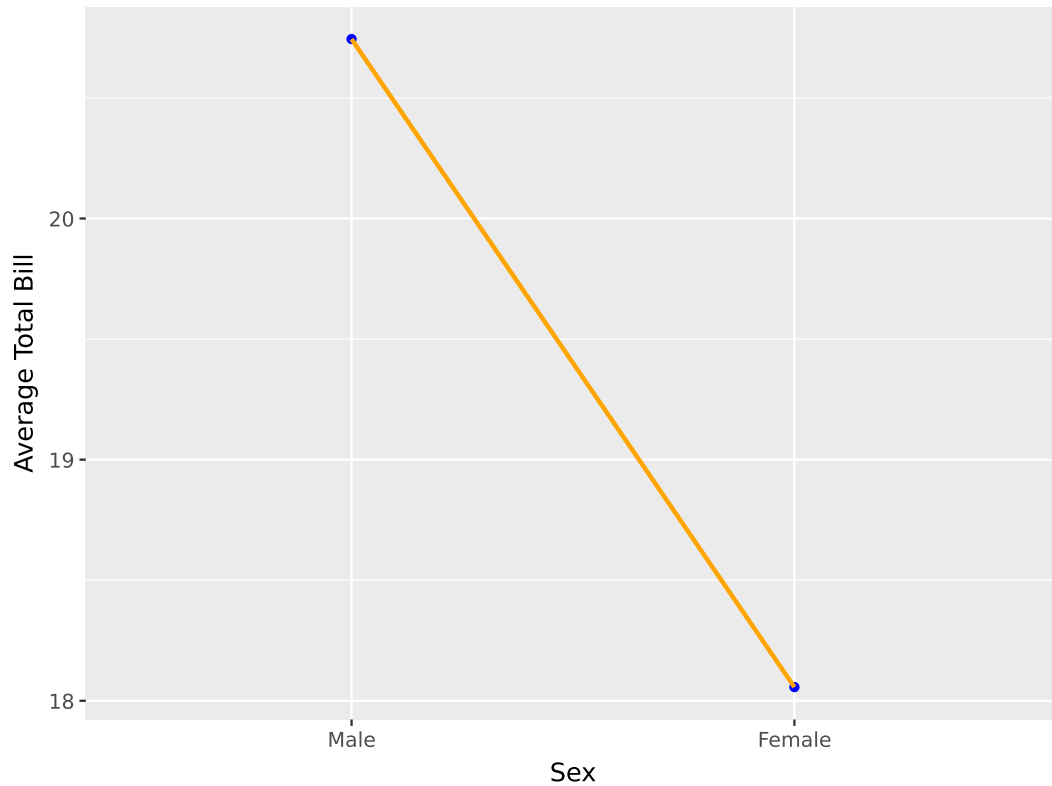
```
px.histogram(tips, x="total_bill", histnorm='probability density',
             width=600, height=400) \
    .update_xaxes(title = "Total Bill") \
    .update_yaxes(title = "Density")
```

4.38 Plotnine

```
#import plotnine as p9
from plotnine import *
import plotly.tools as tls
```

```
df = tips.groupby(["sex"])["total_bill"].agg('mean').reset_index()
```

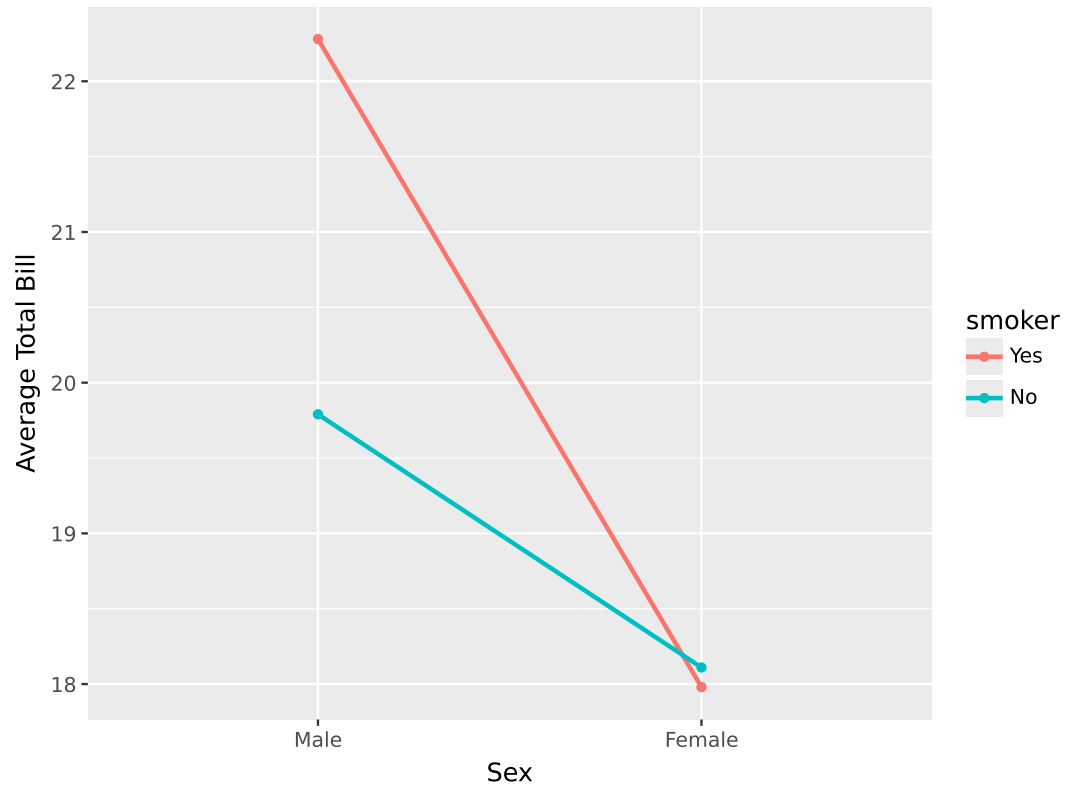
```
(
    ggplot(df, aes(x = "sex", y = "total_bill", group = 1)) +
    geom_point(color = "blue")+
    geom_line(color = "orange", size = 1.1) +
    labs(x = "Sex", y = "Average Total Bill")
)
```



```
plotly_fig = (
    ggplot(df, aes(x = "sex", y = "total_bill", group = 1)) +
    geom_point(color = "blue")+
    geom_line(color = "orange", size = 1.1)
)
tls.mpl_to_plotly(plotly_fig.draw()).show()
```

```
df2 = tips.groupby(["sex", "smoker"])["total_bill"] \
    .agg('mean') \
    .round(2) \
    .reset_index()
```

```
(
    ggplot(df2, aes(x = "sex", y = "total_bill", group = "smoker", color = "smoker")) +
    geom_point() +
    geom_line(size = 1.1) +
    labs(x = "Sex", y = "Average Total Bill")
)
```



4.39 Exercises # 02

5 Dashboard for Visualization

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Learn about the Meaning of Dashboard for Visualization
- Understand the Importance of Dashboard Visualization
- Learn about Different R and Python Packages for Dashboard Creation
- Create Dashboard using `Quarto` and `Vizro`

5.1 What is Dashboard?

A dashboard for visualization is a user interface that displays a collection of visual data representations, such as charts, graphs, tables, and metrics, to provide users with an interactive and comprehensive overview of key information. Dashboards are commonly used in business, data science, finance, health-care, and other fields to monitor performance, track metrics, and explore data trends in real time.

The main goal of a dashboard is to present complex data in an easy-to-understand format, enabling users to quickly grasp insights and make data-driven decisions. Dashboards often combine multiple visual elements into a single screen or page, allowing users to view different aspects of the data simultaneously. They typically include interactive features like filters, drill-downs, and tooltips, which allow users to interact with the data and explore deeper insights without needing to understand the underlying data structures. Some benefits of dashboards are -

- **Data Aggregation:** Dashboards bring together data from various sources, providing a unified view of different datasets.
- **Visualization Elements:** They use visual elements such as bar charts, line graphs, pie charts, heatmaps, and more to represent data in a visually appealing and informative way.

- **Interactivity:** Users can interact with the visual elements, applying filters, adjusting time frames, or drilling down into specific data points to gain more detailed insights.
- **Real-Time Data:** Dashboards can display real-time data, updating visualizations dynamically to reflect the latest information, which is especially useful for monitoring live systems or business performance.
- **Customization:** Dashboards are highly customizable, allowing users to tailor the layout, visualizations, and data to meet their specific needs.

Dashboards for visualization are widely used for performance monitoring (e.g., tracking KPIs), data exploration (e.g., identifying trends), and decision-making (e.g., comparing metrics). They make data analysis more accessible to a wider audience by simplifying the complexity of raw data into clear and actionable insights.

5.2 Importance of Dashboard

5.3 Interactive Dashboard

For interactive visualization, see Figure ??.

5.4 Quarto - R and Python Dashboard

Quarto Dashboard is a powerful and flexible open-source tool to create interactive dashboard in R or Python. Quarto dashboards are easy to create and support a wide variety of visualization and interactive components¹. More about quarto dashboard can be learned from [Quarto Dashboard Web-site](#). To learn more about interactivity on quarto dashboard using Shiny, please visit webpages for [R](#) and [Python](#). However, it is recommended to use [Python](#) for quarto dashboard if you want to include interactive applications on your dashboard.

There are several components of quarto dashboard:

- Navigation Bar - Top page bar with icon, title of the dashboard, name of author and links to sub-pages
- Pages, Rows, Columns, and Tabsets - Using markdown headings (#) - pages, rows and columns are defined. Tabsets are used to further divide the content within a row or column

¹Shiny widgets and functionality can be incorporated in the quarto dashboard. Therefore, quarto dashboard is a powerful tool for creating interactive visualization.

- Cards, Sidebar, and Toolbars - Cards are containers for text, images, charts, and interactive elements and useful for organizing information into distinct sections within a dashboard. Typically, the contents of cards map to cells in the dashboard. Sidebar is another layout component of quarto dashboard, which contain navigation menus, filters, and controls that allow users to adjust or explore the data presented in the main content. Toolbars

The first step to create a quarto dashboard is to structure YAML in .qmd file. A quarto dashboard YAML look like -

```

---
title: "A simple Quarto Dashboard"
format: dashboard
---

```

Figure 5.1: Quarto Dashboard YAML

In quarto dashboard, each level 1 header (#) introduces a new page, each level 2 header (##) introduces a new row within the given page, and each code chunk within a given row introduces a new column.

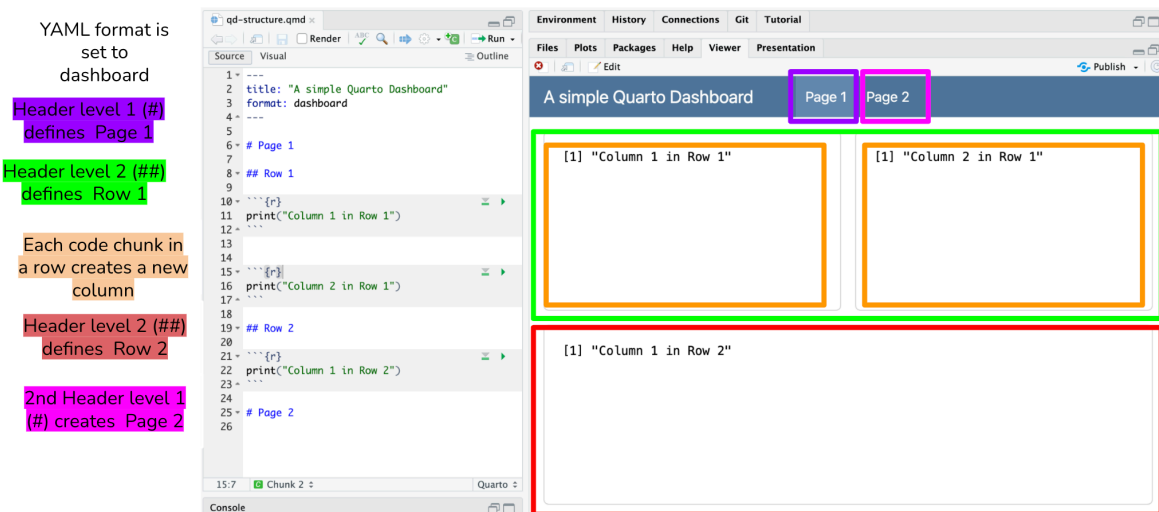


Figure 5.2: Quarto Dashboard Structure

Some other attributes (Table ??) that can be added to the quarto dashboard's rows or columns include

Table 5.1: Some Additional Attributes

Attribute	Explanation
<code>{width=}</code> and <code>{height=}</code>	Set the size of columns, rows, and boxes
<code>{orientation=}</code>	sets the dashboard layout to either rows or columns. This is a global option set in the YAML. However, if your dashboard has multiple pages and you want to specify the orientation for each page, remove <code>orientation:</code> from the YAML and use this attribute instead
<code>{.tabset}</code>	divide columns, rows, or charts into tabs
<code>{.sidebar}</code>	typically creates a sidebar on the page
<code>{.hidden}</code>	excludes a specific page from the navigation bar

5.5 Vizro - Python Dashboard

Built on top of Dash and Plottly, Vizro is a powerful python module to create a dashboard. A vizro dashboard consists of several objects. The first object is **Page**. Each page contains several other sub-objects such as **Comonents**, which can include *Graphs* and *Tables*, **Filters**, which can be *sliders*, *dropdown* boxes and other buttons, and optional **Actions**. To learn more about Vizro, we can explore Vizro [document website](#) and [developer website](#). The key benefits of Vizro include:

- Low-code and Configuration - Vizro only needs a few lines of code code, thus replacing thousand lines of codes and saving valuable time
- In-built Best Practices - Vizro already incorporates standards for visual design and software development.
- Self-service Visualization - Vizro readily assemble dashboards without advanced design or coding experience
- Optional High-code Extensions - Vizro enables limitless customization for advanced users with complex needs
- Modularity - Vizro leverage components that are simple to swap, reuse, maintain, share and scale

- Flexibilit and Scalability - Vizro enables data science ready and to develop python based data visualization applications

```
# Dashboard Creation
from vizro import Vizro
import vizro.models as vm
import vizro.plotly.express as px

# Data Visualization
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# loading dataset
import palmerpenguins
df = palmerpenguins.load_penguins()
```

5.5.1 Standalone Page on Vizro Dashboard (Example # 01)

```
Vizro._reset()

first_page = vm.Page(
    title= " ",
    components= [
        vm.Graph(
            id = 'boxplot',
            figure = px.box (df, x = 'species', y = 'bill_length_mm',
                             color = 'species',
                             labels={'species':'Species', 'bill_length_mm':'Bill Length (mm)'
            ),
        ],
    )

dashboard = vm.Dashboard(pages=[first_page])
Vizro().build(dashboard).run()
```

```
<IPython.lib.display.IFrame at 0x265e43da110>
```

Figure 5.3: A Sample of Vizro Dashboard

5.5.2 Standalone Page on Vizro Dashboard (Example # 02)

```
# loading dataset
df = px.data.gapminder()
gapminder_data = (
    df.groupby(by=["continent", "year"]).
        agg({"lifeExp": "mean", "pop": "sum", "gdpPercap": "mean"}).reset
)

Vizro._reset()
second_page = vm.Page(
    title="First Page",
    components=[
        vm.Card(
            text="""
            # First dashboard page
            This pages shows the inclusion of markdown text in a page and
            can be structured using Layout.
            """,
        ),
        vm.Graph(
            figure=px.box(gapminder_data, x="continent", y="lifeExp", color="continent",
                           labels={"lifeExp": "Life Expectancy", "continent": "Continent"}),
        ),
        vm.Graph(
            figure=px.line(gapminder_data, x="year", y="gdpPercap", color="continent",
                           labels={"year": "Year", "continent": "Continent",
                                   "gdpPercap": "GDP Per Cap"}, title=''),
        ),
    ],
)

dashboard2 = vm.Dashboard(pages=[second_page])
Vizro().build(dashboard2).run()
```

5.5.3 Multiple Pages on Vizro Dashboard

```
Vizro._reset()

third_page = vm.Page(
    title="First Page",
    layout=vm.Layout(grid=[[0, 0], [1, 2], [1, 2], [1, 2]]),
    components=[
        vm.Card(
            text="""
            # First dashboard page
            This pages shows the inclusion of markdown text in a page and
            can be structured using Layout.
            """,
        ),
        vm.Graph(
            figure=px.box(gapminder_data, x="continent", y="lifeExp", color="continent"),
            labels={"lifeExp": "Life Expectancy", "continent": "Continent"},
        ),
        vm.Graph(
            figure=px.line(gapminder_data, x="year", y="gdpPercap", color="continent"),
            labels={"year": "Year", "continent": "Continent", "gdpPercap": "GDP Per Cap"}),
    ],
)

dashboard3 = vm.Dashboard(pages=[third_page])
Vizro().build(dashboard3).run()
```

```
Vizro._reset()

second_page = vm.Page(
    title="Second Page",
    components=[
        vm.Card(
            text="""
            # First dashboard page
            This pages shows the inclusion of markdown text in a page and
            can be structured using Layout.
            """,
        ),
    ],
)
```

```

    ),
    vm.Graph(
        figure=px.box(gapminder_data, x="continent", y="lifeExp", color="continent"),
        labels={"lifeExp": "Life Expectancy", "continent": "Continent"},
    ),
    vm.Graph(
        figure=px.line(gapminder_data, x="year", y="gdpPercap", color="continent"),
        labels={"year": "Year", "continent": "Continent", "gdpPercap": "GDP Per Cap"},
        title='GDP Per Cap',
    ),
],
)

third_page = vm.Page(
    title="Third Page",
    layout=vm.Layout(grid=[[0, 0], [1, 2], [1, 2], [1, 2]]),
    components=[
        vm.Card(
            text="""
            # First dashboard page
            This pages shows the inclusion of markdown text in a page and
            can be structured using Layout.
            """,
        ),
        vm.Graph(
            figure=px.box(gapminder_data, x="continent", y="lifeExp", color="continent"),
            labels={"lifeExp": "Life Expectancy", "continent": "Continent"},
        ),
        vm.Graph(
            figure=px.line(gapminder_data, x="year", y="gdpPercap", color="continent"),
            labels={"year": "Year", "continent": "Continent", "gdpPercap": "GDP Per Cap"},
        ),
    ],
)

dashboard4 = vm.Dashboard(pages=[second_page, third_page])
Vizro().build(dashboard4).run()

```

5.6 Conclusions

5.7 Exercises

1. Create a dashboard from Adidas (Ticker:ADR) sales data (Adidas US Sales Datasets.csv).

6 Data Management in Accounting

Learning Objectives of the Chapter

Learning Objectives

At the End of the Chapter, Students should be Able to -

- Gain an understanding about data management and its importance
- Understand the Concepts of Relational Database
- Develop a database using a Database Management System (DBMS)
- Connect Analytics Platforms Such as R or Python to the database and retrieve data
- Gain an understanding about Online Analytical Processing (OLAP) DBMS
- Create and Use OLAP DBMS

6.1 Data Management

Data management is the practice of collecting, keeping, and using data securely, efficiently, and cost-effectively. Data management is important for a variety of data-driven use cases including end-to-end business process execution, regulatory compliance, accurate analytics and AI, data migration, and digital transformation.

Managing digital data in an organization involves a broad range of tasks, policies, procedures, and practices. The work of data management has a wide scope, covering factors such as how to:

- Create, access, and update data across a diverse data tier
- Store data across multiple clouds and on premises
- Provide high availability and disaster recovery
- Use data in a growing variety of apps, analytics, and algorithms
- Ensure data privacy and security
- Archive and destroy data in accordance with retention schedules and compliance requirements

Data management systems are built on data management platforms and can include databases, data lakes and data warehouses, big data management systems, data analytics, and more.

6.2 Relational Database

Relational databases organize data into rows and columns, which form a table and different tables are connected to each other using either **primary key** or **foreign key**. Here's a simple example of two tables a small business might use to process orders for its products. The first table is a *customer info table*, so each record includes a customer's name, address, shipping and billing information, phone number, and other contact information. Each bit of information (each attribute) is in its own column, and the database assigns a unique ID (a key) to each row. In the second table—a *customer order table*—each record includes the ID of the customer that placed the order, the product ordered, the quantity, the selected size and color, and so on—but not the customer's name or contact information.

These two tables have only one thing in common: the ID column (the key). But because of that common column, the relational database can create a relationship between the two tables. Then, when the company's order processing application submits an order to the database, the database can go to the customer order table, pull the correct information about the product order, and use the customer ID from that table to look up the customer's billing and shipping information in the customer info table. The warehouse can then pull the correct product, the customer can receive timely delivery of the order, and the company can get paid.

6.3 Relational Database Management Systems (RDBMS)

While a relational database organizes data based off a relational data model, a relational database management system (RDBMS) is a more specific reference to the underlying database software that enables users to maintain it. These programs allow users to create, update, insert, or delete data in the system, and they provide:

- Data structure
- Multi-user access
- Privilege control
- Network access

Examples of popular RDBMS systems include MySQL, PostgreSQL, and IBM DB2. Additionally, a relational database system differs from a basic database management system (DBMS) in that it stores data in tables while a DBMS stores information as files.

6.4 Creating a Relational Database Using PostgreSQL

PostgreSQL is a widely used open-source Relational Database Management System (RDBMS). Since its release in 1996, PostgreSQL has carved a niche as one of the most robust and reliable database systems in the Tech industry. PostgreSQL is known for its rich feature set, reliability, scalability, and strong community support network.

6.4.1 Installing PostgreSQL

To install PostgreSQL locally on your computer, visit the [installer by EDB](#), and download the newest version compatible with your operating system. To install PostgreSQL on Windows, you need to have administrator privileges. Below, we delineate the steps to install the PostgreSQL in windows.

Step 01: When the downloading is complete, double click the downloaded file and an installation wizard will appear and guide you through multiple steps. Click “Next” to specify directory.

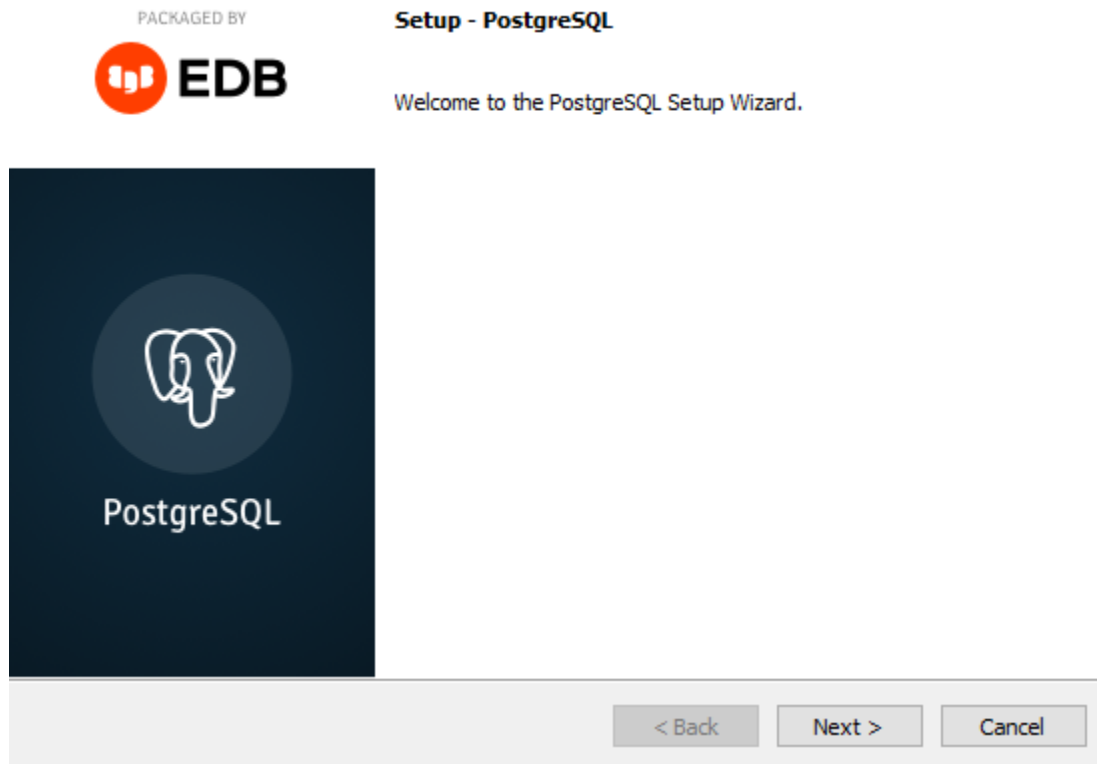


Figure 6.1: Installation Wizard

Step 02: You can specify the location of PostgreSQL, or go with the default choice

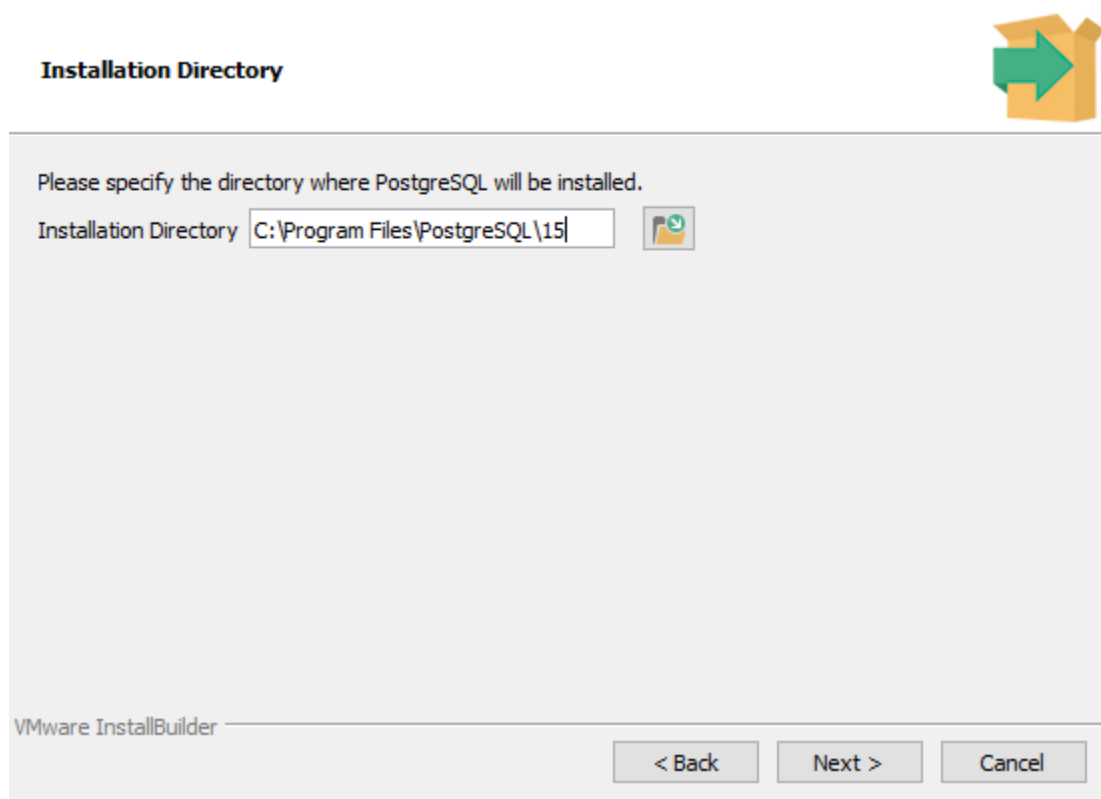


Figure 6.2: Installation Directory

Step 03: To use PostgreSQL, you need to install PostgreSQL server. For our installation and uses purposes, we select the following servers - PostgreSQL Server, pgAdmin4, and Command Line Tools.

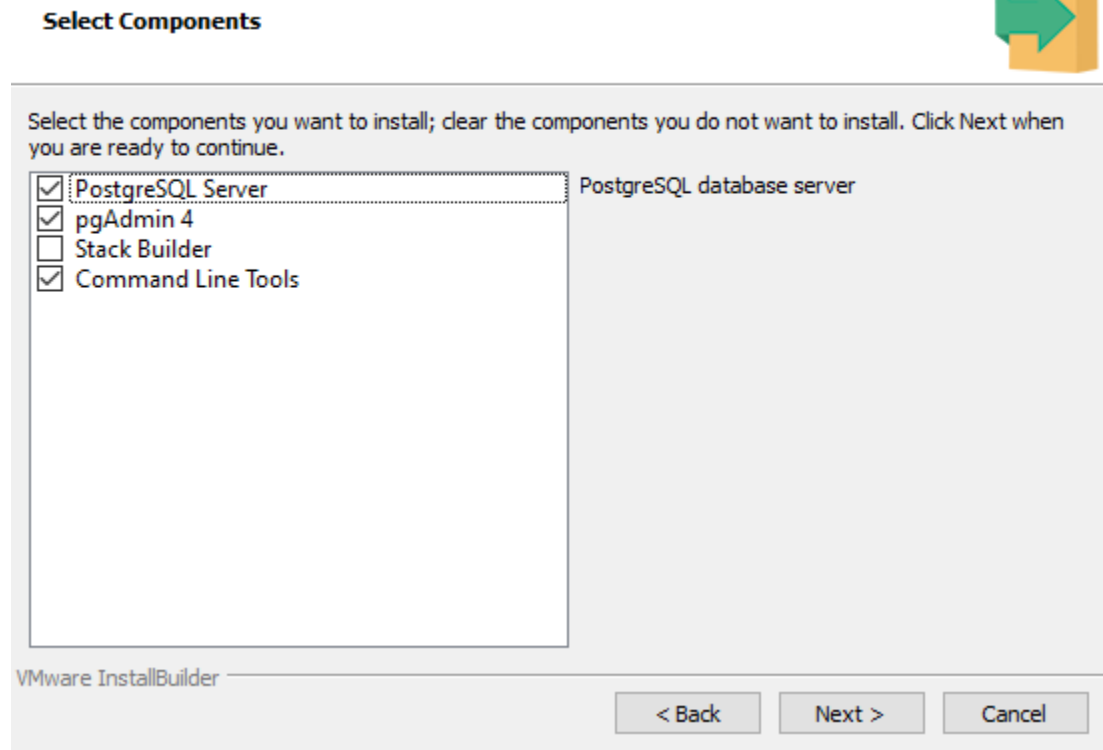


Figure 6.3: Server Selection

Step 04: Then, we need to choose the storage directory. Alternatively, we can go with the default directory.

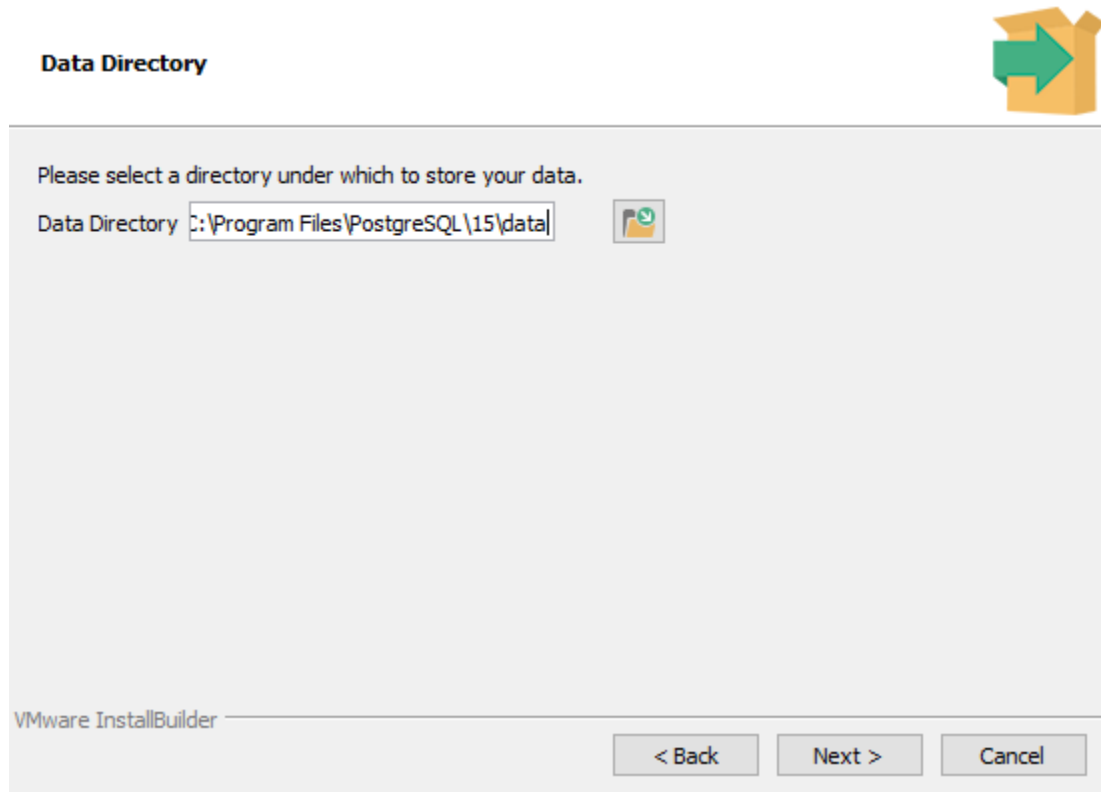
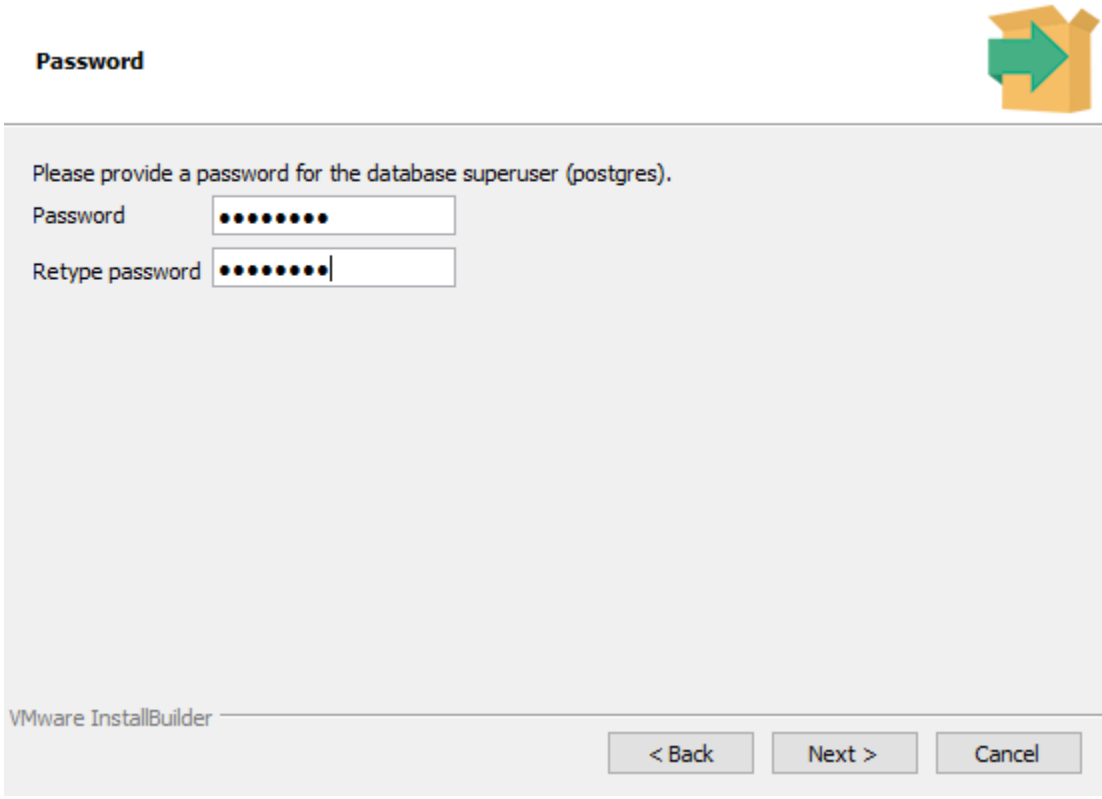


Figure 6.4: Storage Directory

Step 05: At this point, you need to choose your password. It is important to note that the password will be used to connect to the database. Therefore, you must remember your password.



The image shows a 'Password' dialog box from VMware InstallBuilder. It has a title bar with the word 'Password' and a yellow box icon with a green arrow. The main area contains the text 'Please provide a password for the database superuser (postgres).' followed by two input fields: 'Password' and 'Retype password', both filled with black dots. At the bottom, there is a footer with 'VMware InstallBuilder' on the left and three buttons: '< Back', 'Next >', and 'Cancel'.

Figure 6.5: Password Selection

Step 06: Next you need to select the port. The default port number is 5432. If you change the port, you also need to remember the port number because like passowrd, you need to use the port number to connect to the database.

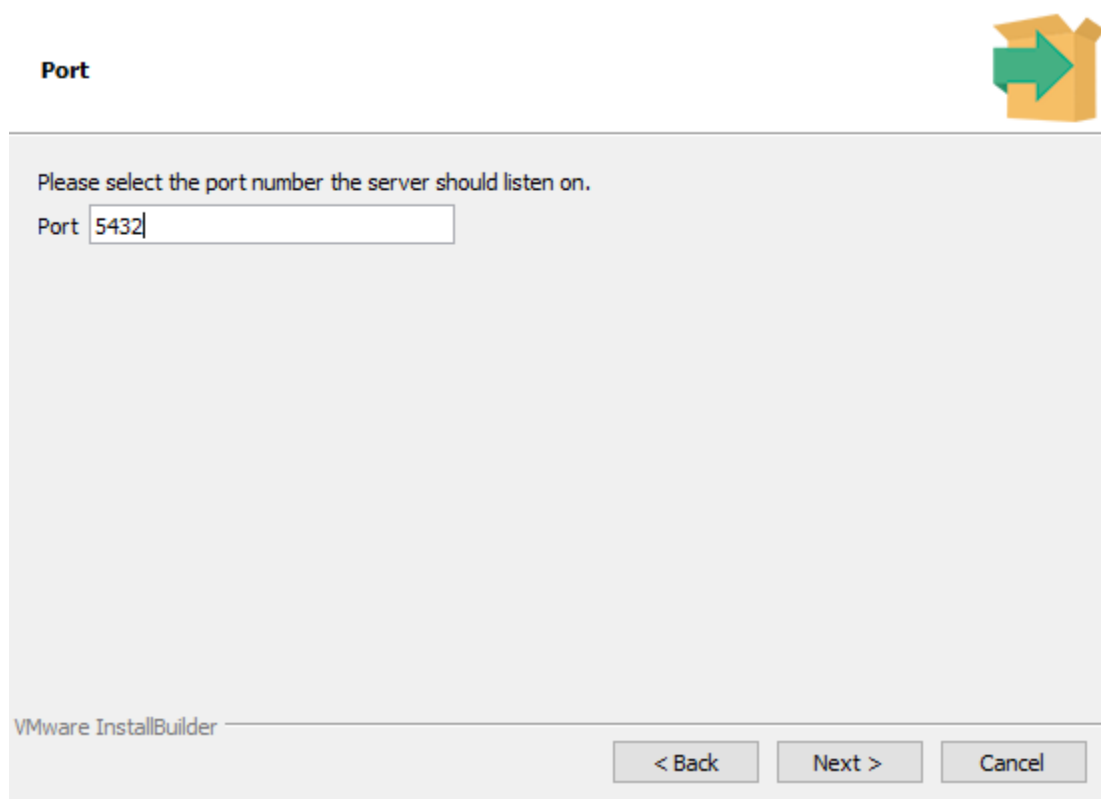


Figure 6.6: Port Selection

Step 07: We also need to select the geographical location of the database server. We can go with the default choice.

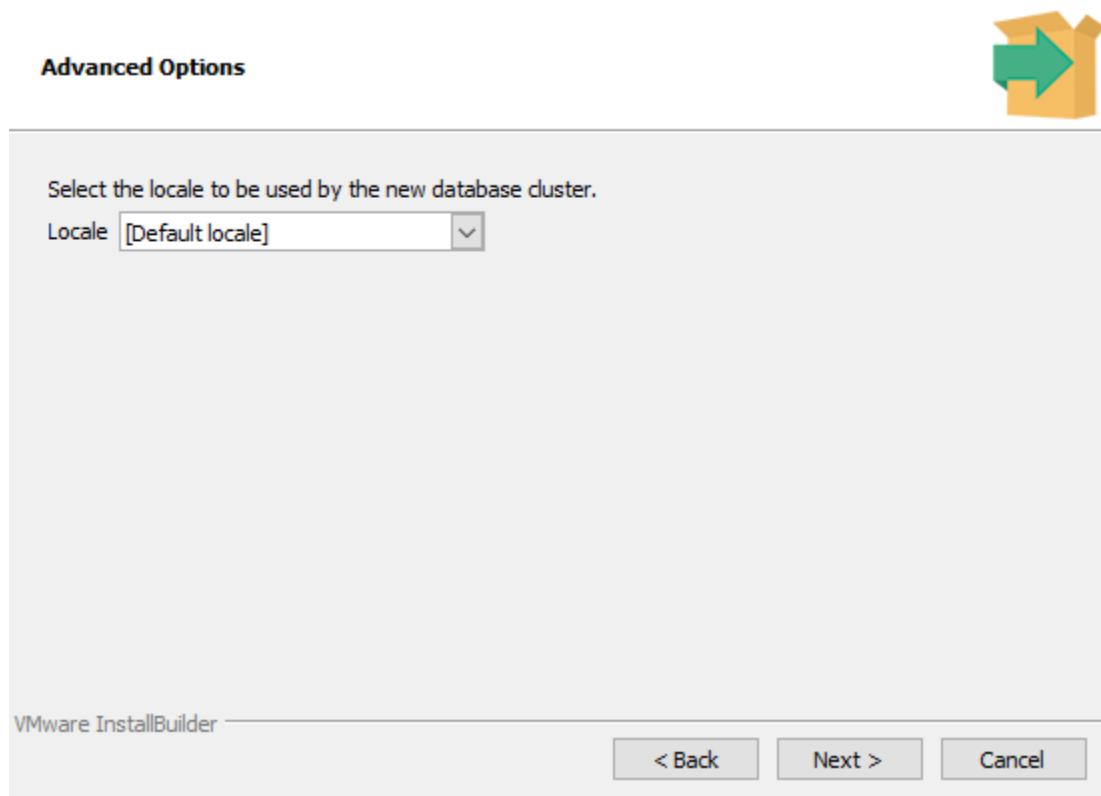


Figure 6.7: Locale Selection

Then if everything looks ok, you can move forward by clicking 'Next'. However, it is recommended that you copy and save the information from the "Pre Installation Summary" in some safe place so that you can access them in future, if necessary.

Pre Installation Summary

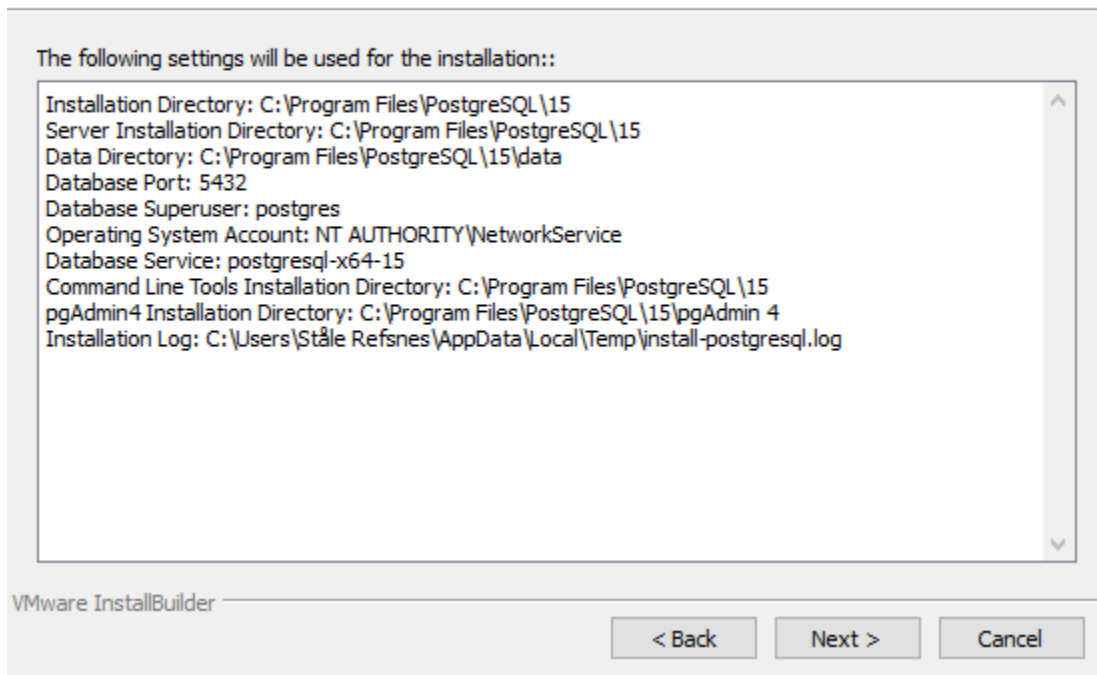


Figure 6.8: Pre Installation Summary

Step 08: Then you can click “Next” in ‘Ready to Install’ windows and the installation process will start and it will take a while to finish the installation. Once the installing is done, please click ‘Finish’ button on ‘Complete the PostgreSQL Setup Wizard’ window.

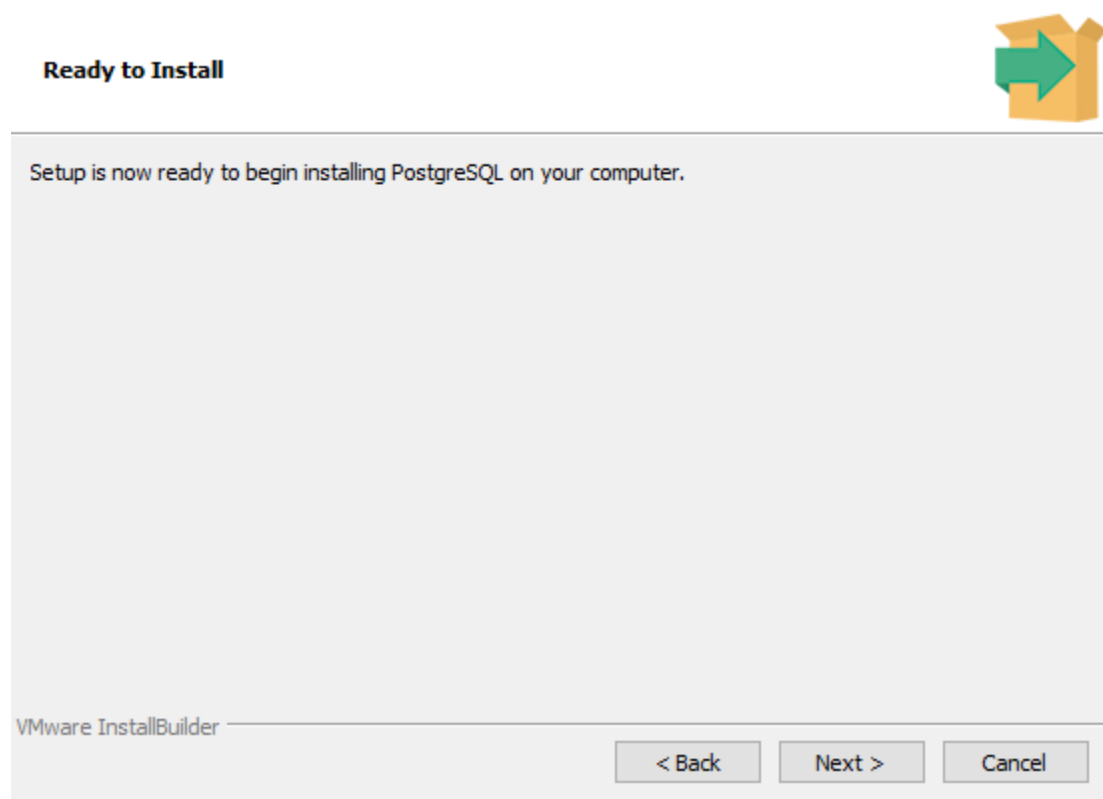


Figure 6.9: Ready to Install

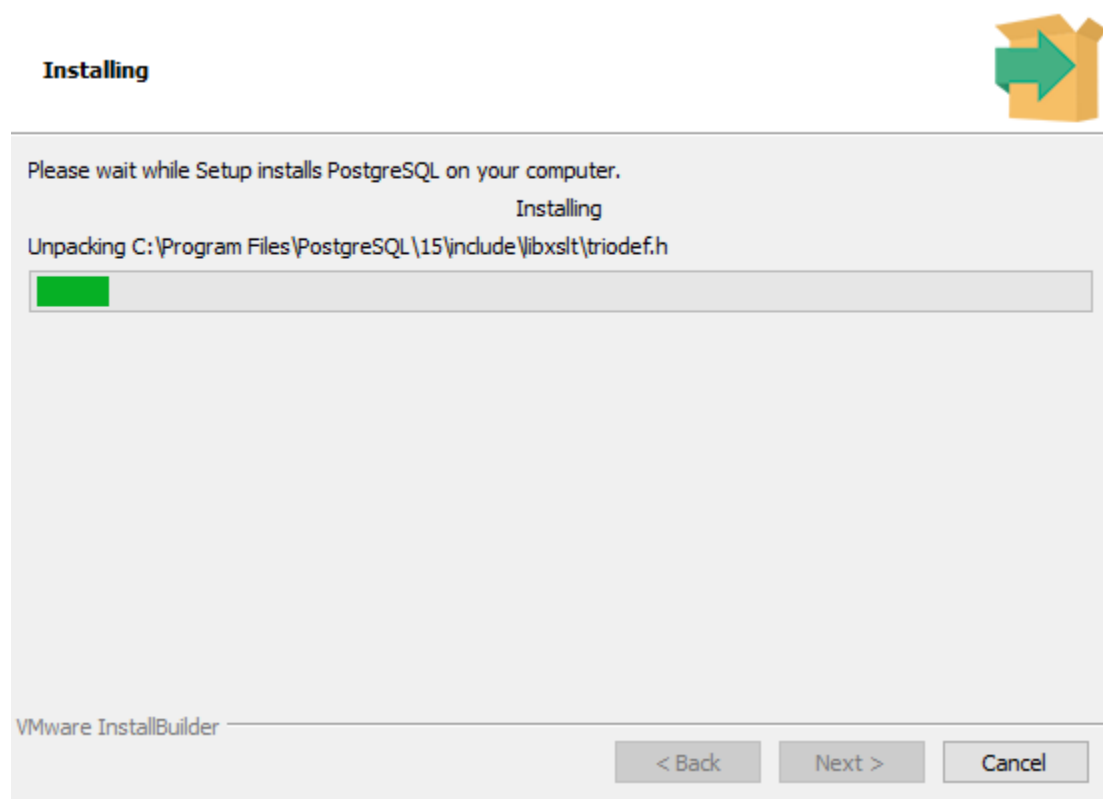


Figure 6.10: Installing

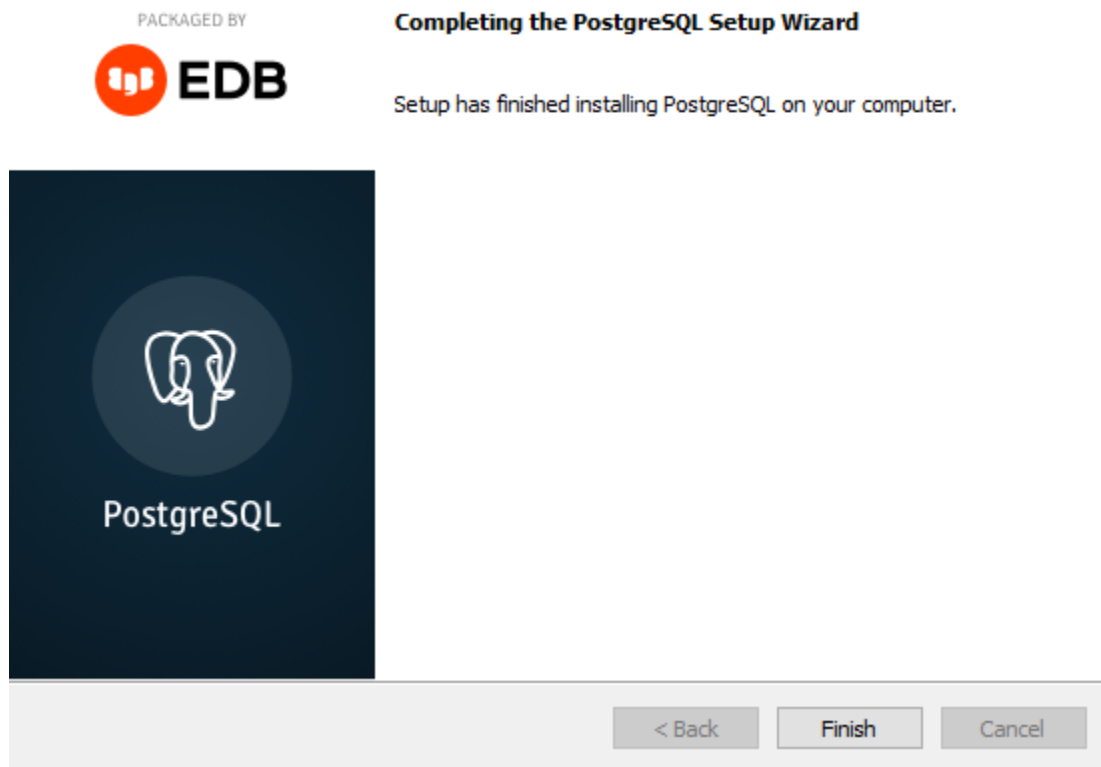


Figure 6.11: Complete the PostgreSQL Setup Wizard

 Tip

Some good sources to learn more about PostgreSQL -

1. [W3 Schools](#)
2. [PostgreSQL Tutorial](#)

6.5 Connecting to the Relational Database

In Section ??, we described the steps of installing PostgreSQL database; now, we will try to connect to that database. There are several ways to connect the database we created. These ways include -

- SQL Shell (psql)
- Command Shell (cmd)
- pgAdmin 4

Tip

To connect to PostgreSQL Database using pgAdmin 4, please use the following link -

1. [Connecting using pgAdmin 4](#)

6.5.1 Connection using SQL Shell (psql)

First, we will discuss SQL Shell (psql) method. Basically, SQL Shell (psql) is a terminal based program where you can write and execute SQL syntax in the command-line terminal. You will find the SQL Shell (psql) in the start menu under PostgreSQL. If you do not find it please search “psql” on windows search box.

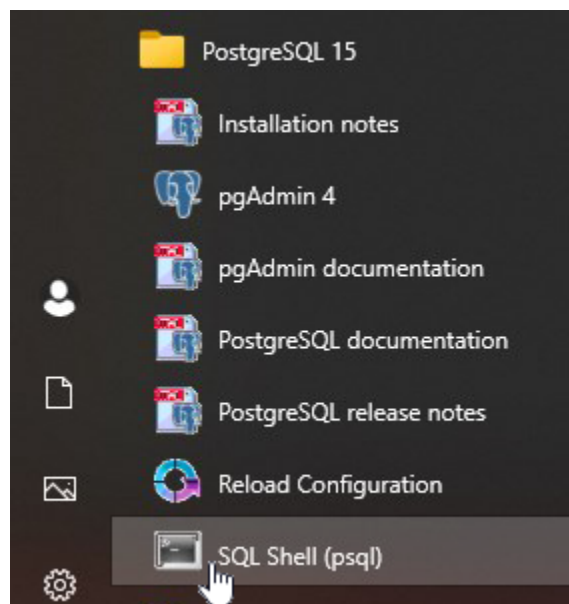


Figure 6.12: SQL Shell on Start Menu

Once the program is open, you should be able to see a window like below. Usually, the name of the server is localhost as shown below; however, if you choose a different server while installing the PostgreSQL, you must mention your server name. Then, click “enter” on your keyboard.

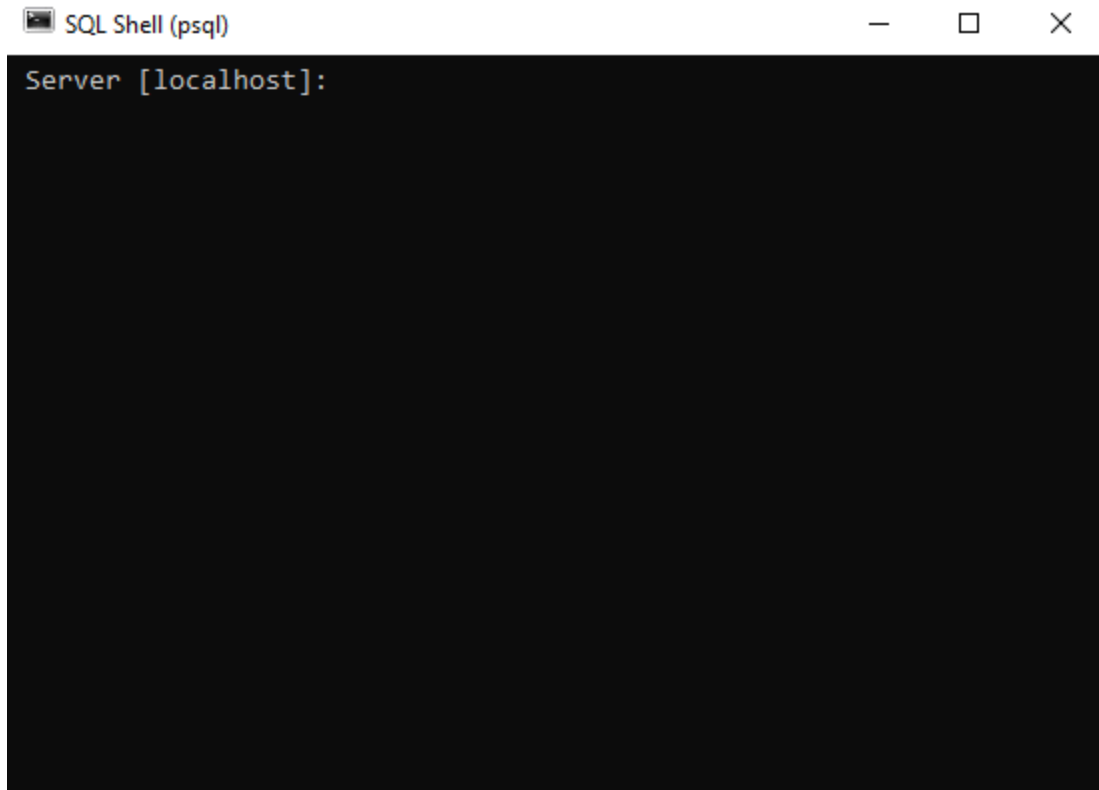


Figure 6.13: Server

The default name of your database is `postgres` as shown below. If you choose a different name, please enter that name. Then, click “enter” on your keyboard.

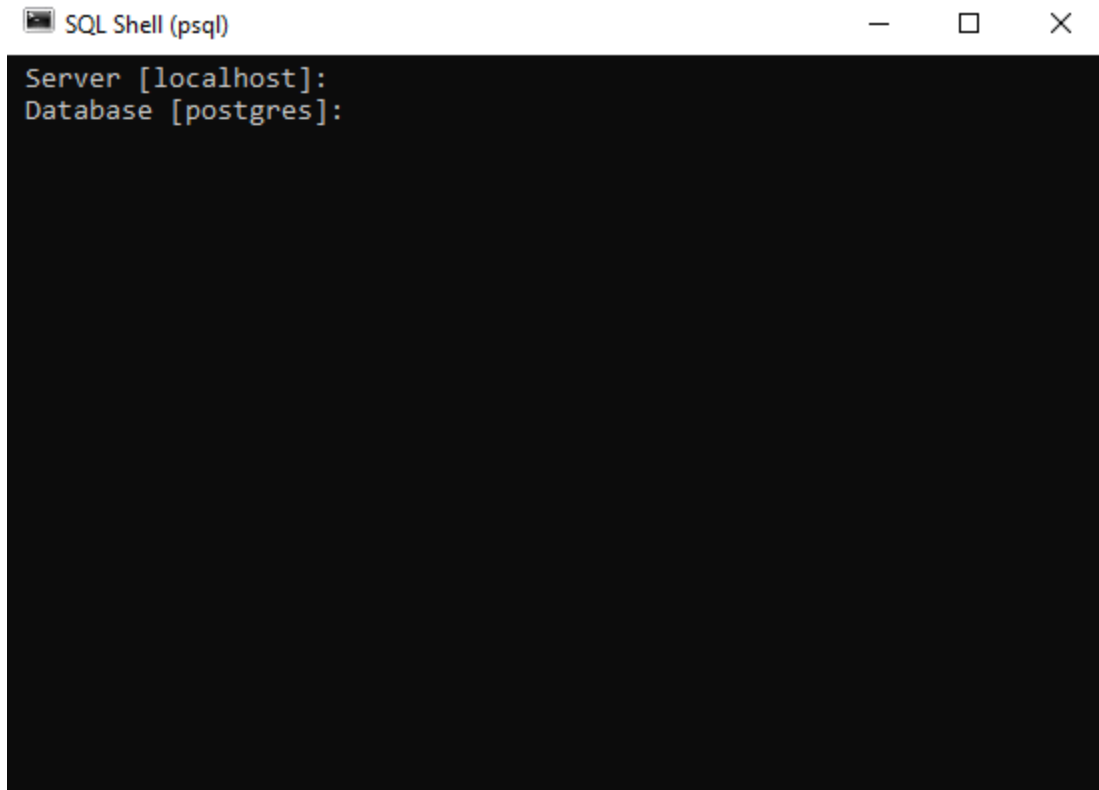


Figure 6.14: Database

The default port number is 5432. If you use the default port number, then please click “enter” on your keyboard. Otherwise, use the port number that you used while installing PostgreSQL and click “enter” on your keyboard.

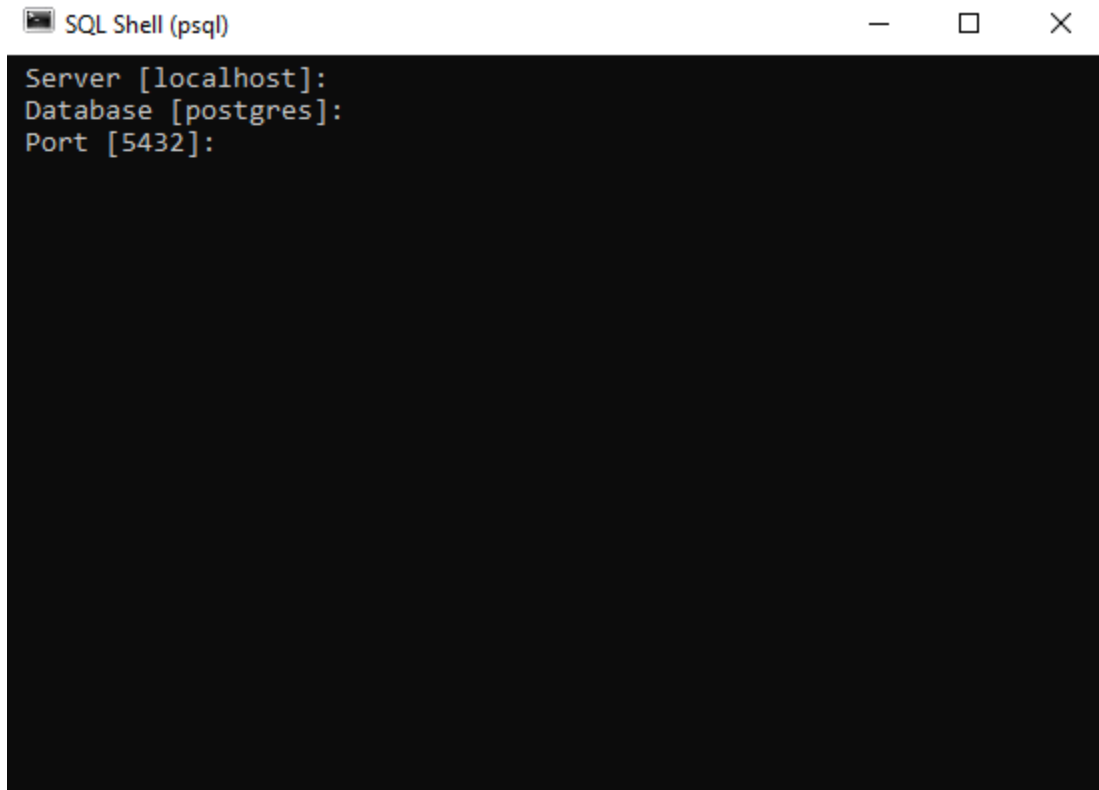


Figure 6.15: Port

Similarly, the default user name is `postgres`. Please click “enter” on your keyboard if your default name is the same.

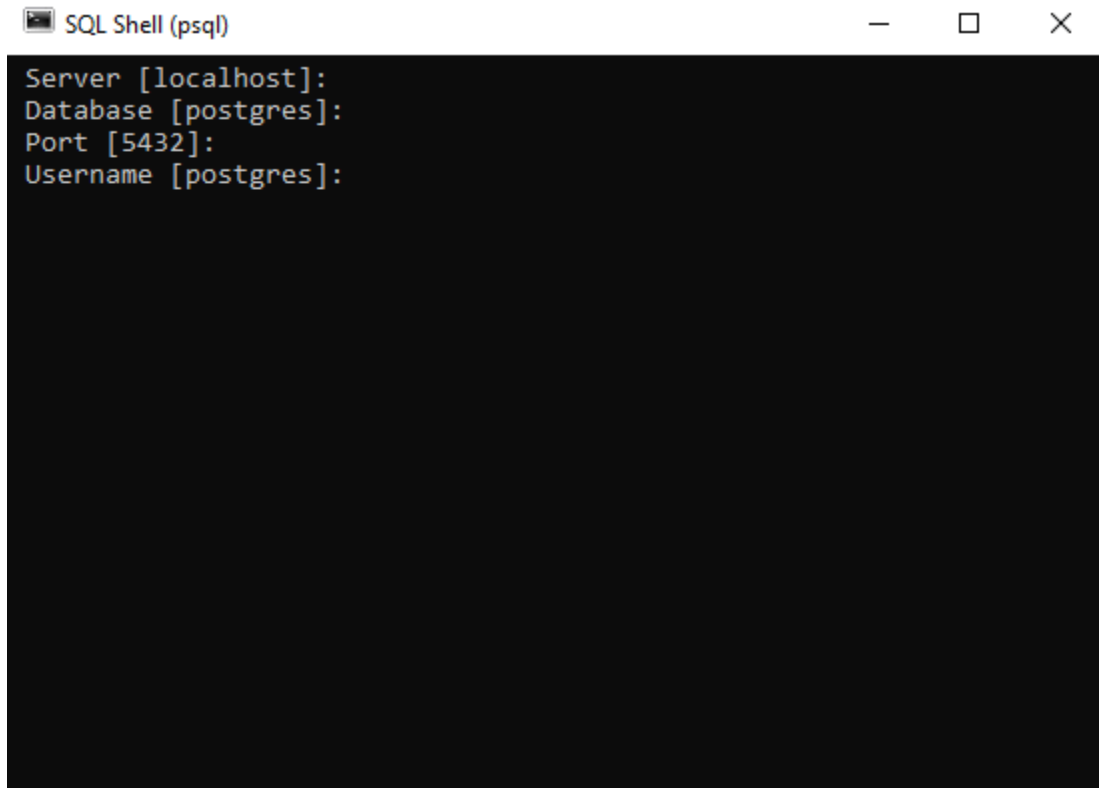


Figure 6.16: Username

Next, you need to provide password that you set while installing the database.

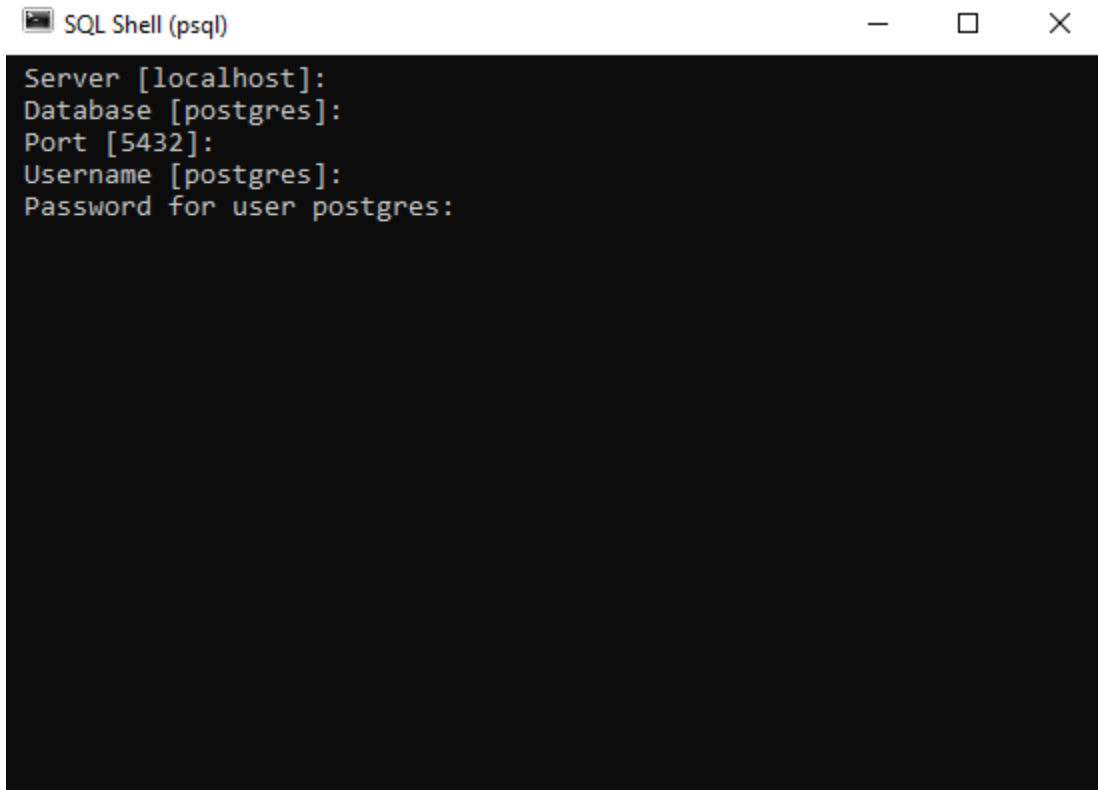


Figure 6.17: Password

6.5.2 Connection using Command Shell (cmd)

Using terminal on Windows and Mac, we can connect to the database. On Windows, we use command prompt (cmd) and on Mac we use Terminal App to use terminal.

6.6 Windows

To use the command prompt (cmd) on Windows to connect to PostgreSQL database, you need to use the following steps -

- Step # 01 - Open command prompt by writing cmd in windows search bar as shown below. Then command prompt will open.